Regulators

- The regulators of these networks depends on the nature of process,
- A regulator can be;
  - A gene
  - DNA
  - RNA
  - Protein
- OR complexes on any of these correspondently

Upregulating and Down regulating genes

- When the regulators tend to be genes and they enhance or suppress the working of a correspondent particular gene they are said to upregulating or down regulating respectively.

Lesson-56

Database for Annotation, Visualization and Integrated Discovery (DAVID)

DAVID

- Free Bioinformatics Resource Software.
- Aims to provide functional interpretation of large datasets of genes derived from different genomic studies such as microarray.
- David has embedded five web-based functional annotation tool suits:
  - DAVID gene functional classification Tool
  - DAVID functional annotation Tool
  - DAVID gene ID conversion Tool
  - DAVID gene name viewer
  - DAVID NIAID Pathogen Genome Browser.



What DAVID does?

- Classifies large gene lists into functionally related groups
- Ranks the importance of the functional gene groups.
- Summarizes the major biology of gene groups discovered.
- Searches other functionally related genes that are not in the gene list but exist in the genome.
- Visualizes the annotations.

Significance

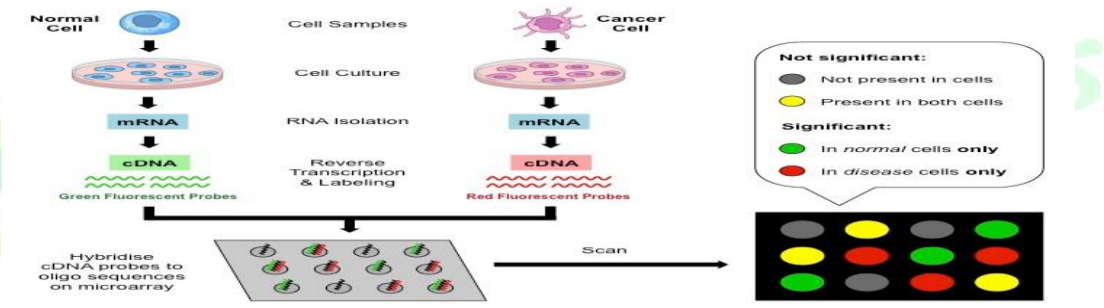
- Genes are highly organized so that they are readable and understandable.
- Ranking the genes help a quick focus on only the significant ones.
- Gene's annotations are displayed in a single view, easy for comparison
- The studies are extended by the digging up of functionally related genes.

Lesson-57

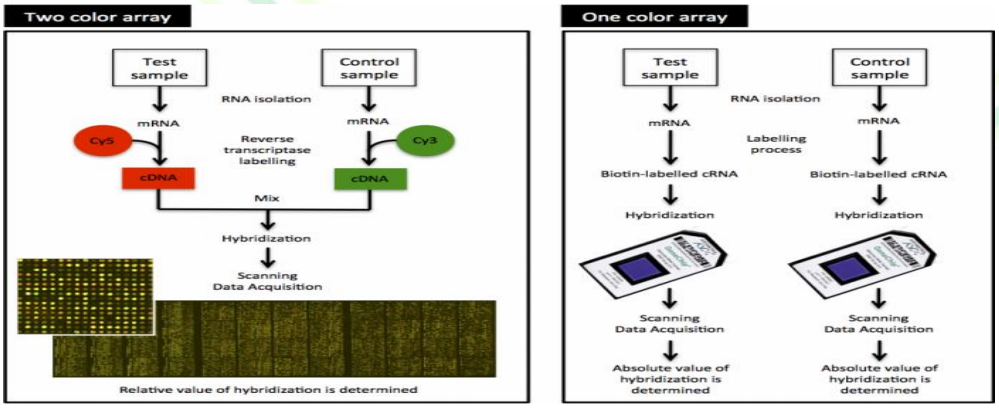
Microarray Data

Microarray

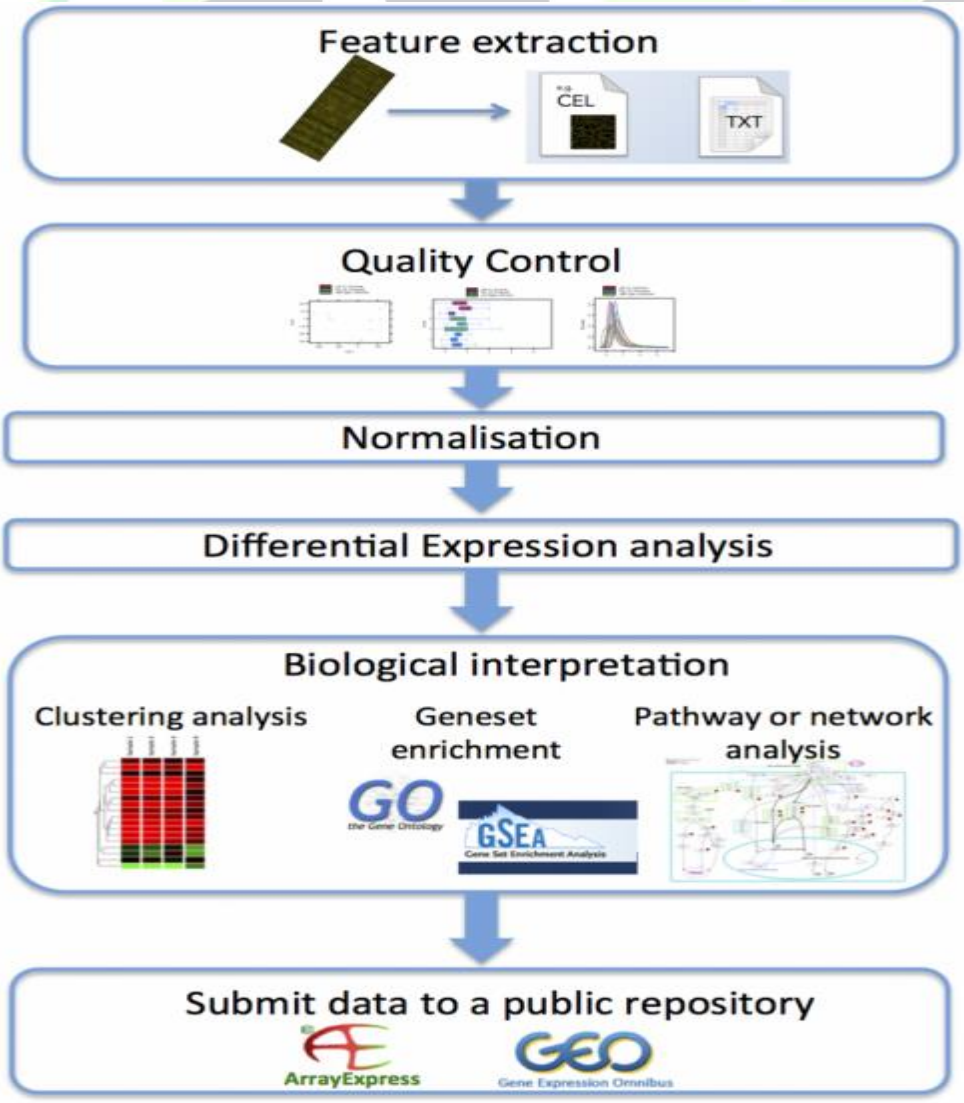
- A collection of microscopic DNA spots attached to a solid surface.
- Used to measure the expression levels of large numbers of genes simultaneously or to genotype multiple regions of a genome



One colour or Two colour arrays



- Microarrays can be used in many types of experiments including
  - genotyping,
  - epigenetics,
  - translation profiling and
  - gene expression profiling
- The process of analysing gene expression data in microarrays involves
  - feature extraction;
  - quality control;
  - normalisation;
  - differential expression analysis;
  - biological interpretation of the results;
  - submission of data to a public database.



Lesson-58

Challenges in Interpreting Gene Microarray Data

Microarray Data

- This technique makes it possible to examine the expression of thousands of genes simultaneously.
- This technology promises to lead to improvements in developing rational approaches to therapy as well as to improvements in cancer diagnosis and prognosis.
- Yet comes along with a lot of challenges to be dealt with its data.

Limitations

- Data variability, especially for genes with low expression levels
- Small sample amounts which limits replication
- Unequal labelling efficiency of flouresecnt dyes.
- Provides no information regarding protein expression levels and function.

Solutions

- Replication to reduce false positives.
- Use of amplified RNA (aRNA)
- Reciprocal labelling to confirm observations.
- Confirm with other biochemical analysis for protein expression (e.g. immunohistochemistry protein arrays)

Lesson-59

Biological Interpretation of Microarray data

sign of bright future

Interpreting the Microarray Data

- Many of the methods for visualizing and interpreting gene expression data can be used for microarray some of them are discussed here

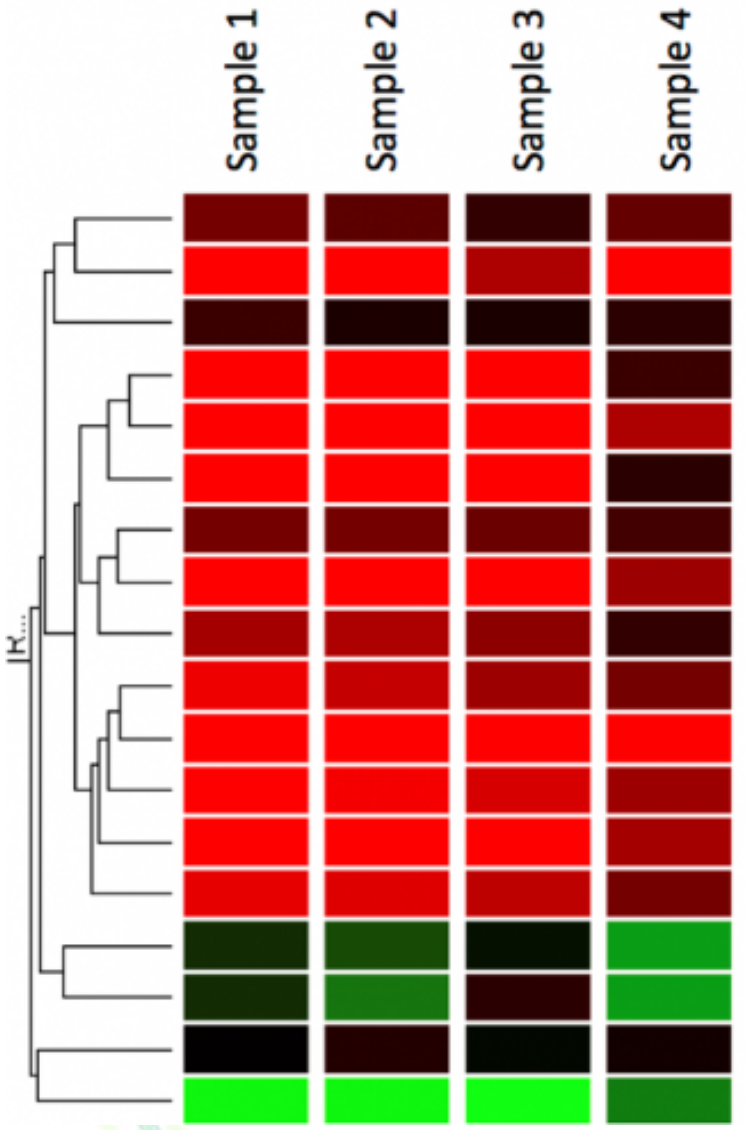
HeatMaps

- A common method of visualising gene expression data is to display it as a heatmap.
- The heatmap may also be combined with clustering methods which group genes and/or samples together based on the similarity of their gene expression pattern.
- This can be useful for identifying genes that are commonly regulated, or biological signatures associated with a particular condition (e.g a disease or an environmental condition)

HeatMaps-Example

- The data is displayed in a grid where each row represents a gene and each column represents a sample.

- The colour and intensity of the boxes is used to represent changes (not absolute values) of gene expression.
- In the example, red represents up-regulated genes and green represents down-regulated genes. Black represents unchanged expression.



Gene set enrichment analysis

- A common approach to interpreting gene expression data is gene set enrichment analysis based on the functional annotation of the differentially expressed genes.
- The Gene Ontology, containing standardised annotation of gene products, is commonly used for this purpose. It works by comparing the frequency of individual annotations in the gene list (e.g differentially expressed genes) with a reference list

Popular tools for gene set enrichment

- DAVID (Free online Tool)
- GSEA (Free)
- Ingenuity (Licensed)

- Reactome (Free)

Network analysis

- Network analysis is complementary to pathway analysis and can be used to show how key components of different pathways interact.
- This can be useful for identifying regulatory events that influence multiple biological processes and pathways

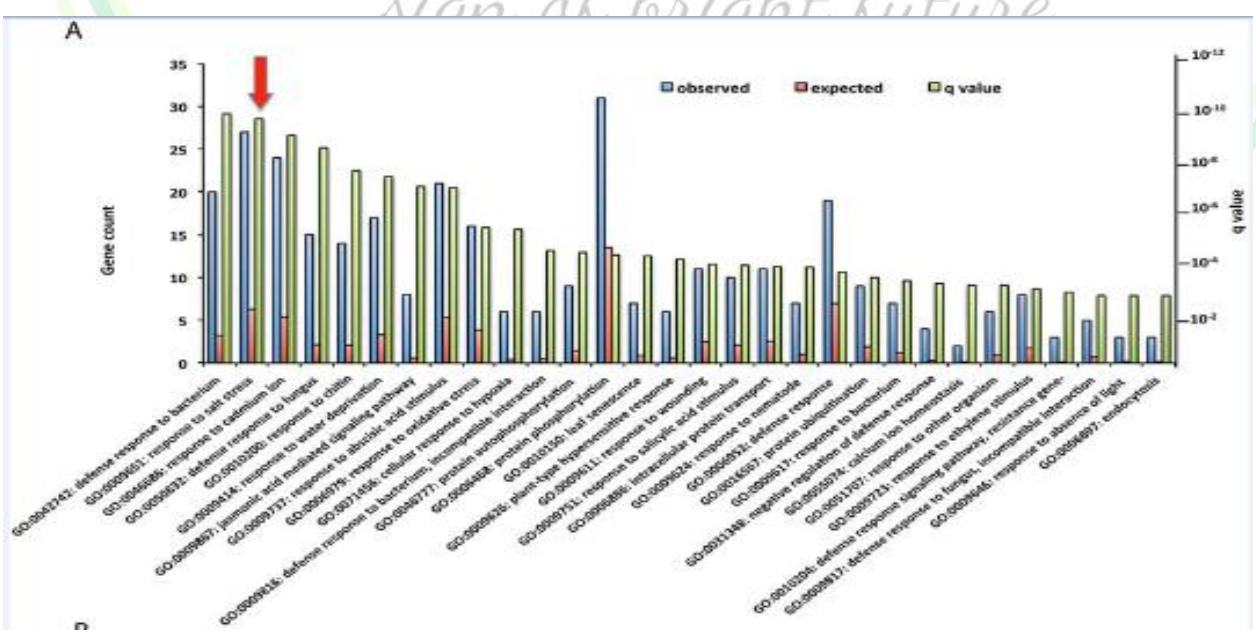
Lesson-60

GO Term and Functional Analysis

GO Term and Functional Analysis

- Gene Ontology (GO) term enrichment is a technique for interpreting sets of genes making use of the Gene Ontology system of classification, in which genes are assigned to a set of predefined bins depending on their functional characteristics
- The Gene Ontology (GO) provides a system for hierarchically classifying genes or gene products to terms organized in a graph structure called an ontology.
- The GO terms are grouped into three categories:
    - molecular function (describing the molecular activity of a gene),
    - biological process (describing the larger cellular or physiological role carried out by the gene, coordinated with other genes)
    - and cellular component (describing the location in the cell where the gene product executes its function). Each gene can be described (annotated with) multiple terms.
- Using the GO it is possible to retrieve the set of terms used to describe any gene, or conversely, given a term, return the set of genes annotated to that term.

Visualization



Lesson-61

Functional Profiling Tools in DAVID

DAVID

- David has embedded five web-based functional annotation tool suits:
  - DAVID gene functional classification Tool
  - DAVID functional annotation Tool
  - DAVID gene ID conversion Tool
  - DAVID gene name viewer
  - DAVID NIAID Pathogen Genome Browser.

DAVID functional annotation Tool

- DAVID functional annotation Tool, mainly provides typical batch annotation and gene-GO term enrichment analysis to highlight the most relevant GO terms associated with a given gene list.
- This function uses a novel algorithm to measure relationships among the annotation terms based on the degrees of their co-association genes to group the similar, redundant, and heterogeneous annotation contents from the same or different resources into annotation groups



Lesson-62

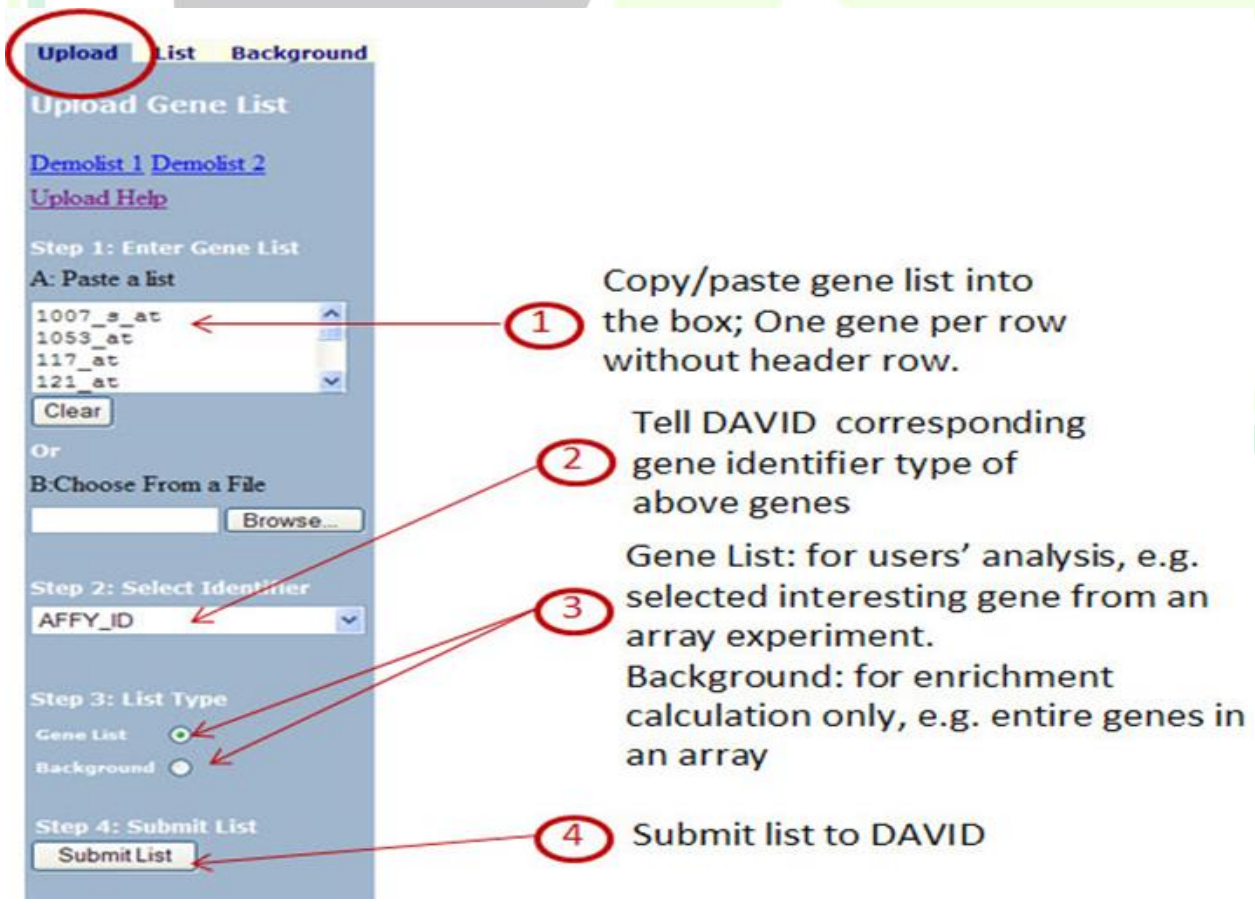
Functional Annotation Analysis Manual

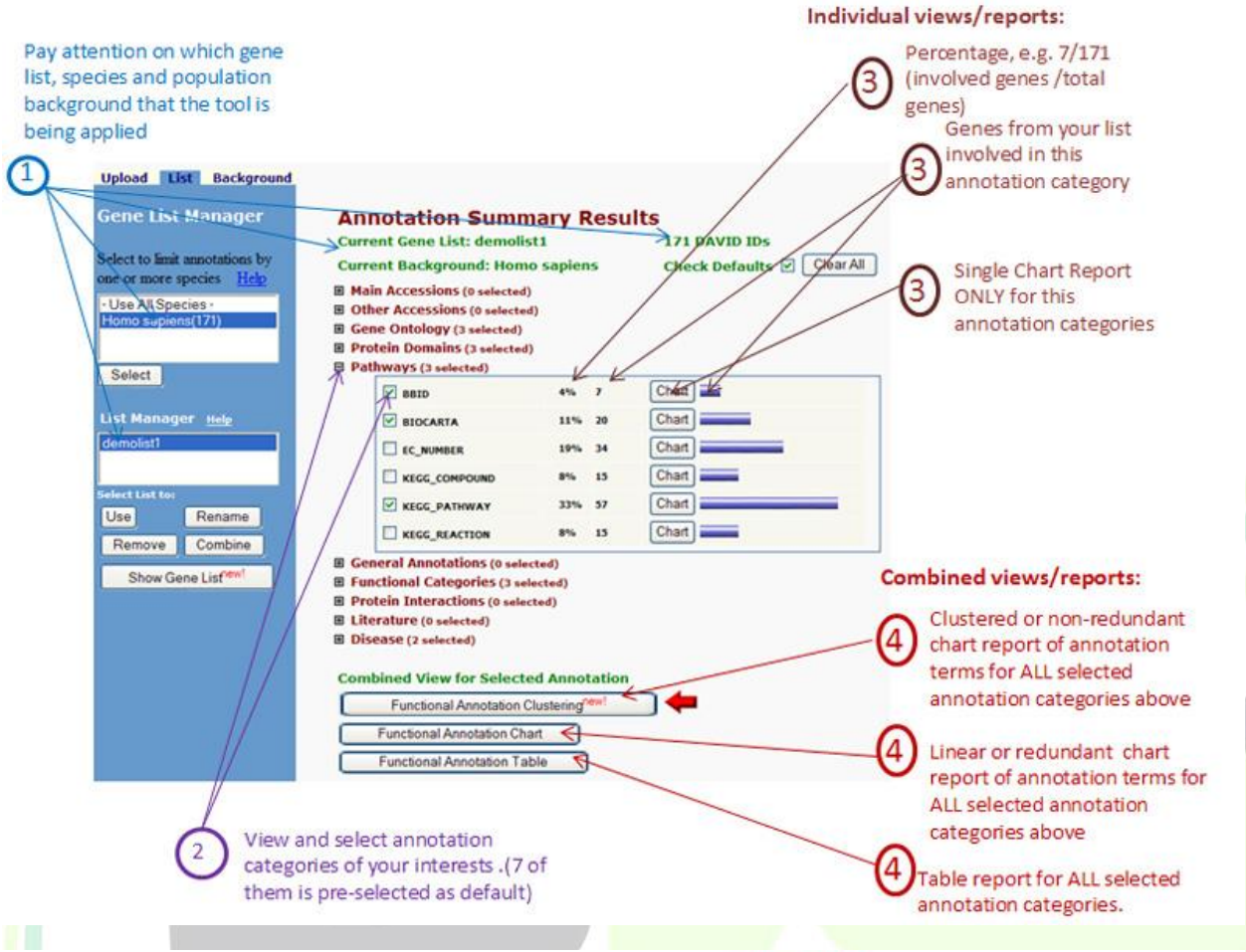
Functional Annotation Analysis

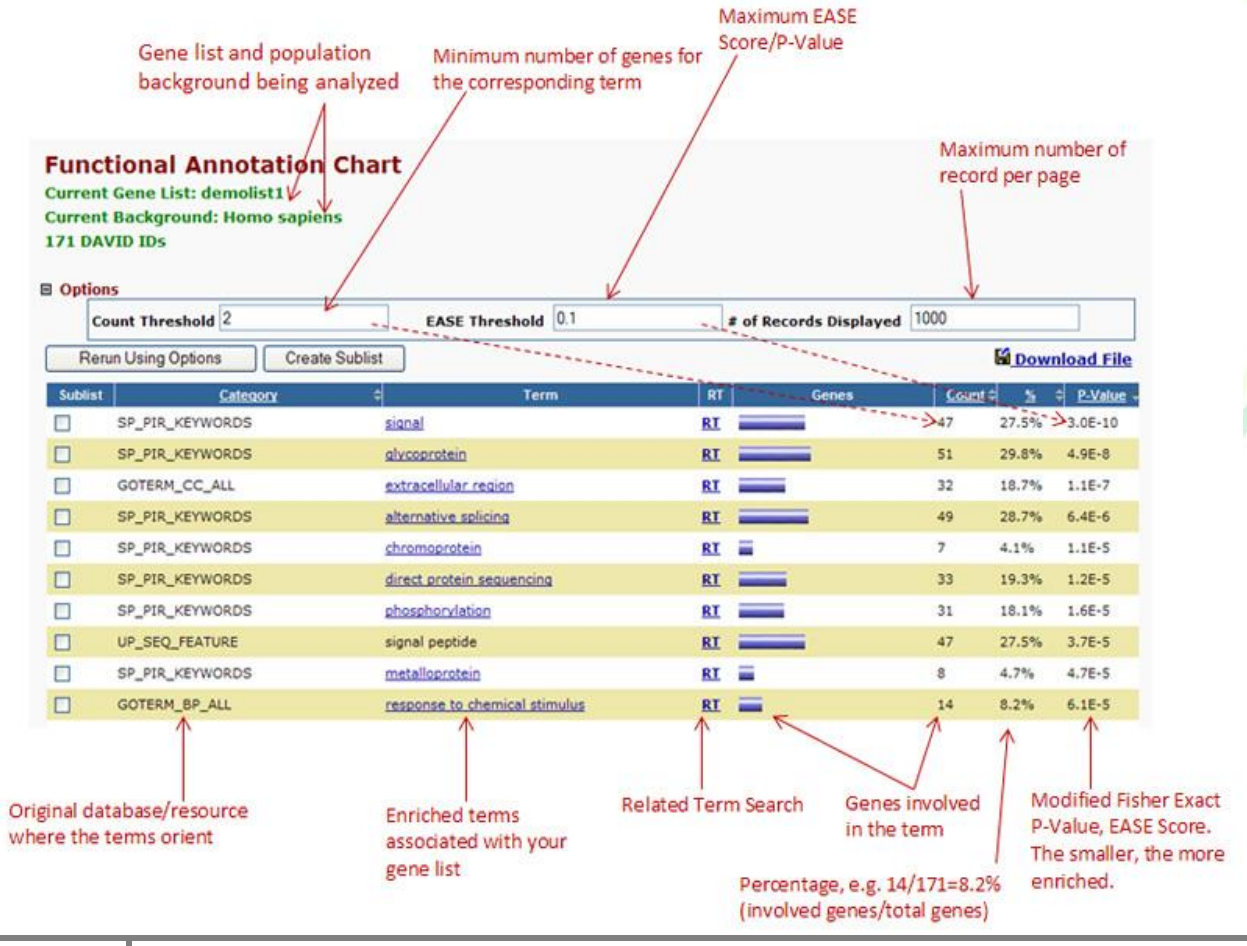


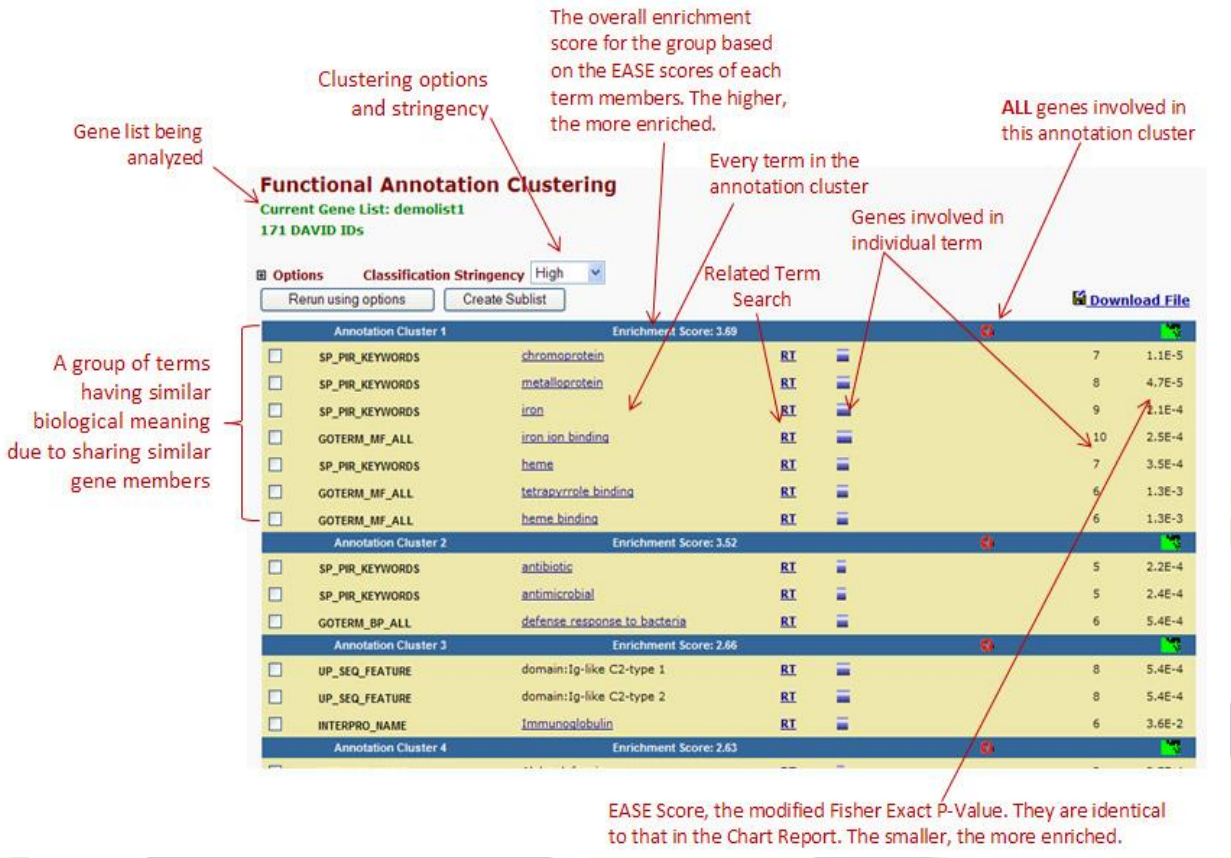
- **Workflow:**

Load Gene List -> View Summary Page -> Explore details through Chart Report, Table Report, Clustering Report, etc. -> Export and Save Results




Individual views/reports:
Pay attention on which gene list, species and population background that the tool is being applied
Percentage, e.g. 7/171 (involved genes /total genes)
Genes from your list involved in this annotation category
Single Chart Report ONLY for this annotation categories
Upload
List
Background
Gene List Manager
Select to limit annotations by one or more species Help
- Use All Species -
Homo sapiens(171)
Select
List Manager Help
demolist1
Select List to:
Use
Rename
Remove
Combine
Show Gene List new!
Annotation Summary Results
Current Gene List: demolist1
171 DAVID IDs
Current Background: Homo sapiens
Check Defaults
Clear All
Main Accessions (0 selected)
Other Accessions (0 selected)
Gene Ontology (3 selected)
Protein Domains (3 selected)
Pathways (3 selected)
BBID 4% 7 Chart
BIOCARTA 11% 20 Chart
EC_NUMBER 19% 34 Chart
KEGG_COMPOUND 8% 15 Chart
KEGG_PATHWAY 33% 57 Chart
KEGG_REACTION 8% 15 Chart
General Annotations (0 selected)
Functional Categories (3 selected)
Protein Interactions (0 selected)
Literature (0 selected)
Disease (2 selected)
Combined View for Selected Annotation
Functional Annotation Clustering new!
Functional Annotation Chart
Functional Annotation Table
Combined views/reports:
Clustered or non-redundant chart report of annotation terms for ALL selected annotation categories above
Linear or redundant chart report of annotation terms for ALL selected annotation categories above
Table report for ALL selected annotation categories.
View and select annotation categories of your interests .(7 of them is pre-selected as default)

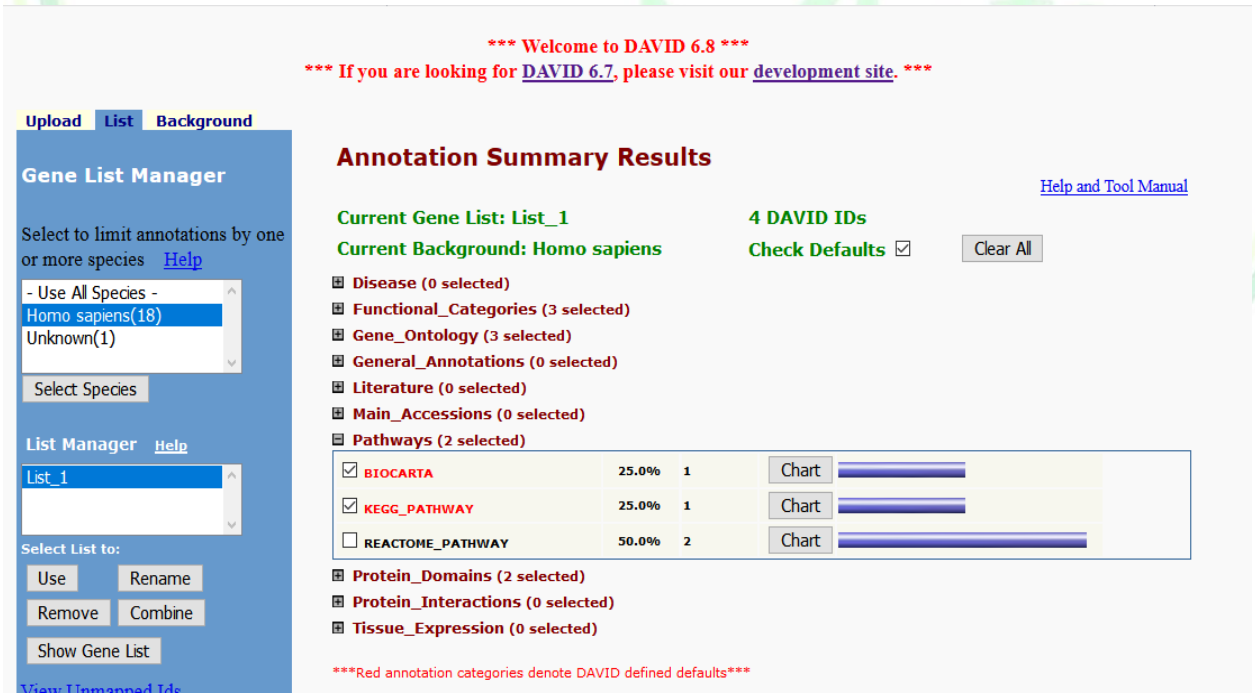

Gene list and population background being analyzed
Minimum number of genes for the corresponding term
Maximum EASE Score/P-Value
Maximum number of record per page
Functional Annotation Chart
Current Gene List: demolist1
Current Background: Homo sapiens
171 DAVID IDs
Options
Count Threshold 2
EASE Threshold 0.1
of Records Displayed 1000
Rerun Using Options
Create Sublist
Download File
Sublist | Category | Term | RT | Genes | Count | % | P-Value
SP_PIR_KEYWORDS | signal | RT | 47 | 27.5% | 3.0E-10
SP_PIR_KEYWORDS | glycoprotein | RT | 51 | 29.8% | 4.9E-8
GOTERM_CC_ALL | extracellular region | RT | 32 | 18.7% | 1.1E-7
SP_PIR_KEYWORDS | alternative splicing | RT | 49 | 28.7% | 6.4E-6
SP_PIR_KEYWORDS | chromoprotein | RT | 7 | 4.1% | 1.1E-5
SP_PIR_KEYWORDS | direct protein sequencing | RT | 33 | 19.3% | 1.2E-5
SP_PIR_KEYWORDS | phosphorylation | RT | 31 | 18.1% | 1.6E-5
UP_SEQ_FEATURE | signal peptide | RT | 47 | 27.5% | 3.7E-5
SP_PIR_KEYWORDS | metalloprotein | RT | 8 | 4.7% | 4.7E-5
GOTERM_BP_ALL | response to chemical stimulus | RT | 14 | 8.2% | 6.1E-5
Original database/resource where the terms orient
Enriched terms associated with your gene list
Related Term Search
Genes involved in the term
Percentage, e.g. 14/171=8.2% (involved genes/total genes)
Modified Fisher Exact P-Value, EASE Score. The smaller, the more enriched.



Lesson-63

KEGG Pathway

KEGG Pathway



*** Welcome to DAVID 6.8 ***
*** If you are looking for DAVID 6.7, please visit our development site. ***

Functional Annotation Table

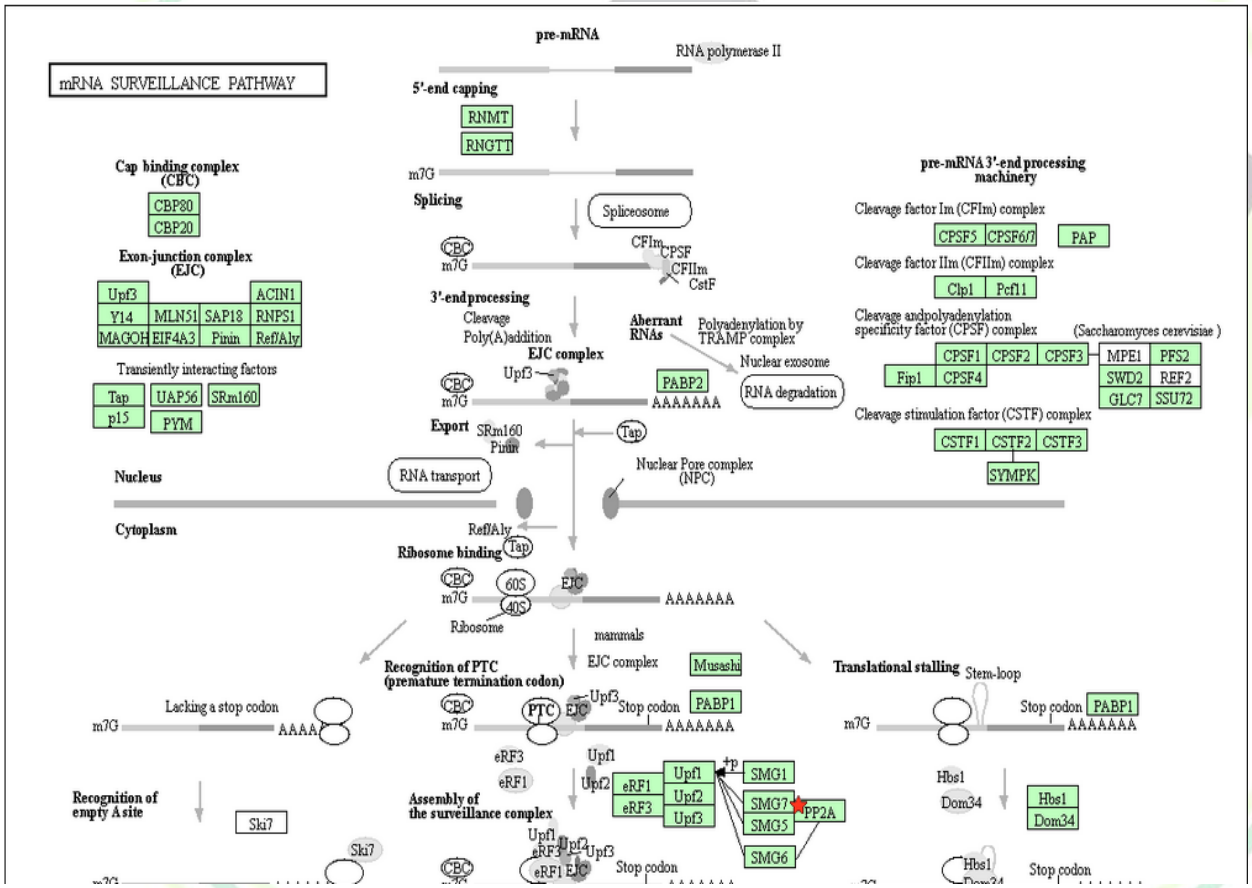
Help and Manual

Current Gene List: List_1
Current Background: Homo sapiens
4 DAVID IDs

1 record(s)

Download File

ENST00000477110, ENST00000381625, ENST00000468169, ENST00000496630, ENST00000390665, ENST00000479438, ENST00000445792	protein phosphatase 2 regulatory subunit B"beta(PPP2R3B)	Related Genes	Homo sapiens
KEGG_PATHWAY	mRNA surveillance pathway, Sphingolipid signaling pathway, PI3K-Akt signaling pathway, AMPK signaling pathway, Adrenergic signaling in cardiomyocytes, Dopaminergic synapse,		



Lesson-64

Gene Network-I

Gene Network

A gene interaction network is a set of genes (nodes) connected by edges representing functional relationships among these genes.

- These edges are named interactions, since the two given genes are thought to have either a physical interaction through their gene products, e.g., proteins, or one of the genes alters or affects the activity of other gene of interest.

- The Gene Regulatory Network runs in the cell to govern the gene expression levels of mRNA and proteins.
- These play a central role in various essential processes of cell and ultimately the body including, morphogenesis, the creation of body structures.

Regulators

- The regulators of these networks depends on the nature of process,
- A regulator can be;
  - DNA
  - RNA
  - Protein
- OR complexes on any of these correspondently

Interactions

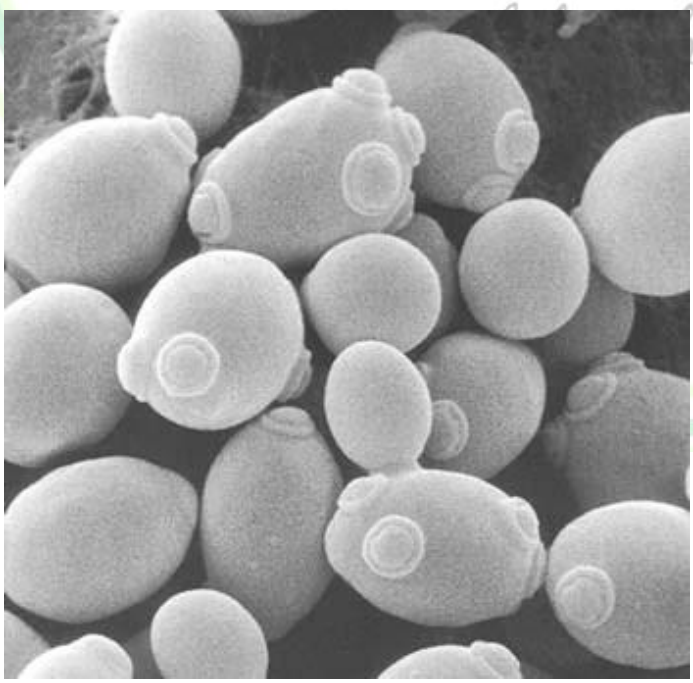
- The interactions between the entities can be both direct or indirect,

For Example,

each mRNA molecule goes on to make a specific protein (or set of proteins). In some cases this protein will be structural, and will accumulate at the cell membrane or within the cell to give it particular structural properties.

Uni-cellular organisms

- The regulatory networks respond to the external environment, optimising the cell at a given time for survival in this environment.
- Yeast cell, finding itself in a sugar solution, will turn on genes to make enzymes that process the sugar to alcohol



Multi-cellular Organisms

- In multicellular animals the same principle has been put in the service of gene cascades that control body-shape.
- Each time a cell divides, two cells result which, although they contain the same genome in full, can differ in which genes are turned on and making proteins.

Studying Gene Networks

- The functional products of genes, e.g., proteins, work together to achieve a particular task, and they often physically associate with each other to function or to form a more complex structure.
- Since these interactions are important to carry out most biological processes, knowledge about interacting proteins is crucial for understanding these biological functions

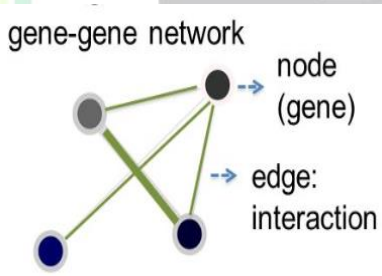
Lesson-65

Gene Network Representation

Nodes & Edges

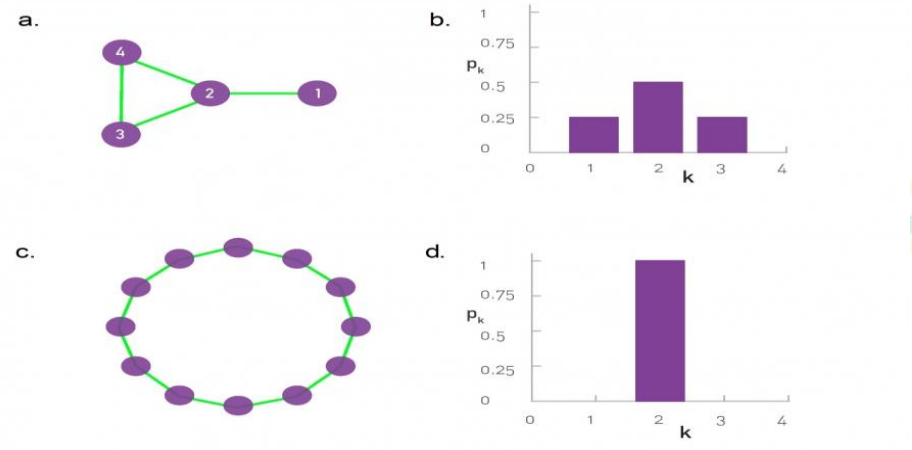
The basic components of any network are

- **Nodes;** Nodes represent units in the network,
- **Edges;** edges represent the interactions between the units



Degree of a Network

- Degree is the number of edges that connect a node, represented by k.

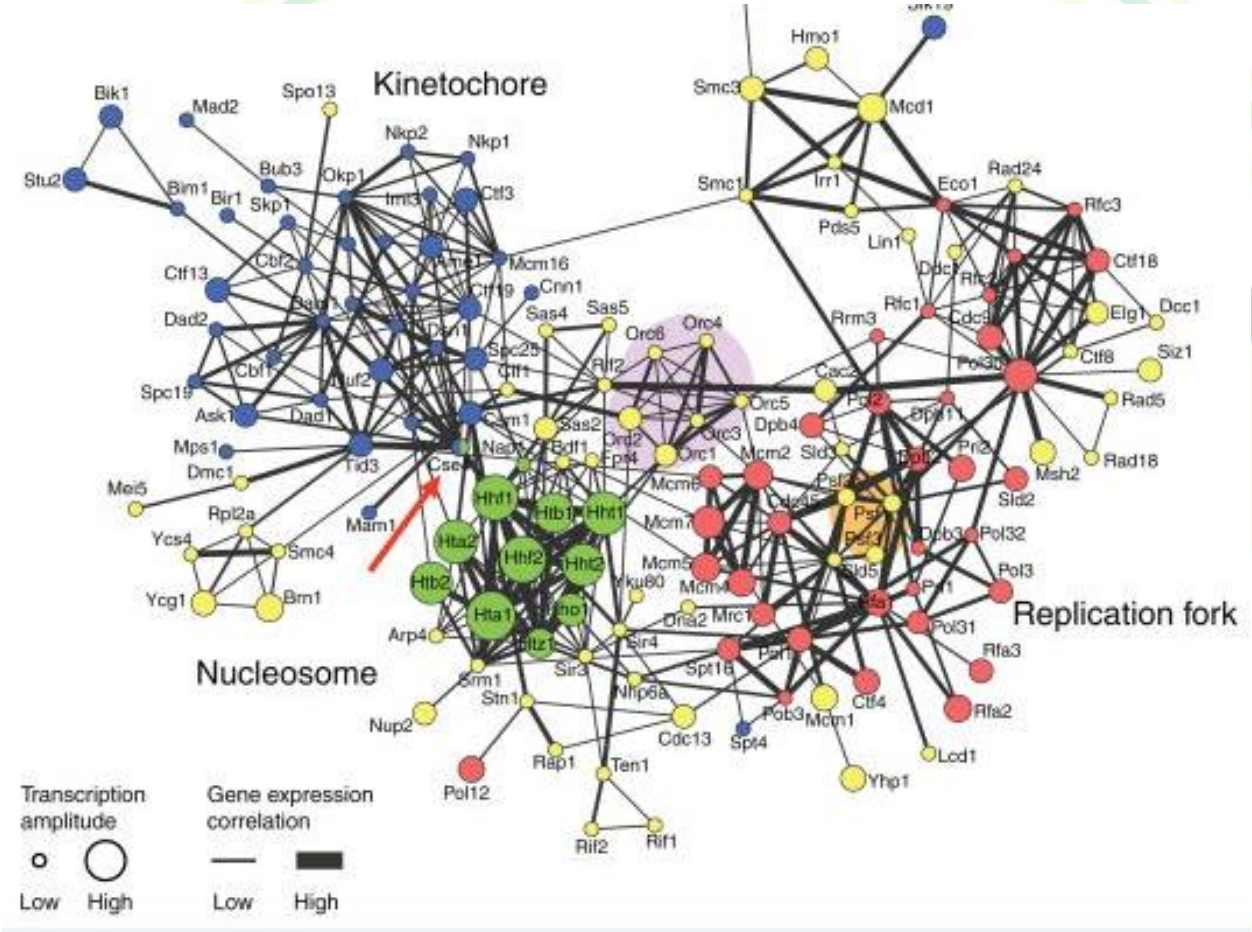


Visual Features

- The different 'visual features'—such as shapes, sizes, colors and line thicknesses depict different properties of a network.

Example

- Node color represents the subcellular localization of a gene. A gene is colored according to whether it localizes to the replication fork (red), nucleosome (green), kinetochore (blue) or other chromosome components (yellow).
- The size of a node and the thickness of an edge convey gene expression data



Example

- Simultaneously visualizing all of these attributes—localization (color), expression level (size) and expression correlation (edge thickness), helps generate consensus results about the gene networks under observation.

Lesson-66

STRING Structural and Functional Protein Networks-I

Introduction

- STRING is a database of known and predicted protein-protein interactions.
- The interactions include direct (physical) and indirect (functional) associations

Data Sources

The sources stem from computational prediction, from knowledge transfer between organisms, and from interactions aggregated from other (primary) databases.

- Genomic context predictions
- High-throughput lab experiments
- Co-expression
- Automated text-mining
- Previous knowledge in databases

How to access?

- The main entry point into the STRING website is the protein search box on its start page
- Users can also arrive via a number of external websites that maintain cross-links with STRING
- A third way to enter STRING is via logging on to the My Data section; this allows users to upload gene-lists

Main entry point



Interaction Scores

- In STRING, each protein-protein interaction is annotated with one or more 'scores'.
- These scores do not indicate the strength or the specificity of the interaction. Instead, they are indicators of confidence, i.e. how likely STRING judges an interaction to be true, given the available evidence
- The scores lie between 0-1; 1 being the highest confidence.

Example

As an example, the model organism 'Escherichia coli K12 MG1655' is shown below — indicating the number of interactions per score type, at a confidence of 'medium' or better (score >= 0.400);

gene neighborhood, normal:	7851 interactions
gene neighborhood, transferred:	11177 interactions
gene fusion:	514 interactions
gene cooccurrence:	35497 interactions
gene coexpression, normal:	12376 interactions
gene coexpression, transferred:	3154 interactions
experiments/biochemistry, normal:	5301 interactions
experiments/biochemistry, transferred:	4113 interactions
annotated pathways, normal:	6726 interactions
annotated pathways, transferred:	1727 interactions
textmining, normal:	27445 interactions
textmining, transferred:	7119 interactions
combined-score, total:	*210914 interactions*

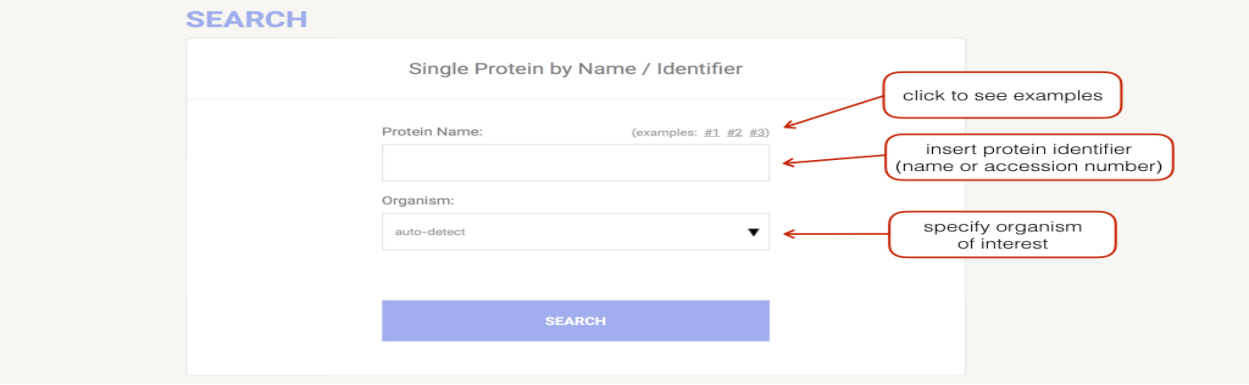
Lesson-67

STRING Structural and Functional Protein Networks-II

Starting Point

To specify your desired starting point of the analysis you have to use the input form at the STRING start page.

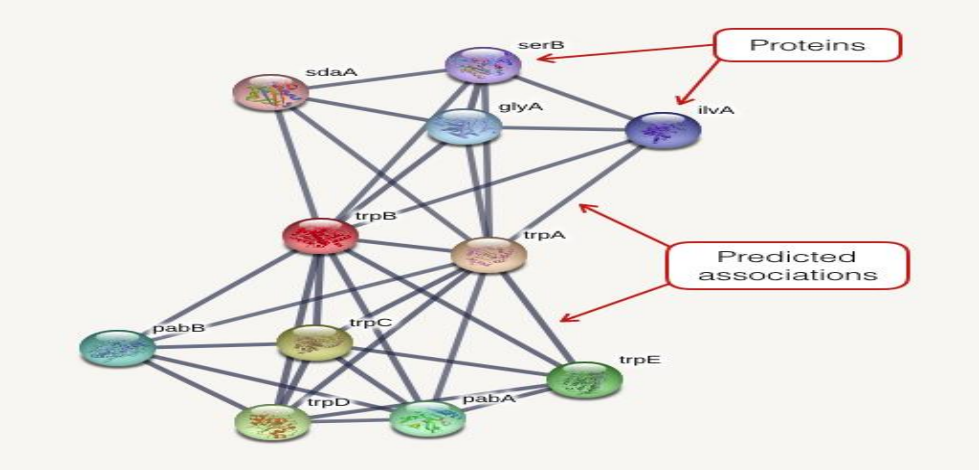
- Protein by name
- Protein by sequence
- Multiple proteins
- Multiple sequences
- Organisms
- Protein families (COGs)
- Examples
- Random entry



You can search STRING by single protein name, multiple names or by amino acid sequence (in any format)

Network

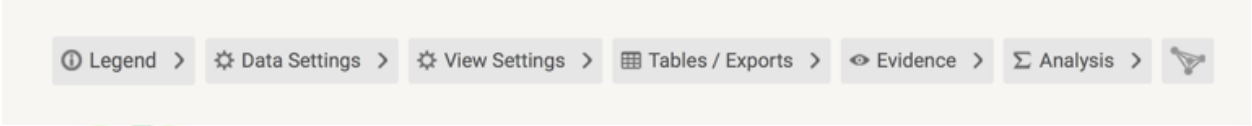
- The network view summarizes the network of predicted associations for a particular group of proteins.
- The network nodes are proteins whereas the edges represent the predicted functional associations.
- Clicking on a node gives several details about the protein. Clicking on an edge displays a detailed evidence breakdown.



Modes

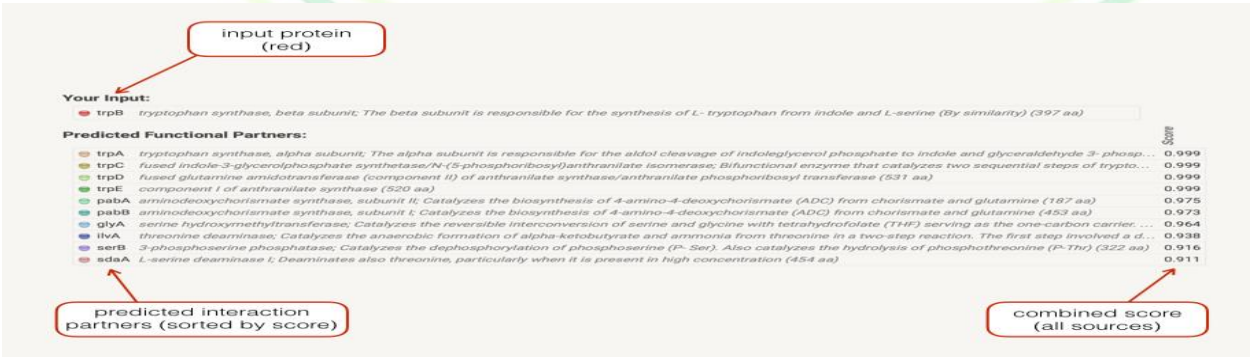
- In *evidence mode*, an edge may be drawn with up to 7 differently colored lines - these lines represent the existence of the seven types of evidence used in predicting the associations.
- In *confidence* mode the thickness of the line indicate the degree of confidence prediction of the interaction.
- *Action* mode show additional information about the prediction, such as, binding, activation, etc.

Navigation Buttons



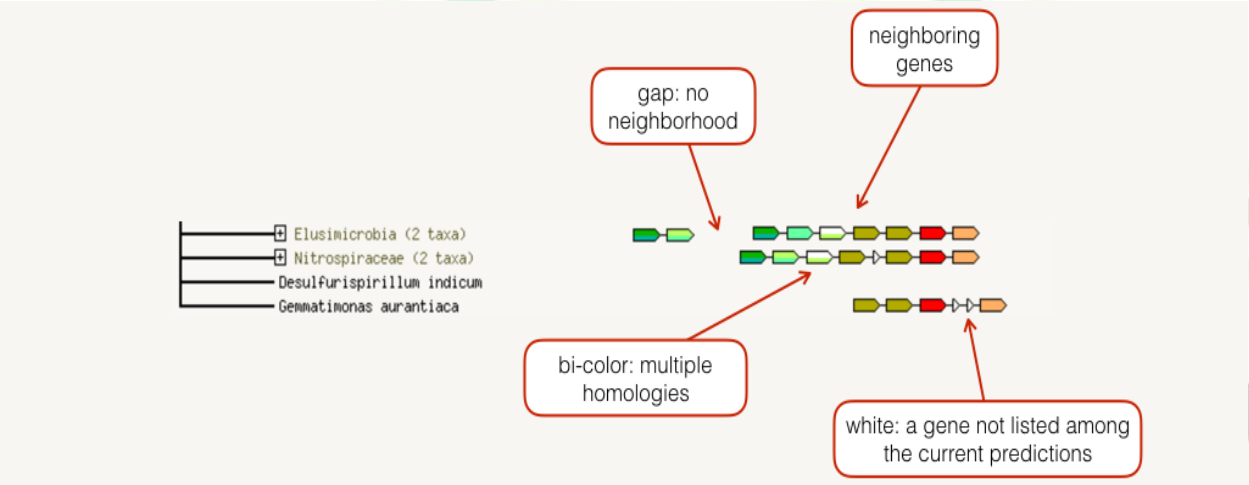
The navigation takes you to different aspects of the data, allowing you change parameters and to see the different types of evidence that supports the predicted associations.

Legend



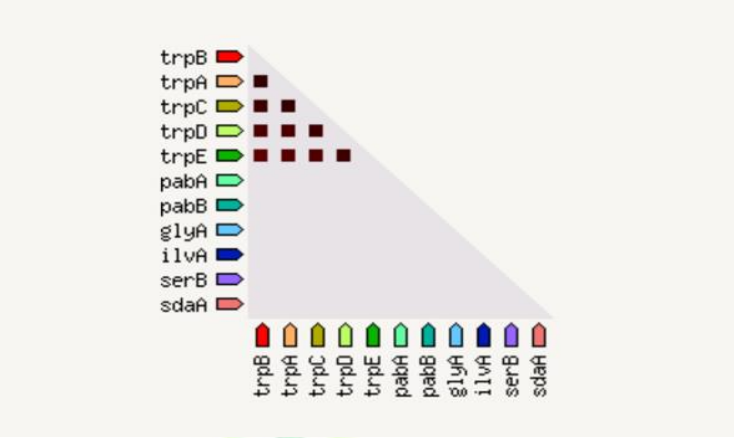
- In the Legend section a list your input(s) is shown. Predicted associations are shown immediately in a list below your input, sorted by score.
- Clicking on the score bullets gives you a breakdown of the individual prediction method scores.
- Clicking on a gene name gives you the protein sequence as well as a list of similar proteins in STRING.

Conserved Neighborhood



Co-expression

- The coexpression view shows the genes that are co-expressed in the same or in other species.
- Co-expression is shown by a red square: more intense color of the square represent a higher association score of the expression data.

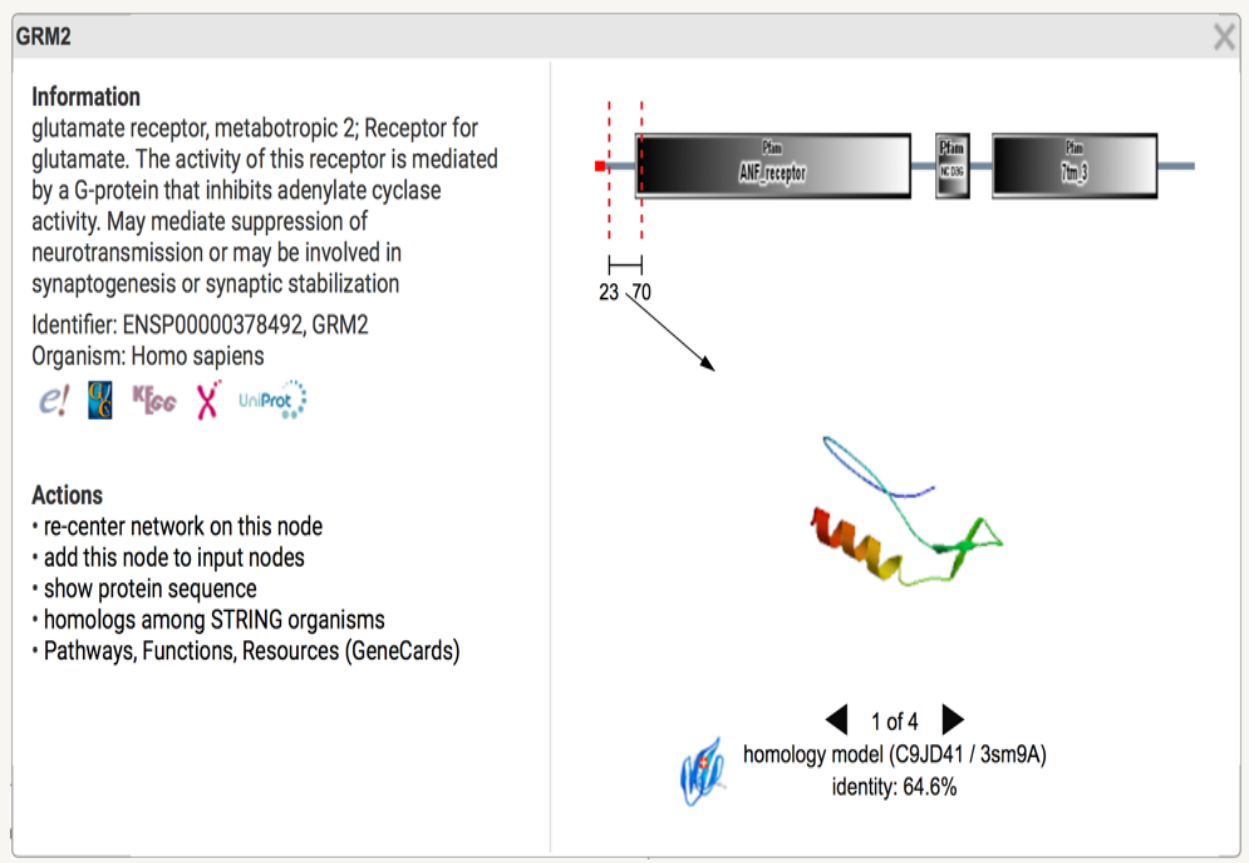


Lesson-68

STRING Structural and Functional Protein Networks-III

Protein Window

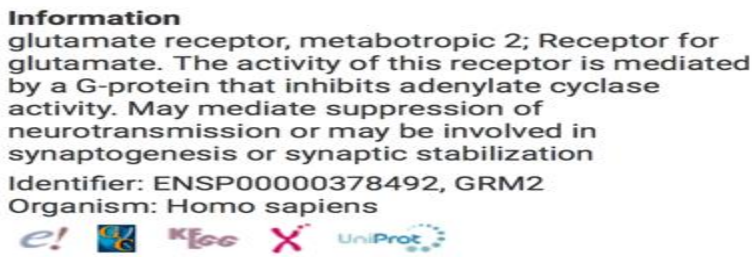
- The protein window is a popup window that appears when you click on a particular protein.
- The protein window shows information about the protein and also provide links to other sections of STRING or to other external websites.



- The protein window is divided in two parts: left and right.
- The right part is displayed only when we have the protein structure (or a model of the protein structure) stored in our database

Information

- A description of the protein is shown.
- The identifier of the protein generally correspond to the identifiers of the import source of the genomic information
- The organism is indicated for clarity.
- Also links to other resources are provided (e.g., UniProt, Ensembl, etc.).



Actions

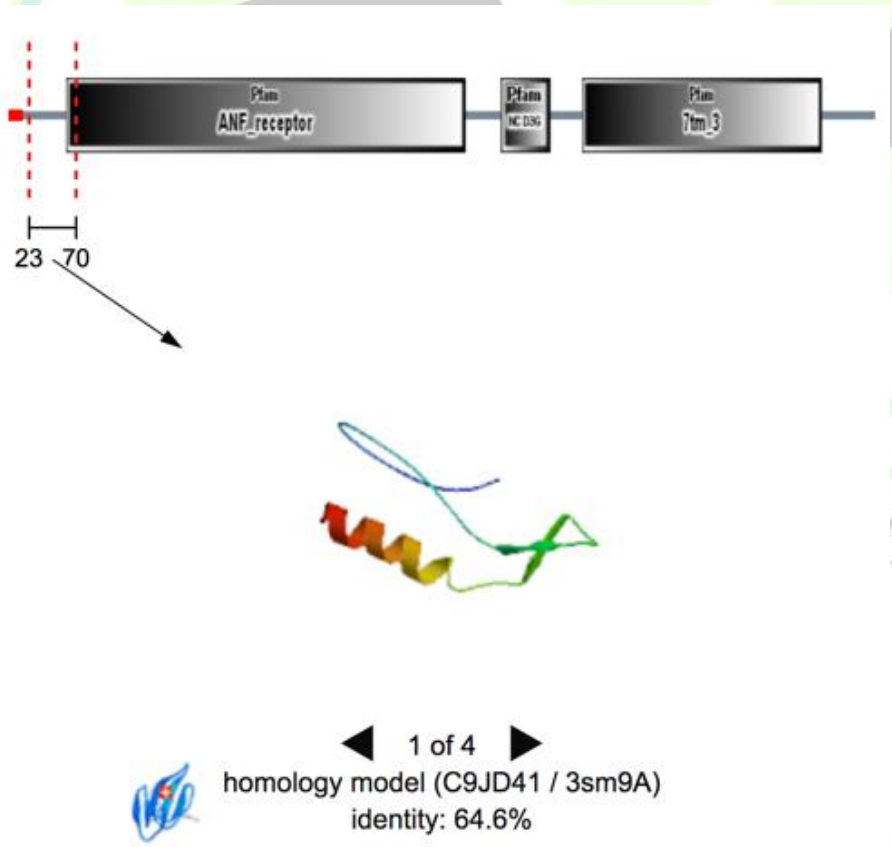
- The action are available are:
- *recenter the network on this node* - adjusts the network to this protein (i.e. look for all the proteins in

STRING that interact with the protein of the window). This is a shortcut with respect to move back to the input page and relaunch STRING with the corresponding protein identifier

- *add this node to the input node* - adds to the network also the proteins that interact with the protein of the window. This is a shortcut with respect to moving back to the input page and relaunch STRING with both the identifiers (i.e. the input protein and the selected protein)
- *show protein sequence* - opens popup window containing the amino acid sequence of the protein
- *homologs in STRING* - brings to a String page where you can see all the homologs of your protein in the same and also in other organisms.
- *Pathway, Functions, Resources (GeneCards)* - redirects to the relative entry in the GeneCards database (only for human proteins).
- *show protein domain (SMART)* - redirects to the relative entry in the SMART database.

Domain and Structure

If no crystal-structure or structure model can be found in our database for the protein (or for similar proteins), this part is not shown.



- Usually the image of the protein structure represents only a part of the protein (domain).
- For the PDB structures whole protein structure is displayed, when this is available.
- In this case, hovering the mouse cursor over the image will change the image of the whole protein structure with the image of the chain

Lesson-69

Visual Features in Cytoscape

Cytoscape- Menus

File

The File menu contains basic file functionality:

File → Recent Session for opening a recently opened Cytoscape session file

File → New for creating a new network.

File → Open... for opening a Cytoscape session file.

File → Save for saving a session file.

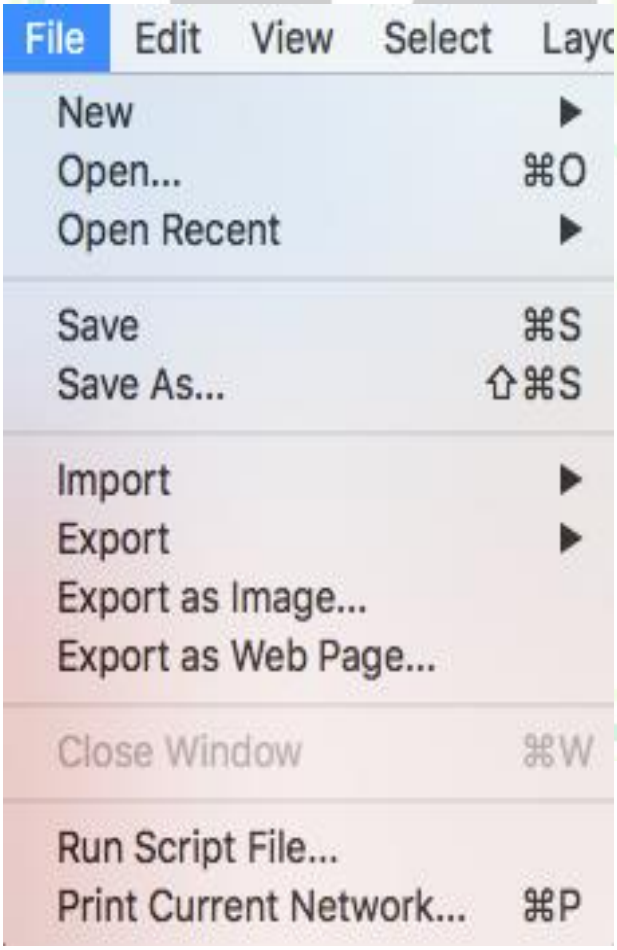
File → Save As... for saving a session file.

File → Import for importing data such as networks and table data.

File → Export for exporting data and images.

File → Run... for running a Cytoscape script file.

File → Print Current Network allows printing.



Edit

The Edit menu contains options for manipulating networks and network views:

Copy, Cut and Paste editing functions.

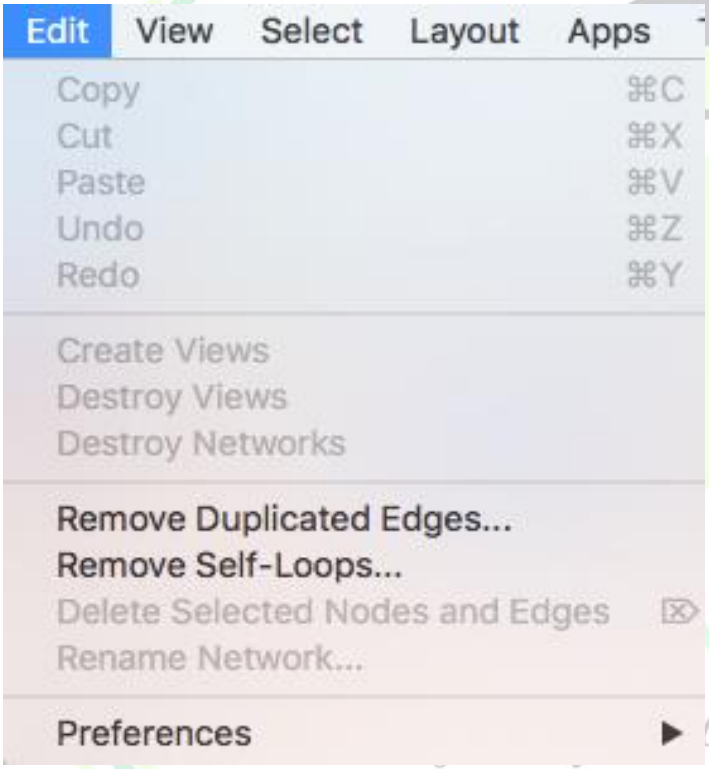
Undo and Redo functions which undo and redo edits made in the Table Panel and the Layout.

Options for creating and destroying views (graphical representations of a network) and networks, and for renaming the network

Options for deleting selected nodes and edges from the current network

Options for removing duplicated edges and self-loops.

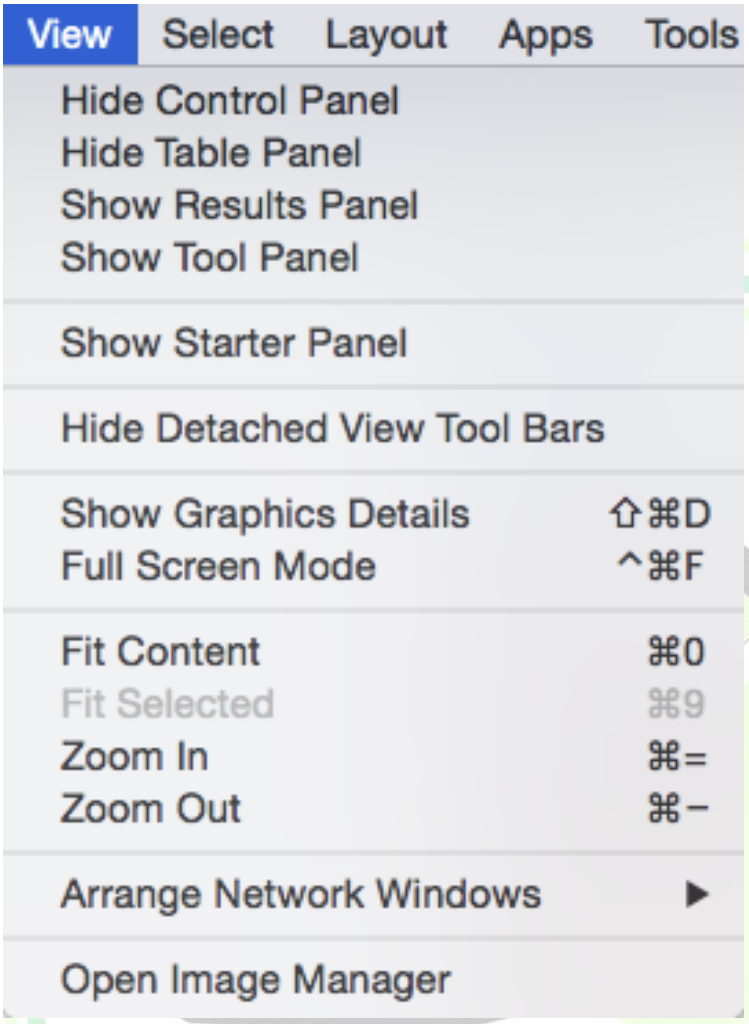
Edit → Preferences → Properties to edit preferences for properties and apps.



View

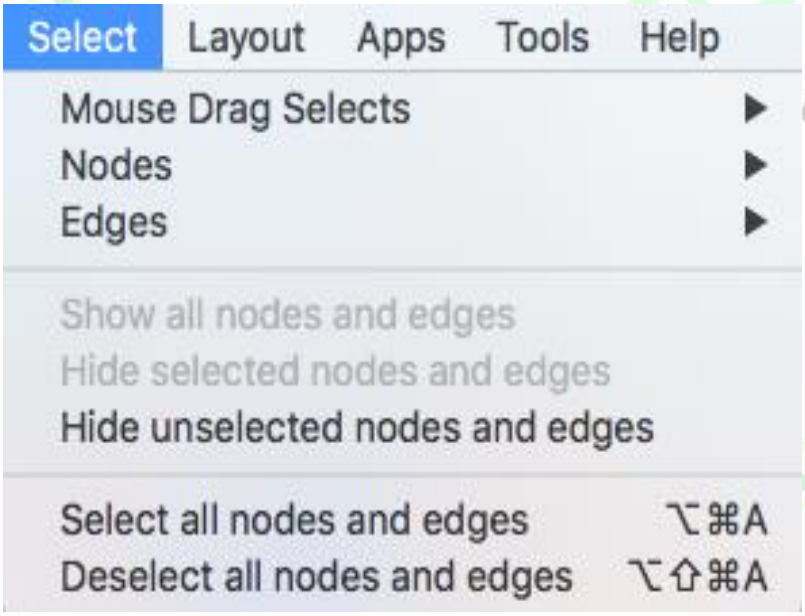
The View menu allows you to display or hide:

- The Control Panel.
- The Table Panel.
- Results Panel. When displayed, the Results Panel appears in the upper right corner of the Cytoscape window.
- Show Graphics Detail. Displays all graphics details, regardless of the current Level of Details set.
- Full Screen Mode utilizes the full display for Cytoscape.
- If multiple network windows are present, you can control how to display them under Arrange Network Windows.



Select

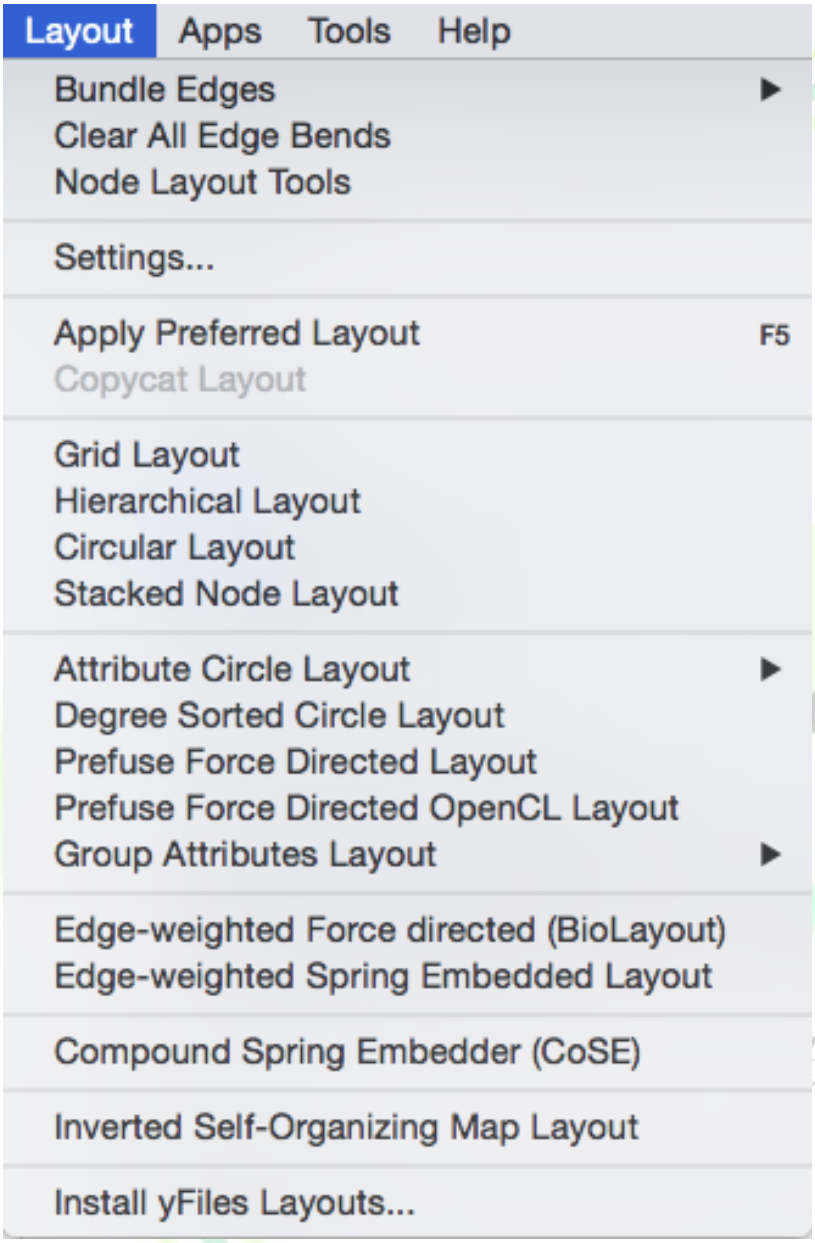
The Select menu contains different options for selecting nodes and edges.



Layout

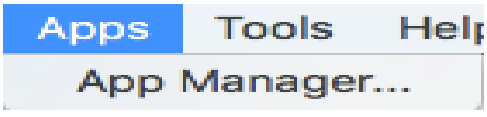
- The **Layout** menu has an array of features for visually organizing the network.

- The features in the top portion of the network (**Bundle Edges**, **Clear Edge Bends**, **Rotate**, **Scale**, **Align and Distribute**) are tools for manipulating the network visualization.
- The bottom section of the menu lists a variety of layout algorithms which automatically lay a network out.



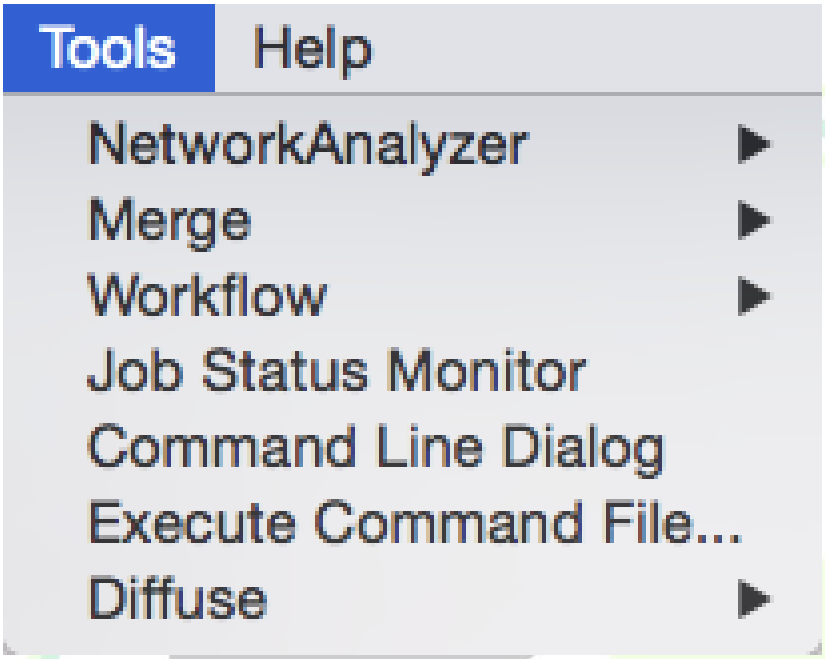
Apps

- The **Apps** menu gives you access to the **App Manager** (**Apps → App Manager**) for managing (install/update/delete) your apps and may have options added by apps that have been installed.
- Depending on which apps are loaded, the apps that you see may be different than what appear here.
- The picture shows a Cytoscape installation without installed apps.



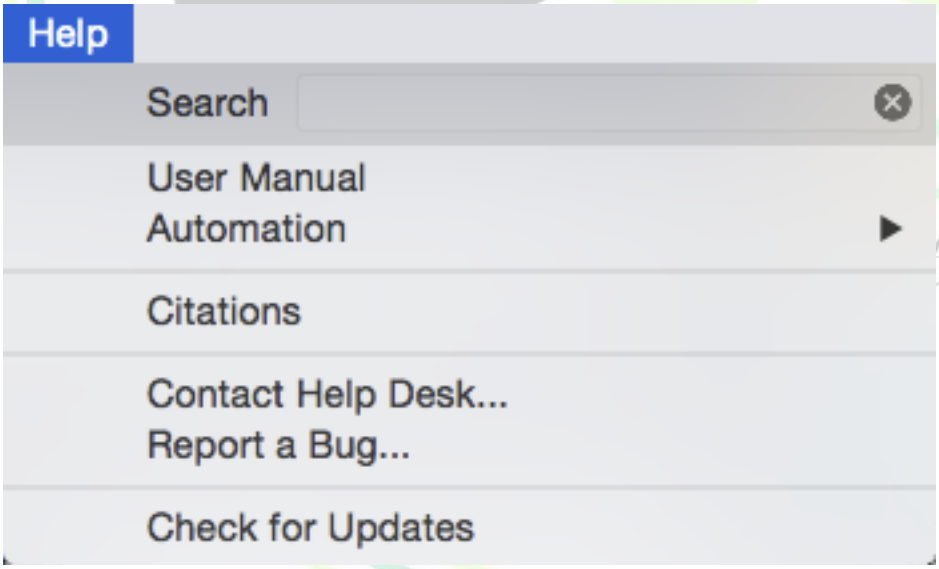
Tools

The **Tools** menu contains advanced features like the **Command Line Dialog**, **Network Analyzer**, **Network Merge** and **Workflow**.



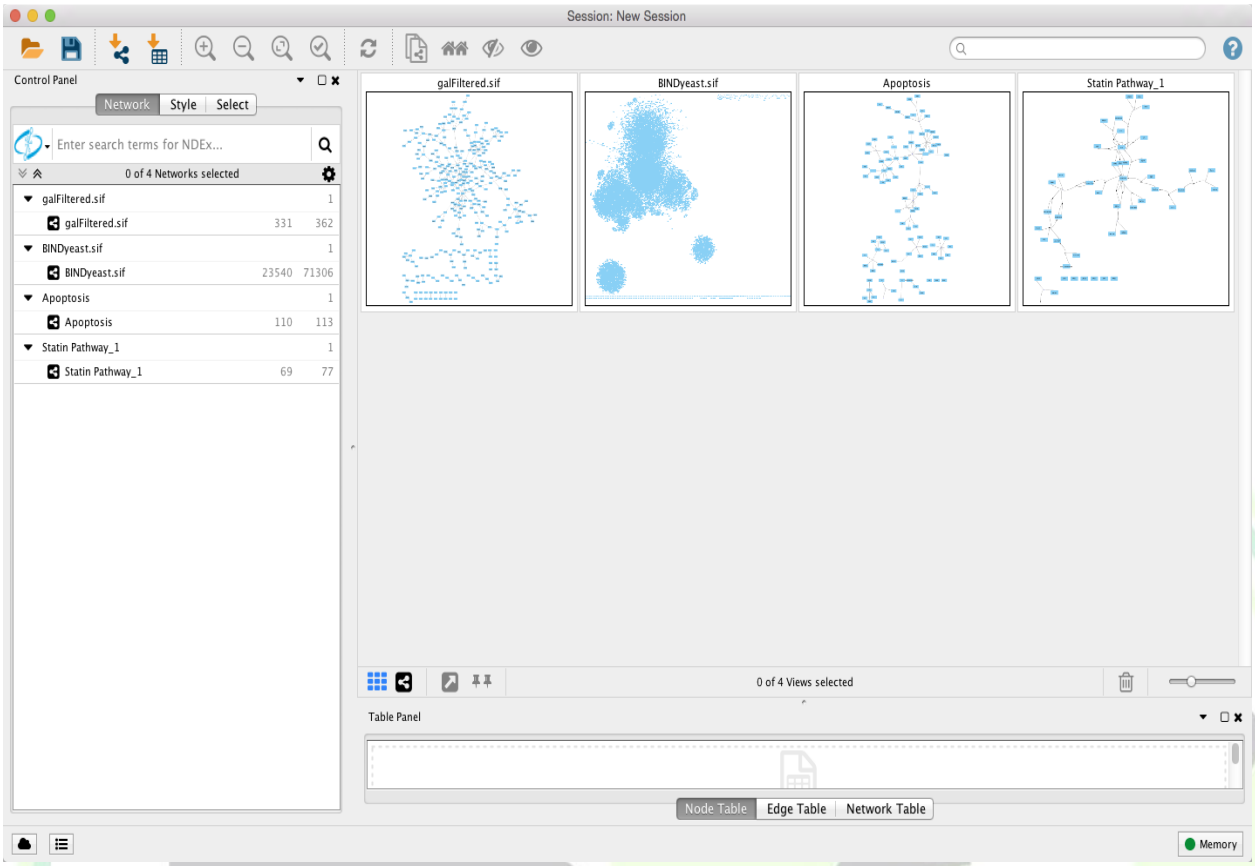
Help

The **Help** menu allows you to launch the online help viewer and browse the table of contents for this manual (**Contents**).



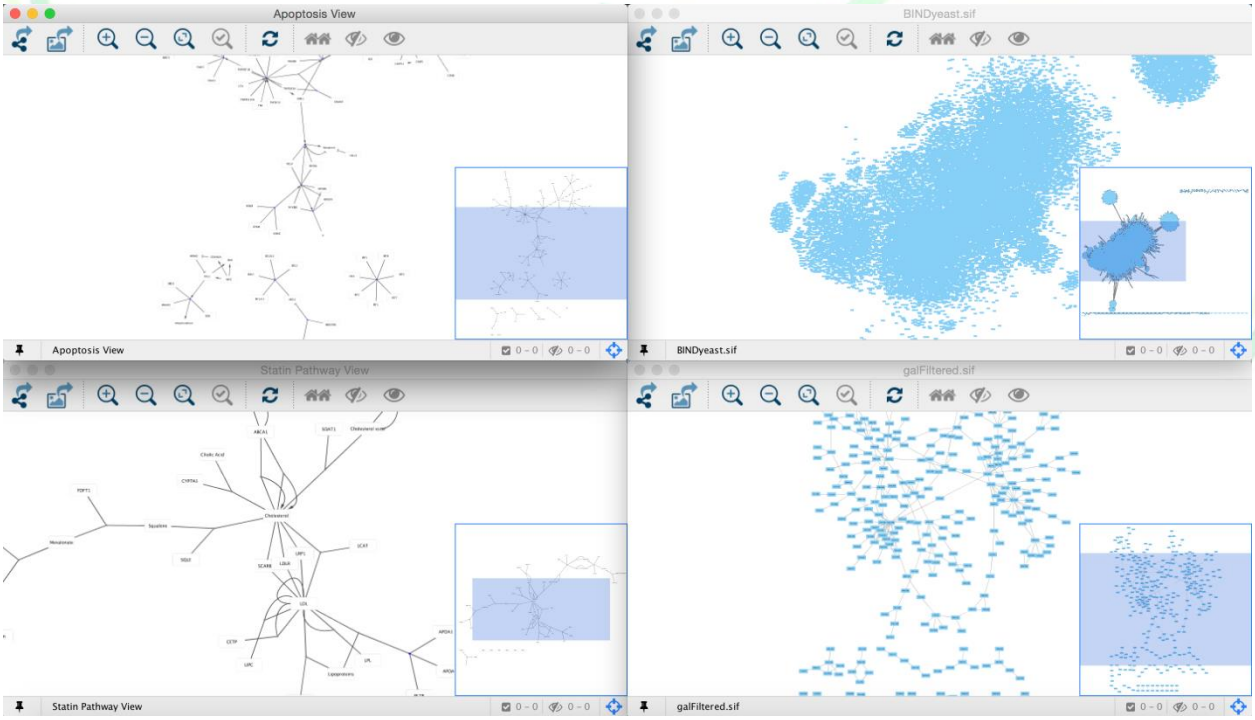
Network Management

- Cytoscape allows multiple networks to be loaded at a time, either with or without a view.
- A network stores all the nodes and edges that are loaded by the user and a view displays them.



Arrange Network Windows

- When you have detached network view windows, you can arrange them by selecting one of these options under **View → Arrange Network Windows**



Lesson-70

Visual Features in Cytoscape-II

Creating Networks

There are 4 different ways of creating networks in Cytoscape:

- Importing pre-existing, fixed-format network files.
- Importing pre-existing, unformatted text or Excel files.
- Importing data from from public databases.
- Creating an empty network and manually adding nodes and edges.

1.Import Fixed-Format Network Files

Network files can be specified in any of the formats described in the **Supported Network Formats** section.

Networks are imported into Cytoscape through the

File → Import → Network menu.

The network file can either be located directly on the local computer, or found on a remote computer

2. Import Networks from Unformatted Table Files

- Cytoscape supports the import of networks from delimited text files and Excel workbooks using **File → Import → Network → File...**.
- An interactive GUI allows users to specify parsing options for specified files.
- The screen provides a preview that shows how the file will be parsed given the current configuration.
- As the configuration changes, the preview updates automatically.

3. Import Networks from Public Databases

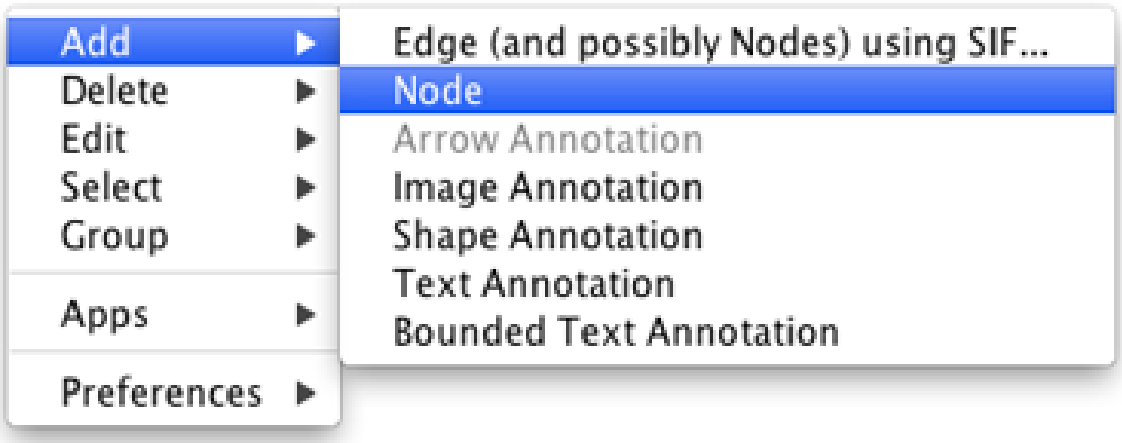
- Cytoscape has a feature called **Import Network from Public Databases**.
- Users can access various kinds of databases through this function, under **File → Import → Network → Public Databases...**.
- As of Cytoscape 3.6, a new **Search bar** is also available at the top of the **Network** panel in the **Control Panel**.
- From the search bar, you can directly access several public databases.

4. Create a New Network or Edit One Manually

- A new, empty network can also be created and nodes and edges manually added.
- To create an empty network, go to **File → New → Network → Empty Network**, and then manually add network components by right clicking on the network canvas or on a node.
- You can edit an existing network using the same process.

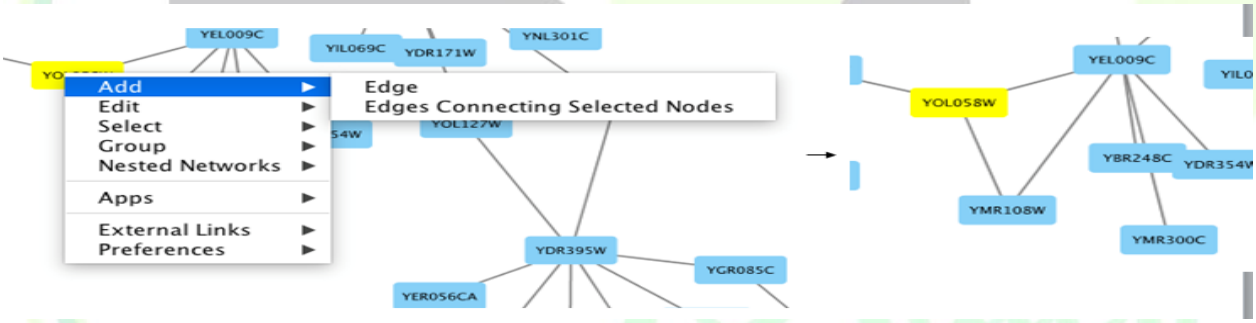
Adding a Node

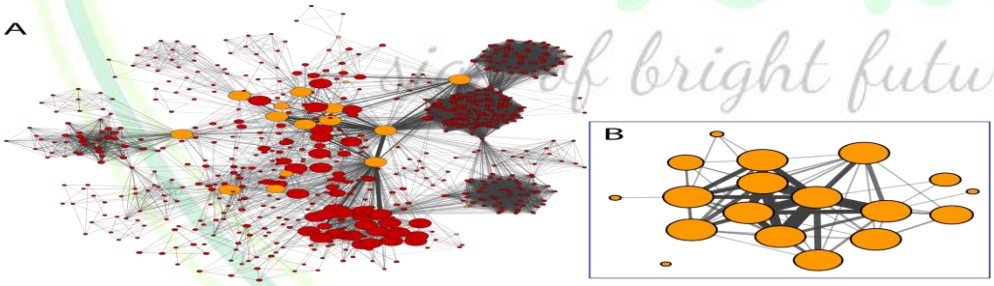
To add a new node, right-click on an empty space of the network view panel. Select Add → Node item from the pop-up menu.



Adding an Edge

- To add an edge to connect nodes, right-click on the source node. Select **Edit → Add Edge** from the pop-up menu. Next, click on the target node.





Lesson-71

Network Layout and Subnetwork

Network Layout

Manual Layout:

The simplest method to manually organize a network is to click on a node and drag it. All of the selected nodes are moved together.

In addition to the ability to click on a node and drag it to a new position, Cytoscape now has the ability to move nodes using the arrow keys on the keyboard.

By selecting one or more nodes using the mouse and clicking one of the arrow keys (←, ↑, →, ↓) the selected nodes will move one pixel in the chosen direction.

If an arrow key is pressed while holding the Shift key down, the selected nodes will move 15 pixels in the chosen direction.

Node Layout Tools

The Tool Panel is available via the menu command: View → Show Tool Panel.

It contains several Node Layout Tools that can help to automate or fine tune a layout.

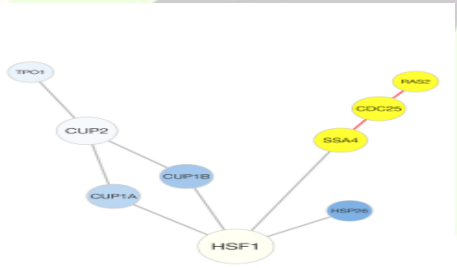
Scale:

Adjust the Scale slider to change the length of edges.

The position of the nodes will be scaled, not the node sizes.

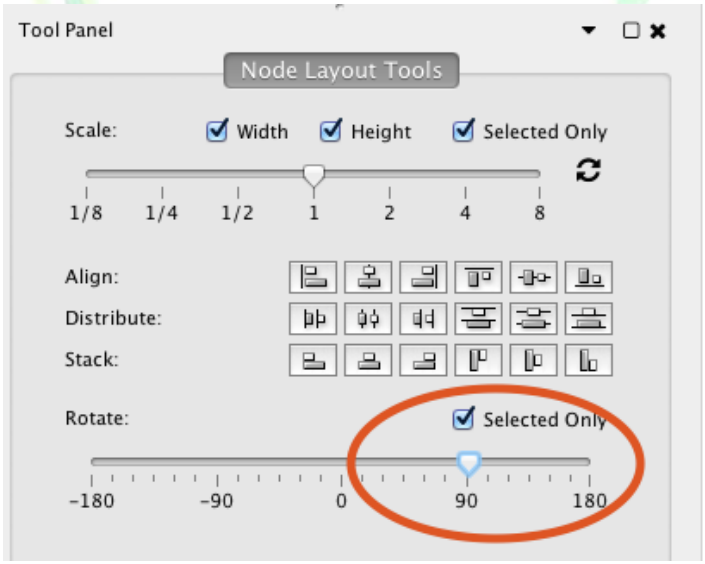
Node size can be adjusted using Styles.

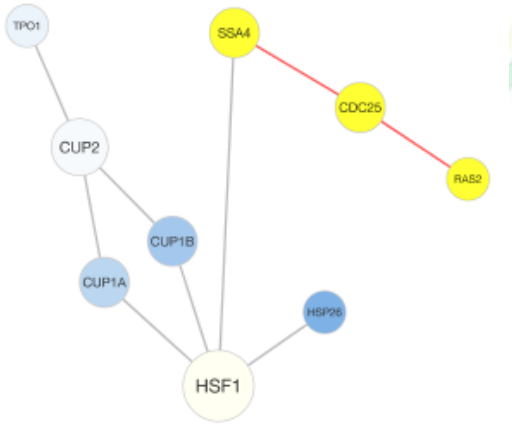




Rotate:

The Rotate function will change the orientation of the entire network or a selected portion of the network. The images below show a network with selected nodes rotated 90 degrees.





Align, Distribute and Stack

Align provides different options for either vertically or horizontally aligning selected nodes against a line.

The differences are in what part of the node gets aligned, e.g. the center of the node, the top of the node, the left side of the node.

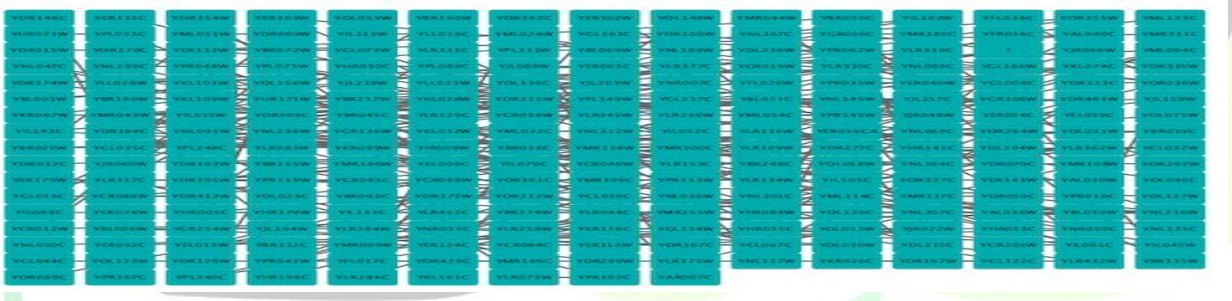
Distribute evenly distributes selected nodes between the two most distant nodes along either the vertical or horizontal axis. The differences are again a function what part of the node is used as a reference point for the distribution.

Stack vertically or horizontally stacks selected nodes with the full complement of alignment options. The table below provides a description of what each button does.

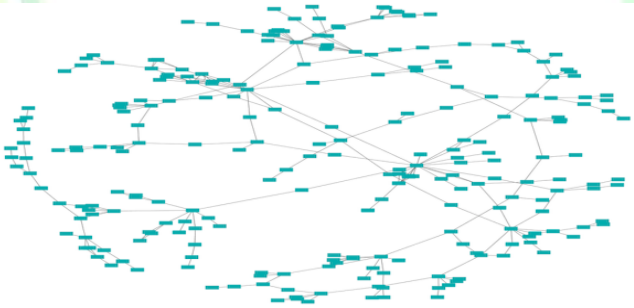
Automatic Layout Algorithms

- Cytoscape Layouts have the option to operate on only the selected nodes, and all provide a Settings... panel to change the parameters of the algorithm.
- Most of the Cytoscape layouts also partition the graph before performing the layout.
- In addition, many of these layouts include the option to take either node or edge columns into account.

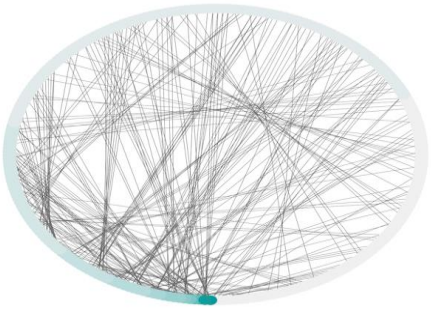
Grid Layout



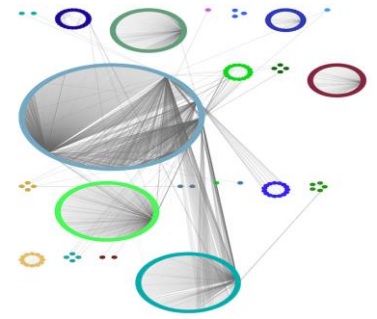
Edge-weighted Spring-Embedded Layout



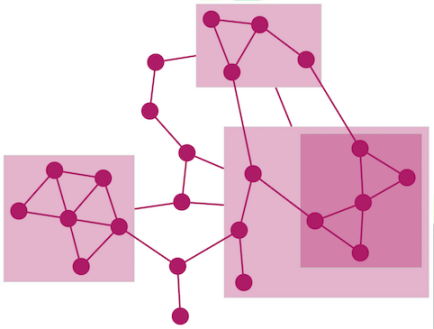
Attribute Circle Layout



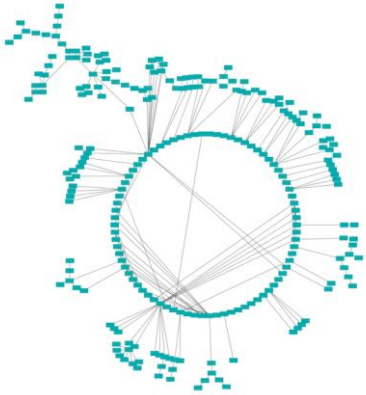
Group Attributes Layout



Compound Spring Embedder Layout



Circular Layout

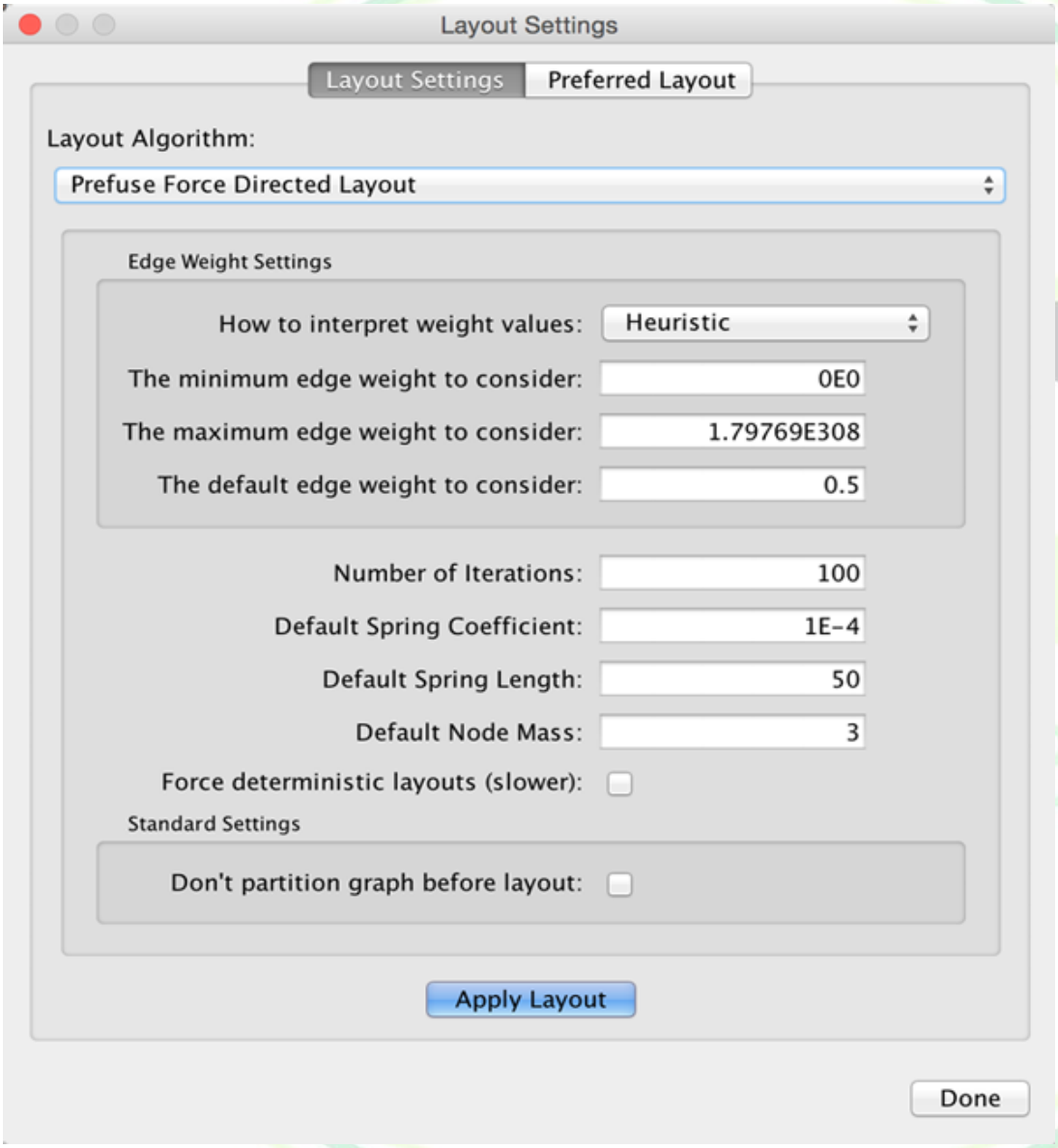


Hierarchical Layout



Layout Parameters

- Many layouts have adjustable parameters that are exposed through the Layouts → Settings... menu option.
- The Layout Settings dialog, which allows you to choose which layout algorithm settings to adjust, is shown below.
- The settings presented vary by algorithm and only those algorithms that allow access to their parameters will appear in the drop-down menu at the top of the dialog



Subnetwork Creation

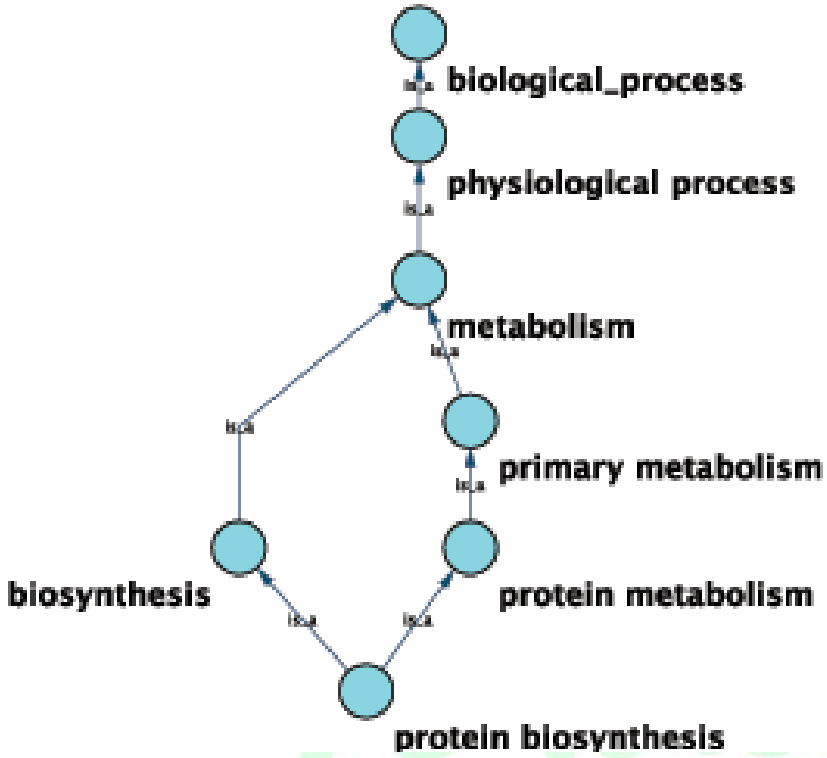
- NetworkAnalyzer allows for the creation of sub-networks of connected components.
- The user selects a number of connected components from a list and each selected component is visualized as a sub-network.
- To create sub-networks from connected components, select Tools → NetworkAnalyzer → Subnetwork Creation → Extract Connected Components.

Lesson-72

Gene Lists- Ontology and Annotation

Ontology and Annotation

- Annotations in Cytoscape are stored as a set of ontologies (e.g. the Gene Ontology, or GO).
- An ontology consists of a set of controlled vocabulary terms that annotate the objects.
- Cytoscape can use this ontology DAG (Directed Acyclic Graph) to annotate objects in networks.



Gene Association File

sign of bright future

- The **Gene Association** files provide annotation only for the Gene Ontology.
- It is a species-specific annotation file for GO terms. Gene Association files will only work with Gene Ontology annotation.

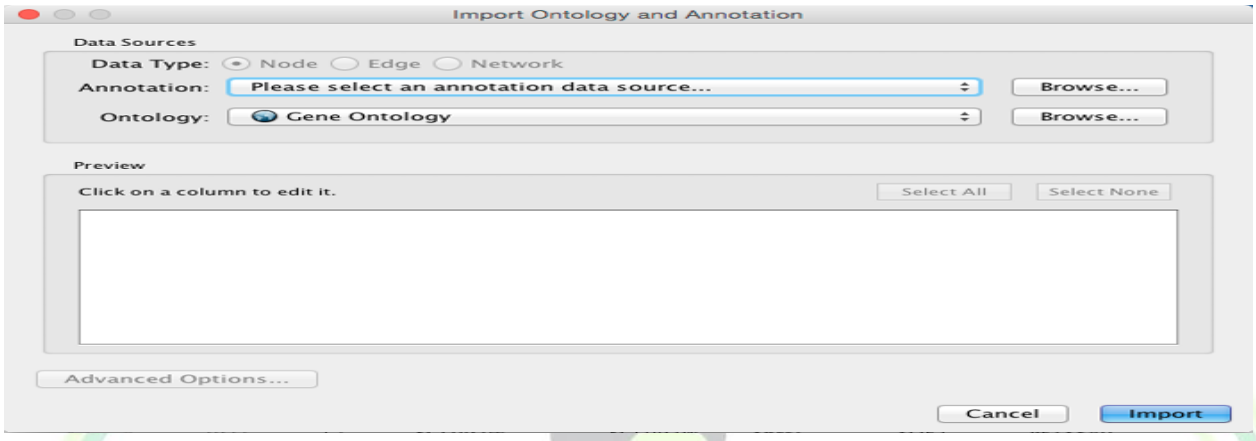
Sample Gene Association File

SGD	S000003916	AAD10	GO:0006081	SGD_REF:S000042151|PMID:10572264	ISS	P	aryl-alcohol dehydrogenase (putative)	YJR155W gene	taxon:4932
SGD	S000005275	AAD14	GO:0008372	SGD_REF:S000069584	ND	C	aryl-alcohol dehydrogenase (putative)	YNL331C gene	taxon:4932

Import Ontology and Annotation

- Select **File → Import → Ontology and Annotation...** to open the "Import Ontology and Annotation" interface.

- From the **Annotation** drop-down list, select a gene association file for your network.
- For example, if you want to annotate the yeast network, select "Gene Association file for Saccharomyces cerevisiae".



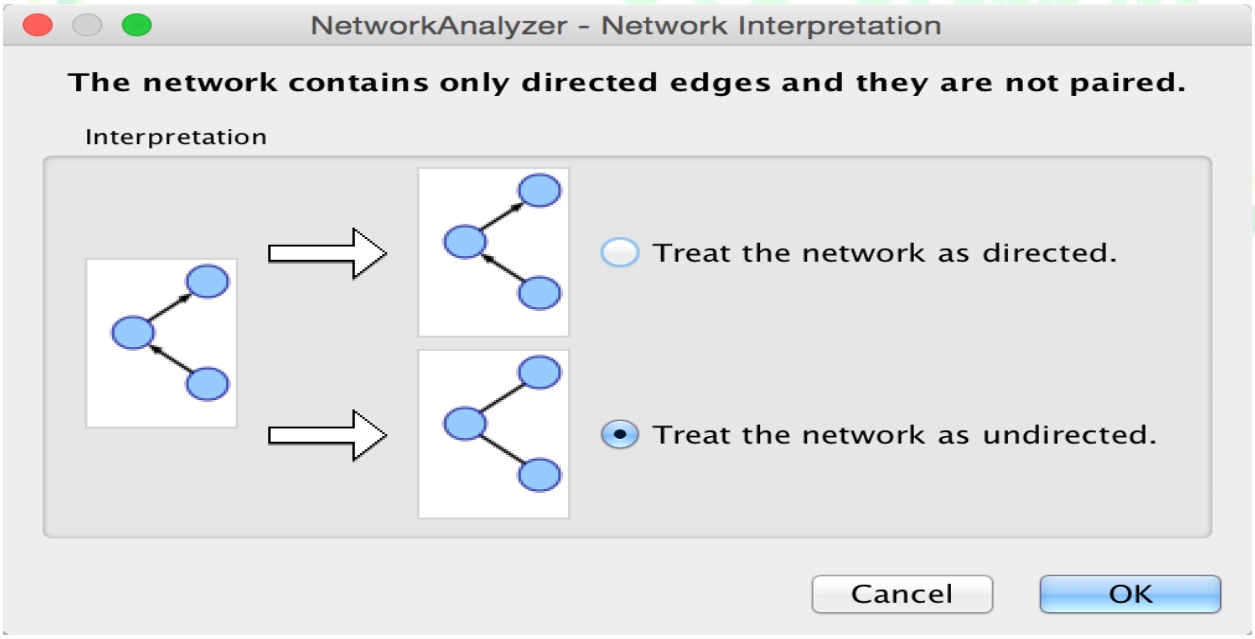
- Once you click the **Import** button, Cytoscape will start loading OBO and Gene Association files from the remote sources. If you choose GO Full it may take a while since it is a large data file.
- When Cytoscape finishes importing files, the import window will be automatically closed. All columns mapped by this function have the prefix "annotation" and look like this: annotation.[column_name].

Lesson-73

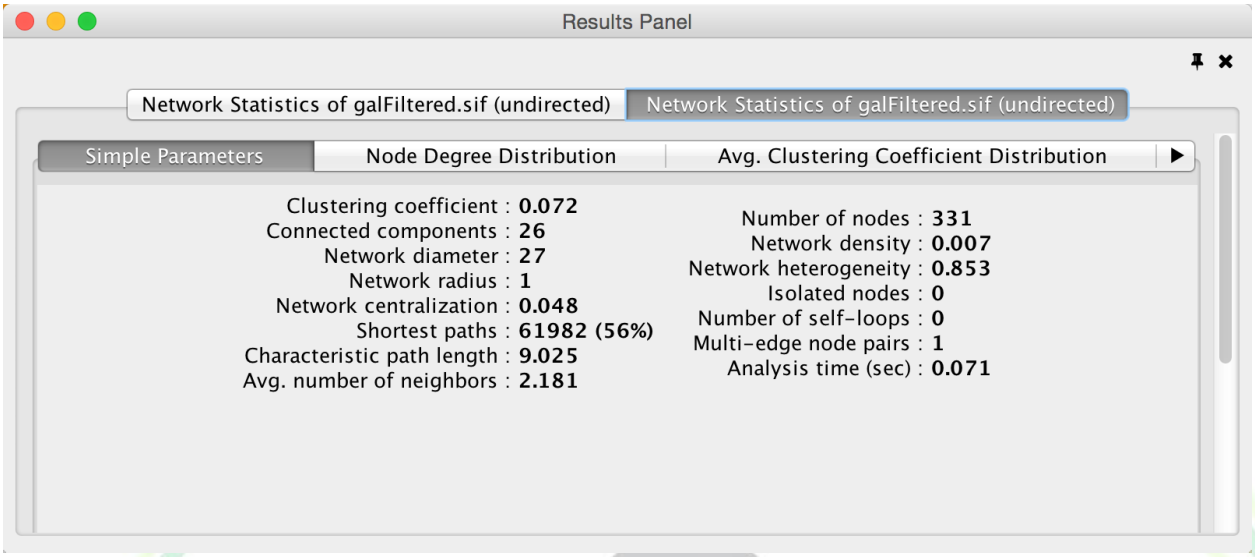
Network Analysis Overview

Network Analysis

- To run NetworkAnalyzer, select Tools → NetworkAnalyzer → Network Analysis → Analyze Network..



When results are calculated, they will appear in the Results Panel.



- The results have multiple tabs.
  - Simple Parameters
  - Node Degree Distribution
  - Avg. Clustering Coefficient Distribution
  - Topological Coefficients
  - Shortest Path Distribution
  - Shared Neighbors Distribution
  - Neighborhood Connectivity Distribution
  - Betweenness Centrality
  - Closeness Centrality
  - Stress Centrality Distribution

Analyze Subset of Nodes

- NetworkAnalyzer provides the Analyze Subset of Nodes option for computing local parameters for a subset of nodes.
- If one or more nodes in the network are selected before starting an analysis, only the sub-network induced by the selected nodes is analyzed.
- Moreover, only local parameters are computed. Shared neighbors distribution and shortest path lengths distribution, among others, are not displayed in the results.

Batch Analysis

- The Batch Analysis option is used to perform topological analysis on all networks stored in specific directory, using all possible interpretations for every network.
- Batch analysis consists of three simple steps:

- Selecting directories
- Analysis
- Inspection of results

Plot Parameters

- The Plot Parameters dialog offers a possibility to plot two parameters against each other.
- The parameters to be plotted can be chosen from two drop-down menus.
- The Table Column 1 menu provides the values for the domain/category axis, and the Table Column 2 menu specifies the values for the range/value axis.
- The plot is updated each time a different parameter is selected in one of the menus.

Lesson-74

BiNGO Plugin

BiNGO

- A Biological Network Gene Ontology tool
- BiNGO is a Java-based tool to determine which Gene Ontology (GO) categories are statistically overrepresented in a set of genes or a subgraph of a biological network.
- BiNGO is implemented as a plugin for Cytoscape, which is a an open source bioinformatics software platform for visualizing and integrating molecular interaction networks.
- BiNGO maps the predominant functional themes of a given gene set on the GO hierarchy, and outputs this mapping as a Cytoscape graph.
- Gene sets can either be selected or computed from a Cytoscape network (as subgraphs) or compiled from sources other than Cytoscape

Advantages:

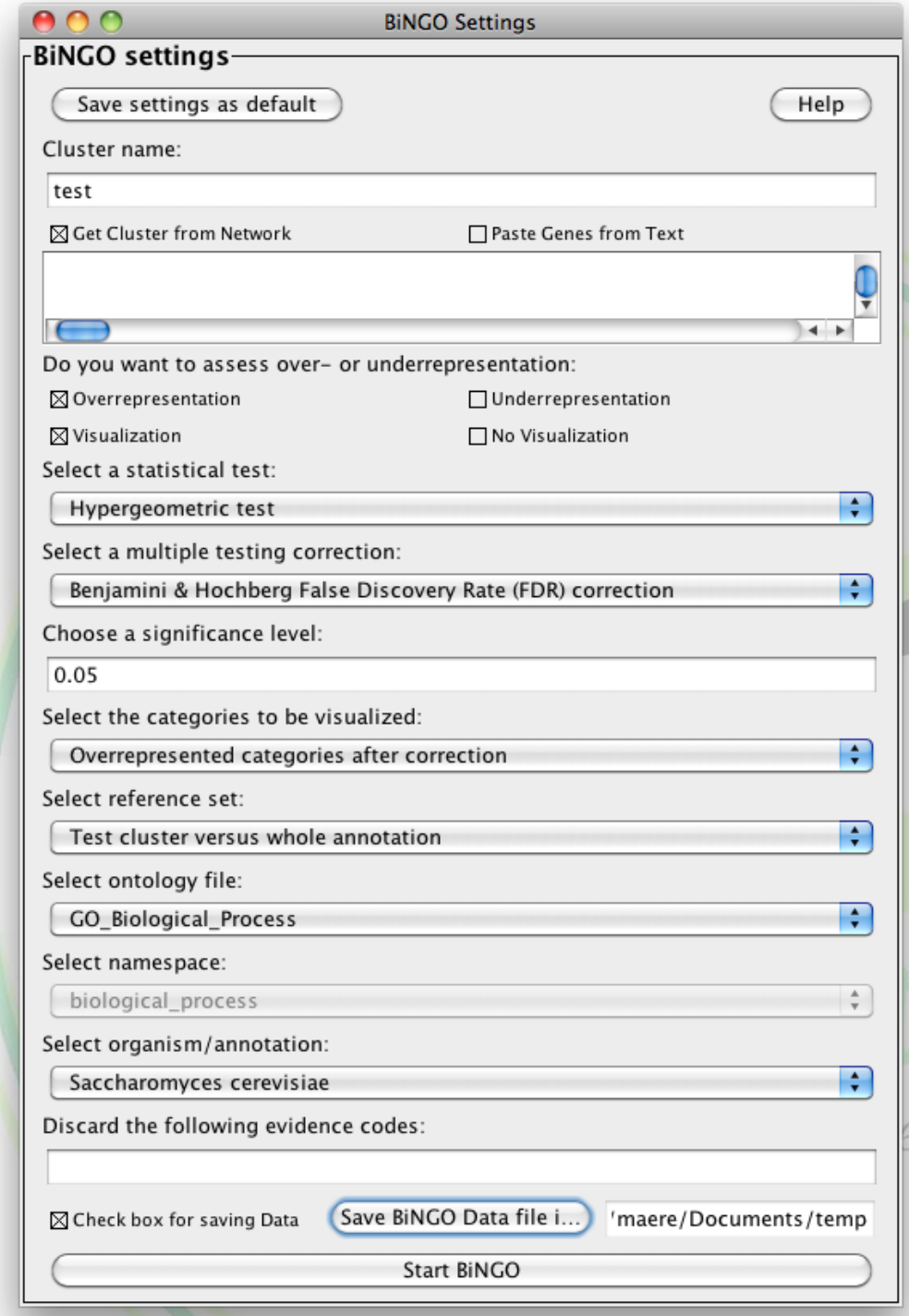
- The main advantage of BiNGO over other GO tools is the fact that it can be used directly and interactively on molecular interaction graphs.
- Another plus is that BiNGO takes full advantage of Cytoscape's versatile visualization environment.
- This allows you to produce customized high-quality figures.

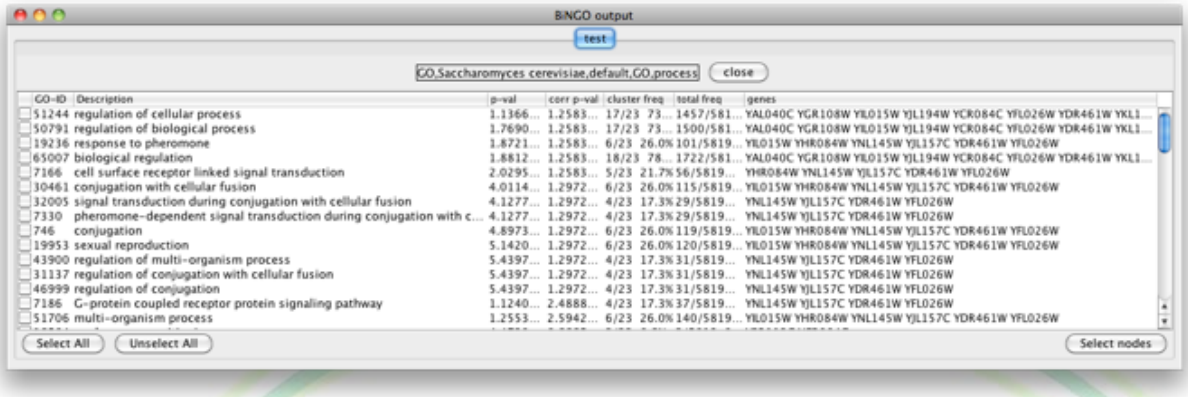
Features include :

- assessing overrepresentation or underrepresentation of GO categories
- Graph or gene list input
- batch mode : analyze several clusters simultaneously using same settings
- Different GO and GOSlim ontologies
- Wide range of organisms

- Evidence code filtering
- Hypergeometric or binomial test for overrepresentation
- Multiple testing correction using Bonferroni (FWER) or Benjamini&Hochberg (FDR) correction
- Interactive visualization of results mapped on the GO hierarchy.
- extensive results in tab-delimited text file format
- make and use custom annotations, ontologies and reference sets
- open source

Bio Mates
sign of bright future


BiNGO Settings
BiNGO settings
Save settings as default
Help
Cluster name:
test
Get Cluster from Network
Paste Genes from Text
Do you want to assess over- or underrepresentation:
Overrepresentation
Underrepresentation
Visualization
No Visualization
Select a statistical test:
Hypergeometric test
Select a multiple testing correction:
Benjamini & Hochberg False Discovery Rate (FDR) correction
Choose a significance level:
0.05
Select the categories to be visualized:
Overrepresented categories after correction
Select reference set:
Test cluster versus whole annotation
Select ontology file:
GO_Biological_Process
Select namespace:
biological_process
Select organism/annotation:
Saccharomyces cerevisiae
Discard the following evidence codes:
Check box for saving Data
Save BiNGO Data file i...
'maere/Documents/temp
Start BiNGO

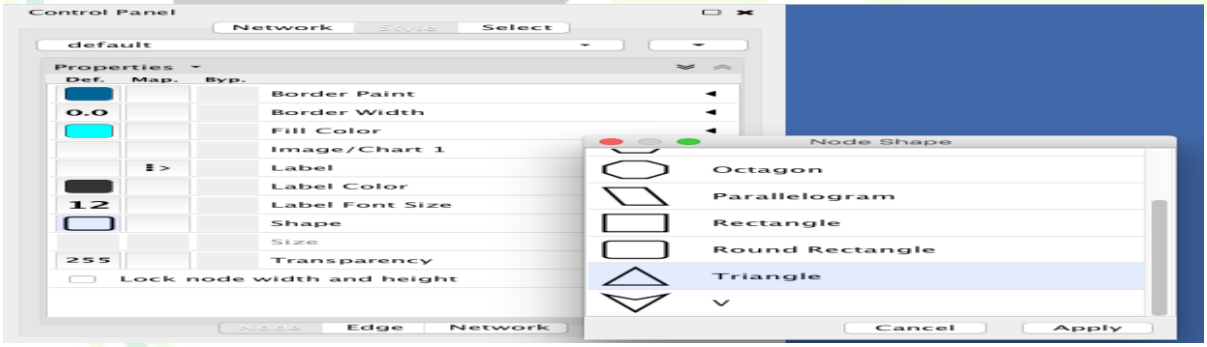


Lesson-75

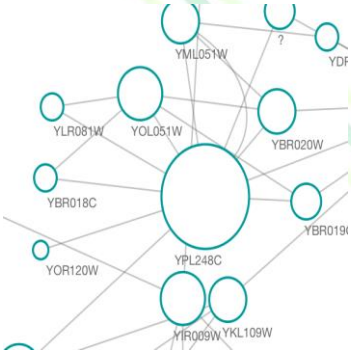
Styles

Styles

- One of Cytoscape's strengths in network visualization is the ability to allow users to encode any table data (name, type, degree, weight, expression data, etc.) as a property (such as color, size of node, transparency, or font type) of the network.
- A set of these encoded or mapped table data sets is called a Style and can be created or edited in the Style panel of the Control Panel.
- Specify a default color and shape for all nodes



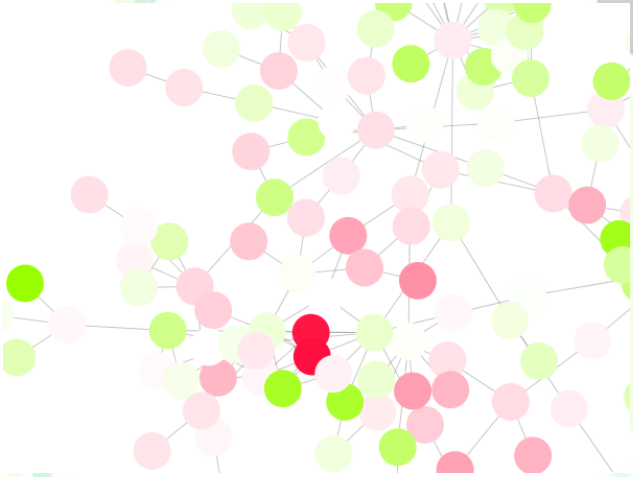
Set node sizes based on the degree of connectivity of the nodes. You can visually see the hub of a network...



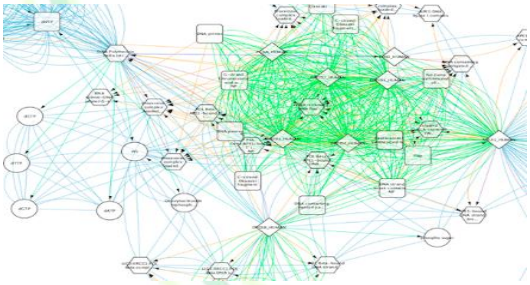
...or, set the font size of the node labels instead.



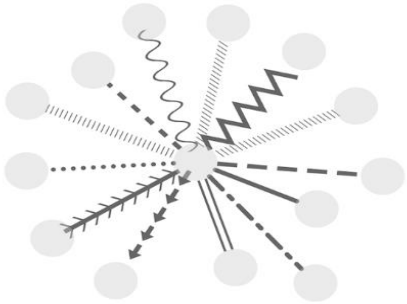
Visualize gene expression data along a color gradient.



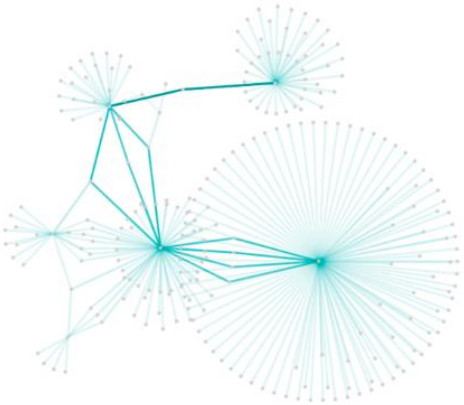
Encode specific physical entities as different node shapes.



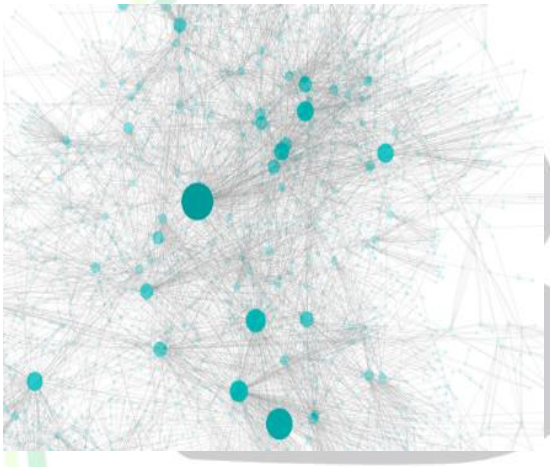
Use specific line types to indicate different types of interactions.



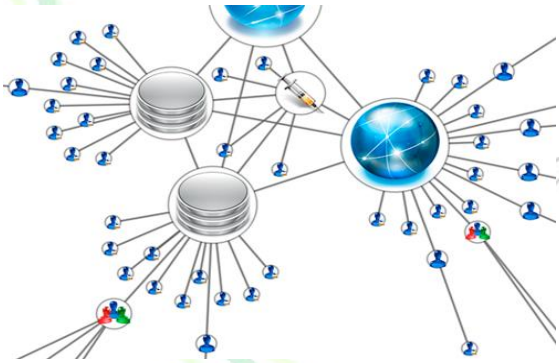
Control edge transparency (opacity) using edge weights.



Browse extremely-dense networks by controlling the opacity of nodes.



Add photo/image/graphics on top of nodes.



Lesson-76

Network Clustering

Network Clustering

- Cytoscape has the ability to associate a Nested Network/ Cluster with any node.
- A nested network is any other network currently defined in Cytoscape.
- This capability allows for creation of network hierarchies as well as circular relationships.

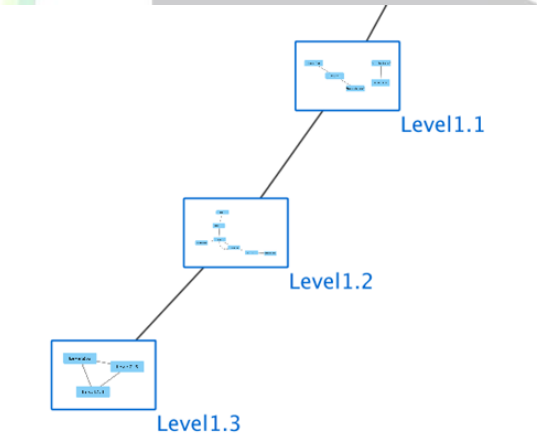
- Note that **Nested Networks** are similar in concept to **Groups**, but with very different consequences.
- A Nested Network incorporates an independent network that does not share nodes or attributes with the current network – it can be edited only as a separate network collection or in a separate Cytoscape session.
- Its nodes cannot be connected to nodes in the current network and cannot be found or filtered using Cytoscape functions or apps.

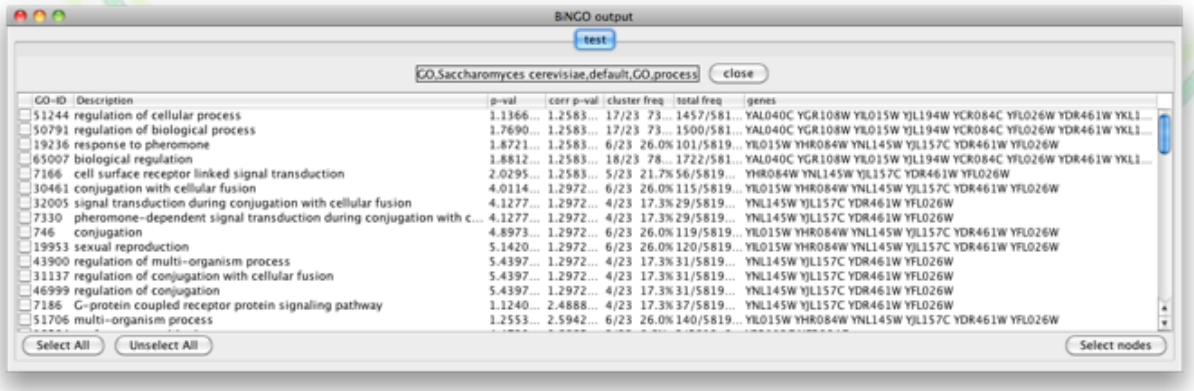
Creating Nested Networks

- There are currently two ways in which a user can create nested networks.
  - By importing a Nested Network Format (NNF) file.
  - By manually constructing networks and assigning nested networks to individual nodes through the right-click node context menu.

Visualization of Nested Networks

- Nodes containing nested networks that are zoomed in sufficiently display an image for the nested network.
- If no current network view exists for the nested network the image will be a default icon, otherwise it will be a low-resolution rendering of the nested network's current layout



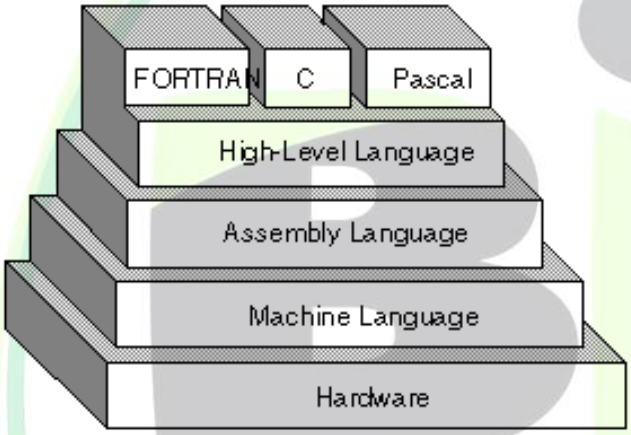


Lesson-77

Python Introduction

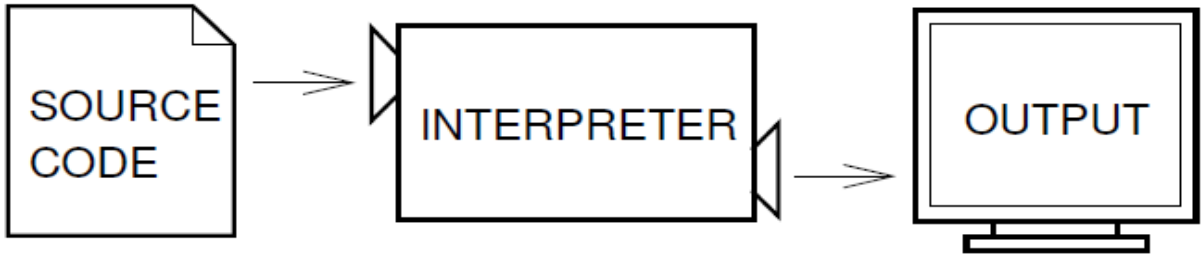
Programming Languages

- Low-level languages
  - Language a computer understands to execute programs.
- High-level languages:
  - Disadvantages?
  - Advantages?

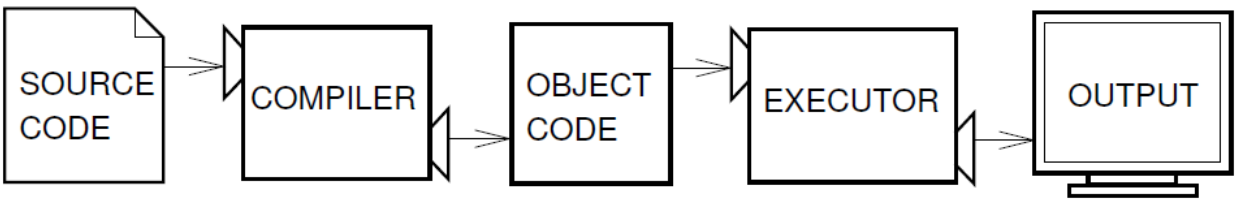


Processing of High-level language into a Low-level language

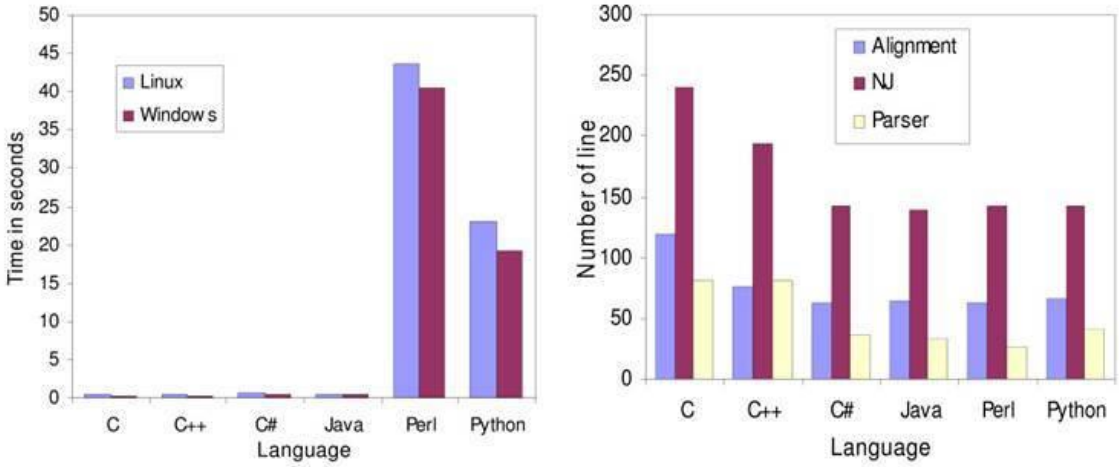
- **Interpreters:**



- **Compilers:**



A Comparison of Bioinformatics Programming Languages (2008)



Why Python?

- Easy to use
- Great for text files.
- Built-in data types for lists, dictionaries, strings. No need to import special classes like in java or C++
- Clean syntax, Portable

Installing Python

- Python for Win/Mac/Linux is available from www.python.org.
  - Installation is generally easy, follow the steps.
- Slight differences between versions 2.x and 3.x.
- In lab, version 3.4.0 is installed. It can be downloaded from https://www.python.org/downloads/release/python-340/

Basic Datatypes

- Integers

For example 2, 3, 5, 0, -1

- Floats (numbers with decimal point)

For example 1.50

- Strings

Either double quotes(“”) or single quotes(‘’) can be used. For example “python” and ‘python’ are same strings.

Unmatched ones can occur within the string. “datatype’s”

Comments

- Add notes in natural language inside code.
- Start comments with # – anything infront of this will be ignored.
- Can use # in a separate line or in front of any code line.

#add two numbers

2+3 #add two numbers

Variables

- Assignment statement will create a new variable.

Syntax: variable_name = value (string, int, float)

```
>>>  message = "Hello"
>>>  n = 17
>>> pi = 3.1415
```

- Variable types don't need to be declared.
- Python figures out the variable types on its own.

Multiple Assignment

- You can also assign to multiple names at the same time.

```
>>> a, b, c = 17, 3.14, "test"
>>> a
 17
>>> b
 3.14
>>> c
 'test'
```

Lesson-78

Python Basics

Printing Output

- You can print a value to the screen using "print."

```
>>> print("hello world")
hello world
```

"Print" automatically adds a newline to the end of the string.

- If you include a coma separated list, it will concatenate them with a space between them.

```
>>> print("hello world", 17)
hello world 17
```

Operators for Numbers

- + , - , *, /, % (for integers), ** (exponential)
- All operators produce results as expected except division operator. In 3.4.0, it's performing as expected

```
>>> 9/10  #integer division rounds down
0
>>> 9/10.0  # Alternative
0.9
```

- Order of operations: PEMDAS
- 2*(3-1) = 4 (1+1)**(5-2) = 8

Comparison Operators

x == *y* # *x* is equal to *y*

x != *y* # *x* is not equal to *y*

x > *y* # *x* is greater than *y*

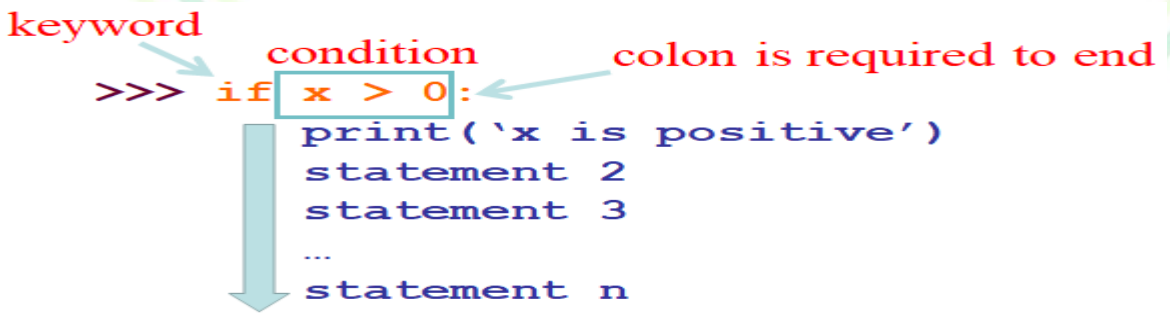
x < *y* # *x* is less than *y*

x >= *y* # *x* is greater than or equal to *y*

x <= *y* # *x* is less than or equal to *y*

Conditional Execution: if, elif, and else

- Allows us to change the behavior of program based on certain conditions.



Conditional: Two Possibilities

- Condition determines which one will be executed.

```
>>> if x >= 0:
      print('x is positive')
          else:
        print('x is negative')
```

- Indentation should match. Both if and else should have at least one statement.

Review of Functions: Calling Built-In Functions

```
>>> type("Hello world")
  <class, 'str'>
```

Function name: type

Input: takes a single value or variable called an *argument*

Output: displays the type of a value or variable. The result is called the return value

Type Conversion: String to Int

- Collection of built-in functions that convert values from one type to another.
- Convert strings to integers.

```
>>> int("17")  # string to integer
  17
>>> int("Python")
Traceback (most recent call last):
  File "<pyshell#24>", line 1, in <module>
    int('Python')
ValueError: invalid literal for int() with base 10: 'Python'
```

Type Coercion

- Remember Integer division problem

```
>>> 9/10
0
```

- Type coercion: automatic type conversion in Python. If one operand is float other is automatically converted to float.

```
>>> 9/10.0
0.9
```

Keyboard input

- Python provides built-in functions to get input from the user.

```
>>> input_value = input()
>>> prompt = “Enter your value=”
# use prompt as an argument to function
>>> input_value = input(prompt)
```

- When these functions are used, the program stops and waits for the user to provide a value and press “enter”.

Lesson-79

Python Loops

Strings

- Strings are enclosed in either single (') or double ("") quotation marks.
- Strings are a composite or compound data type
- Built from another data type, characters
- Within strings characters are stored in sequence, therefore, strings are called sequences.
- Strings can be of any length, for example empty_string = ''

Finding Length of a String

- Use len() function to find the number of characters in a string

```
>>> DNA = "ATGCAGCT"
>>> len(DNA)
8
>>> DNA = "" #empty string
>>> len(DNA)
0
```

Empty strings have zero characters

String Slices: Access Single Characters

```
>>> DNA = "ATGCAGCT"
>>> DNA[0]     #first character
A
>>> DNA[3]     #fourth character
C
>>> DNA[8]     #last character
```

Error

>>> DNA[7] #last character

T

>>> length = len(DNA) // 8

>>> DNA[length-1] #last character

T

String Slices: Segments

- Use a range to specify a slice (sub-string)
  - **Format:** *sample[start:end]*
  - Includes the start index but excludes the last index.

>>> DNA = "ATGCAGCT"

>>> DNA[0:4] #0-3 indices

ATGC

>>> DNA[5:8] #5-7 indices

GCT

>>> DNA[2:50] #2:7 indices

GCAGCT

- Omit the start index to select the start of a sting

>>> DNA = "ATGCAGCT"

>>> DNA[:3] #0-2 indices

ATG

- Omit the last index to select the end of a sting

>>> DNA[5:] #5-7 indices

GCT

String Slices: Segments with Negative Indices

- Omit the start index and use negative index to select the start of a sting

>>> DNA = "ATGCAGCT"

>>> DNA[:-1] #0-6 indices

ATGCAGC

- Omit the last index and use negative index to select the end of a sting

>>> DNA[-3:] #5-7 indices

GCT

Strings are Immutable

- We can't change individual characters in a string through assignment operator.

>>> DNA = "ATGCAGCT"

>>> DNA[0] = "G" # error

- A new string can be created with new value at zeroth index.

>>> new_str = "G" + DNA[1:]

>>> new_str

GTGCAGCT

Strings Membership

- Use "in" operator to check whether a character or substring exists in another string.

>>> DNA = "ATGCAGCT"

>>> base = "W"

>>> base in DNA

False

>>> base not in DNA

True

>>> not base in DNA

True

Some Useful String Functions

- Find substring:

>>> DNA = "ATGCAGCTTAC"

>>> DNA.find('GCT')

5

>>> DNA.find('TTT')

-1

>>> DNA.index('GCT')

5

```
>>> DNA.index('TTT')
ValueError: substring not found
```

- ***count()*** returns number of non-overlapping occurrences of the substring:

```
>>> DNA = "ATCGCGG"
>>> DNA.count('G')
3
>>> DNA.count('G', 0, 5)
1
>>> seq = "ATCCCCCGG"
>>> seq.count("CC")
2
```

- ***split()*** returns a list of the words after splitting string on a specified delimiter.

```
>>> DNA = 'ATGCTT'
>>> DNA.split('GC')
['AT', 'TT']
>>> DNA.split('GC')[0]
'AT'
```

Or

```
>>> a = DNA.split('GC')
>>> a[0]
'AT'
```

- ***replace()*** returns a copy of string with all occurrences of old substring replaced by a new substring:

```
>>> DNA = "ATCGCGG"
>>> DNA.replace('G', 'U')
ATCUCUU
>>> DNA.replace('G', 'U', 2)
ATCUCUG
```

Lesson-80

Python Loops-II

while Loop

- Loops are used to perform repetitive tasks.
- A *while* loop needs a condition that will eventually becomes False.

```
>>> DNA = "ATGCAGCT"
>>> i = 0
>>> while i < len(DNA):
      print(DNA[i])
      i += 1
   print('outside loop')
```

***for* Loop**

- for loop provides an easy way to access elements of the string.

```
>>> DNA = "ATGCAGCT"
>>> for base in DNA:
      print(base, end="")
   print("\noutside loop")
```

Output:

ATGCAGCT

Outside loop

break, continue, pass,* and loop *else

break: jumps out of the closest enclosing loop.

continue: jumps to the header line of the closest enclosing loop

pass: does nothing at all

Loop else block: this gets executed if and only if the loop is exited normally (i. e., without any break statement)

General Format

```
while <test1>:
        <statements1>
        if <test2>:
```

```
        break      #Exit loop now, skip else
    if <test3>:
        continue   #Go to test1 of top loop
else:          #Run if didn't hit a 'break'
    <statements2>
```

- Usually *break* and *continue* are nested further inside *if* to specify an action in response to a certain condition.
- *break* and *continue* can appear anywhere inside the body of the while or for loop.

```
while True:
    <statements>
    break      #Exit loop now
while True:
    continue   #Go to header of while loop
    <statements>
```

- The <statements> after the continue never gets executed.

Example: *continue*

- continue jumps to the header line of the closest enclosing loop.
- For example, continue will skip all odd and only even gets printed.

```
x = 10
while x:      # x = 0 will terminate loop
   x = x−1        # Or, x -= 1
   if x % 2 != 0:
      continue    # Odd -- skip print
   print(x)
```

Example: break

- *break* jumps out of the closest enclosing loop. In the example, *break* will terminate the loop when user enters "stop".

```
while True:
   name = input('Enter name:')
   if name == 'stop':
      break
```

```
age = input('Enter age: ')
print('Age is', int(age) ** 2)
```

pass

- *pass*: does nothing at all. It can be used when the syntax requires a statement and you don't have any useful statement.

```
while True:  # infinite loop
    pass
```

OR

```
x = 1
if x > 0:   #x is greater than zero--True
   pass
```

OR

```
def testFunc():
    pass
```

Loop *else*

- Loop *else*: this gets executed if and only if the loop is exited without any *break*.

```
def dnaDt(dna):
   for base in dna:
            if base not in 'ATCG':
          print('Invalid')
                    break
   else:
            print('Valid DNA')
dnaDt('ATGCTGAC')  # printed: Valid DNA
dnaDt('ATGWGT')    # Printed: Invalid
dnaDt('12344')     # Printed: Invalid
```

Lesson-81

Regular Expressions

Regular Expressions

- Regular expressions are also referred as "REs" or "regex" or "regexp".

- Use RE to specify the rules for the set of possible strings that you want to match.
- It's a powerful string manipulation tool to search, replace, and parse complex patterns of characters or words.
- Strings methods for searching (***index***, ***find***, and ***count***), replacing (***replace***), and parsing (***split***) use hard-coded substrings and are case-sensitive.
- REs offers a flexible way to:
  - Search a string (methods: ***search*** and ***match***)
  - Replace parts of a string (method: ***sub***)
  - Break strings into smaller pieces (method: ***split***)
- REs are very useful in bioinformatics. We can use them to find DNA and Protein patterns
- DNA patterns
  - restriction sites
  - promoters/transcription factor binding sites
- Protein patterns
  - conserved domains (motifs)
  - active sites

Pattern and String Description

- **Pattern:** this is the regular expression to be matched
- **String:** This is the string, which would be searched to match the pattern anywhere in the string.

REs Syntax

- A pattern can be just a text string: 'GATC'
- It can have alternative characters: 'G[AT]TC'

The 'G[AT]TC' pattern will match a DNA string that starts with G; has A or T in the 2nd position, followed by TC.

- We can specify characters individually or as a range. For example 'G[abc]TC' or 'G[a-c]TC'
- A caret (^) symbol at the start of the character class matches any one character except those given in the square brackets **'G[^AT]TC'**

  This **'G[^AT]TC'** pattern will match a DNA string that starts with G; has C or G in the 2nd position, followed by TC. Here: **'GCTC'** or **'GGTC'**.
- A pattern can contain a wildcard (a dot) that matches any single character except newline: **'G.TC'**

Repeated Things

- x* matches zero or more number of x

'ca*' matches 'c', 'ca', 'caa', etc.

- x+ matches one or more number of x

  'ca+' matches 'ca', 'caa', etc.

- x? matches zero or one x

  'home-?work' matches 'homework' or 'home-work'

- x{m, n} matches i number of x, where m≤i≤ n 'a{2,3}' matches 'aa' or 'aaa'

Repeated Things: Equivalence

- *x{m, n}* matches i number of *x*, where *m≤i≤ n* **'a{2,3}' matches 'aa' or 'aaa'**
- *x*{0,} is same as *x**
- *x*{1,} is same as *x*+
- *x*{0,1} is same as *x?*

REs Syntax

- Parentheses can be used for grouping '(abc)+' matches 'abc', 'abcabc', 'abcabcabc'.
- A vertical line "|" means "or"; it allows you to look for either of two completely different patterns:

  'GAAT|ATTC'

Lesson-82

Regular Expressions-II

How to Use REs in Python?

- In order to use regular expressions in the code, we need to import *re* module.
- The two basic methods *re.search()* and *re.match()* are used to search a pattern.
  - Search looks for a pattern anywhere in a string
  - Match looks for a match at the beginning
- Both methods return None if the pattern isn't found and a "match object" instance if it is.

Search and Match

```
import re
pat = 'a*b'
re.search(pat, 'fooaaabcde')
<_sre.SRE_Match object; span=(3, 7), match='aaab'>
re.match(pat, 'fooaaabcde')
None
```

```
re.match('fooa*b', 'fooaaabcde')
<_sre.SRE_Match object; span=(0, 7), match='fooaaab'>
```

Match Object

- It is an instance of the match class with the details of the match result.

```
import re
res = re.search('a*b', 'fooaaabcde')
res.group()  # group returns string matched
'aaab'
res.start()  # index of the match start
3
res.end()    # index of the match end
7
res.span()   # tuple of (start, end)
(3, 7)
```

- It is an instance of the match class with the details of the match result.

```
import re
res = re.match('fooa*b', 'fooaaabcde')
res.group()  # group returns string matched
‘fooaaab'
res.start()  # index of the match start
0
res.end()    # index of the match end
7
res.span()   # tuple of (start, end)
(0, 7)
```

***split()* method**

- The re.split() is like split but can use patterns.

```
re.split('\W+', 'This... is a test,  short and sweet, of split()')
   ['This', 'is', 'a', 'test', 'short', 'and', 'sweet', 'of', 'split', '']
```

The '\W+' means split on all non-alphanumeric characters.

sub() and _findall()_ methods

- ***re.sub()*** substitutes all occurrences of RE pattern in string with a new pattern.

re.sub('(blue|white|red)', 'black', 'blue socks and red shoes')

'black socks and black shoes'

- ***re.findall()*** finds all matches and returns them as a list

re.findall('\d+','12 dogs,11 cats, 1 egg')

['12', '11', '1']

Compile Patterns

- If you plan to use a re pattern more than once, compile it to a ***re*** object

```
import re
p = re.compile('a*b')
res = p.search('fooaaabcde')
res.group()  # group returns string matched
'aaab'
res.start()  # index of the match start
3
res.end()    # index of the match end
7
res.span()   # tuple of (start, end)
(3, 7)
```

Backslash and Raw Strings

- RE use backslash character ('\') to indicate special meanings. However, if we want to use them without their special meaning then we have escape them either using backslashes or raw strings (preferable).

Regular String	Raw string
"ab*"	r"ab*"
"\\\\section"	r"\\section"
"\\w+\\s+\\1"	r"\w+\s+\1"

Lesson-83

Strings

Ways to Code Strings?

- So far we have declared strings that are either enclosed in single (‘’) or double (“”) quotation marks.
- Strings can declared with triple quotation marks (single or double). These are called block strings or triple-quoted strings.
- Python concatenates all lines together into a single line, and end-of-line (‘\n’) character is added where line breaks.

Block Strings: Example

```
dna = ”””abcd
efgh
Ijkl”””
print(dna)
print(dna.split('\n'))
```

Output:

```
abcd
efgh
ijkl
['abcd', 'efgh', 'ijkl']
```

Multiline Comments: Triple Quotes

- Remember: single line comment starts by adding # symbol at the start of a line.
- We can also use triple quotes (single or double) to add multi line comments in the code.

```
'''
this is a multi-line
comment'''
dna = ”””abcd
efgh
Ijkl”””
print(dna)
```

Implicit and Explicit Concatenation

- Explicit: using '+' operator
- Implicit: python automatically concatenates adjacent strings.

```
s = "Bio" "infor" "matics"
print(s)
Bioinformatics
ss = "Bio"+"infor"+"matics"
print(ss)
Bioinformatics
```

Escape Sequences

- Escape sequences are used to introduce an alternative interpretation of a series of characters.
- In Python, backslash is used to introduce escape sequences in the code. Backslash allows us to add difficult to type characters into the strings.
- Some examples you already know: '\n', '\t'

Escape	Meaning
\\	Backslash (stores one \)
\'	Single (stores ')
\"	Double (stores ")
'\n'	Newline
'\t'	Horizontal tab
'\v'	Vertical tab
'\b'	Backspace

Raw Strings

- Problems using escape sequences

```
>>> path = "C:\new\test.txt"
>>> print(path)
C:
ew      est.txt
```

- Use letter 'r' or 'R' to suppress the escape sequences

Raw Strings Suppress Escape Sequences

- Examples of raw strings:

```
>>> path = r"C:\new\test.txt"

>>> print(path)

C:\new\test.txt

>>> path = R"C:\new\test.txt"

>>> print(path)

C:\new\test.txt
```

Alternative Way to Suppress Escape Sequences

- An alternative way is to use double backslash (not a raw string).

```
>>> path = "C:\\new\\test.txt"

>>> print(path)

C:\new\test.txt
```

Functions That Return a Value

- We can write functions that can return a single or multiple values. The value can be numbers, strings, lists, dictionaries. e. g.,

```
def add_s(a, b):
    return a+b,a,b

ss,a,b = add(3,4)

print(ss,a,b)

7 3 4
```

Lesson-84

Lists

Lists

- List are ordered collection of values. Each value is called an element.
  - Left to right order among elements.
  - Can contain any object types such as number, strings, lists, dictionaries etc.
  - Like strings, these are sequences. Sequence methods that were used for strings can also be used for lists.
  - Lists are similar to Arrays. However, size of the list is not fixed and list can handle multiple data types.

Ways to Create a List

- Square brackets are used create a list

```
>>> [10,20] #list of two integers
[10,20]
>>> ["A", "g", "T"] #three strings
['A', 'a', 'T']
>>> ["A",10,["T",20],30.0] #diff.types
['A', 10, ['T', 20], 30.0]
>>> [] #empty
[]
```

Assigning List to Variables

- Like strings, we can assign lists to variables.

```
>>> a = ["A", "T"] #list
>>> b = [] #empty list
```

- We can pass lists to functions.

```
>>> print(a,b) # passing to functions
['A', 'T'] []
>>> type(a) #type is list
<class 'list'>
```

***while* Loop**

- Loops are used to perform repetitive tasks.
- A *while* loop needs a condition that will eventually becomes False.

```
>>> DNA = ["A", "T", "C", "G"]
>>> i = 0
>>> while i < len(DNA):
        print(DNA[i], end=",")
        i += 1
    A,T,C,G,
```

***for* Loop**

- for loop provides an easy way to access elements of the string.

```
>>> DNA = ["A", "T", "C", "G"]
>>> for base in DNA:
```

```
    print(base, end=",")
  print("\noutside loop")
```

Output:

A,T,C,G,

Outside loop

Ways to Create List of Integers

- range() function can be used in many ways to create list of consecutive integers.

```
>>> range(5) #one argument
range(0,5)
>>> range(1,5) #two arguments
range(1,5)
>>> range(1,10,2) #Three argument
range(1,10,2)
```

- Third argument is called step size that specifies the numbers of digits to skip.
- We can use loop to iterate the newly created lists.

```
>>> for i in range(5):
      print(i,end=",")
0,1,2,3,4,
>>> for i in range(1,10,2):
      print(i,end=",")
1,3,5,7,9,
```

Finding Length of a List

- Use ***len()*** function to find the number of elements in a list.

```
>>> DNA = ["A", "T", "G", "C"]
>>> len(DNA)
4
>>> DNA = [] #empty list
>>> len(DNA)
0
```

- Like strings, ***len()*** function returns zero for the empty lists

Length of a List of Lists

- Nested lists are counted as a single element by the *len()* function.

```
>>> DNA = ["A", "T",["T", "G"]]
>>> len(DNA)
3
>>> len([[90,3],["T",20]])
2
```

List Indexes and Slices

- A single element and a part of a list is accessed by indexing and slice operator, respectively.
- List index starts from zero.

```
>>> DNA = ["A", "T", "C", "G"]
>>> DNA[0] # first element
'A'
>>> DNA[3] # fourth element
'G'
```

List Negative Indices

```
>>> DNA = ["A", "T", "C", "G"]
>>> DNA[-1]    #last element
'G'
>>> DNA[-4] #first element
'A'
>>> DNA[-5]   #out of bound
IndexError: list index out of range
```

List Slices (Segments)

- Use a range to specify a slice (sub-list)
- **Format:** *sample[start:stop:step]*
- **start:** the beginning index of the slice, it will include the element at this index unless it is the same as *stop*, defaults to 0, i.e. the first index. If it's negative, it means to start n items from the end.
- **stop:** the ending index of the slice, it does *not* include the element at this index, defaults to length of the sequence being sliced, that is, up to and including the end.

- **step:** the amount by which the index increases, defaults to 1. If it's negative, you're slicing over the iterable in reverse.

```
>>> DNA = ["A", "T", "C", "G"]
>>> DNA[0:2]    # first two
['A', 'T']
>>> DNA[2:]    # last two
['C', 'G']
>>> DNA[50:100] # out of range
[]
```

Lesson-85

Lists-II

List Operations

- Concatenate

```
>>> dna_at = ["A", "T"]
>>> dna_cg = ["C", "G"]
>>> print(dna_at + dna_cg)
['A', 'T', 'C', 'G']
```

- Repeat

```
>>> ["A"]*4
['A', 'A', 'A', 'A']
```

List Membership Test

- “in” operator is used to check membership.

```
>>> "W" in ["A","T","C","G"]
False
>>> "W" not in ["A","T","C","G"]
True
>>> "A" in ["A","T","C","G"]
True
```

Strings and Lists

- We discussed split() function that returns a list.

```
>>> DNA = "A T C G"
>>> DNA.split()
['A', 'T', 'C', 'G']
>>> a = DNA.split()
>>> type(a[0])
<class 'str'>
```

- We can use join() function to convert a list to string. It's opposite of split().

```
>>> DNA = ["A", "T", "C", "G"]
>>> print("".join(DNA))
'ATCG'
```

Lists are Mutable: Updating

- Unlike strings, one or more than one elements of the lists can be changed using assignment operation.
- Index assignment: we can update single element using both positive and negative indexes.

```
>>> DNA = ["A", "T", "C", "G"]
>>> DNA[0] = "T"
>>> DNA[-2] = "G"
>>> print(DNA)
['T', 'T', 'G', 'G']
```

Strings to list conversion: *list()*

- Built-in list() function can be used to convert strings to lists

```
>>> DNA = "ATGC" #change A to G
>>> DNA = list(DNA)
>>> print(DNA)
['A', 'T', 'G', 'C']
>>> DNA[0] = "G"
>>> print("".join(DNA))
'GTGC'
>>> list(range(5))
```

[0,1,2,3,4]

Methods to Grow List

- New elements can also be inserted into Lists to increase it's size.
- Objects can be added to the end of the list using *append()* function.

```
>>> DNA = ["A", "T"]
>>> DNA.append("C")
>>> print(DNA)
['A', 'T', 'C']
>>> DNA.append(["G", "A"])
>>> print(DNA)
['A', 'T', 'C',['G', 'A']]
```

Methods to Delete Elements: *del*

- Built-in *del* function can be used.

```
>>> DNA = ["A", "T", "C", "G"]
>>> del DNA[0]   # single
>>> print(DNA)
['T', 'C', 'G']
>>> del DNA[0:2]   # slice
>>> print(DNA)
['G']
>>> DNA = ["A", "T", "C", "G"]
>>> del DNA[-1]   # single
>>> print(DNA)
['A', 'T', 'C']
```

Methods to Delete Elements: *pop()*

- Built-in pop() function deletes an element by index.

```
>>> DNA = ["A", "T", "C", "G"]
>>> DNA.pop()   #deletes from end
'G'
>>> print(DNA)
```

['A', 'T', 'C']

>>> DNA.pop(1) #deletes from middle

'T'

>>> print(DNA)

['A', 'C']

Lesson-86

Lists-III

STACK – LIFO

- Stack concept: Last In First Out (LIFO)
- The *append()* and *pop()* functions can be used together.

>>> DNA = ["A", "T", "C"]

>>> DNA.append("G")

['A', 'T', 'C', 'G']

>>> DNA.pop()

'G'

>>> DNA

['A', 'T', 'C']

Methods to Delete Elements: *remove()*

- Built-in remove() function takes a single argument. The argument will be the element to remove.

>>> DNA = ["A", "T", "C", "G"]

>>> DNA.remove("A")

>>> DNA

['T', 'C', 'G']

>>> DNA.remove("t")

ValueError: list.remove(x): x is not in list

- Only the first occurrence of the element is removed from the list.

>>> DNA = ["A", "T", "C', "G", "A"]

>>> DNA.remove("A")

>>> DNA

['T', 'C', 'G', 'A']

Other Useful Methods

- The minimum, maximum, number of occurrences, and sum of the elements of list.

```
>>> a = [1,4,20,2,5]
>>> sum(a) # add all elements
32
>>> min(a) # find minimum value
1
>>> max(a) # find maximum value
20
>>> a.count(2) # count occurrences
1
```

- We can also use slices with these methods.

```
>>> a = [1,4,20,2,5]
>>> sum(a[3:]) # add all elements
7
>>> min(a[3:]) # find minimum value
2
>>> max(a[3:]) # find maximum value
5
>>> a[1:].count(1) #count occurrences
0
```

***reverse()* and *index()* Functions**

- The reverse() function will reverse the list.

```
>>> a = [1,4,20,2,5]
>>> a.reverse() # reverse order
>>> print(a)
[5,2,20,4,1]
```

- The index() function will find the index of the element. Returns error when not found.

```
>>> a = ["A", "T", "C"]
>>> a.index("T")
```

```
1
 >>> a.index("G")
ValueError: 'G' not in list
```

Matrix

- The matrix can be declared in the form of nested lists.

```
>>> b = [[4,5],[2,3],[8,0]]
>>> b[0]
[4,5]
>>> b[0][0]
4
```

Sorting Lists: sort() function

- The *sort()* function is used for lists.

```
>>> a = [1,4,20,2,5]
>>> a.sort() # ascending order
[1,2,4,5,20]
>>> a.sort(reverse=True) #descending
[20,5,4,2,1]
```

  - Can be used with nested lists.

```
>>> b = [[4,5],[2,3],[8,0]] #nested
>>> b.sort() #by zero index
[[2,3],[4,5],[8,0]]
```

  - We can specify index as well for sorting.

```
>>> from operator import itemgetter
>>> b = [[4,5],[2,3],[8,0]]
>>> b.sort(key=itemgetter(1))
[[8,0],[2,3],[4,5]]
>>> b.sort(key=itemgetter(1),\
        reverse=True)
[[4,5],[2,3],[8,0]]
```

- Alternatively, sorted() function can also be used. Unlike sort(), it won't change the original list.

```
>>> from operator import itemgetter
>>> b = [[4,5],[2,3],[8,0]]
>>> sorted(b)
[[2,3],[4,5],[8,0]]
>>> print(b)
[[4,5],[2,3],[8,0]]
>>> sorted(b, reverse=True)
[[8,0],[4,5],[2,3]]
```

- Alternatively, *sorted()* function can also be used.

```
>>> from operator import itemgetter
>>> b = [[4,5],[2,3],[8,0]]
>>> sorted(b,key=itemgetter(1))
[[8,0],[2,3],[4,5]]
>>> sorted(b, key=itemgetter(1),\
        reverse=True)
[[4,5],[2,3],[8,0]]
```

Lesson-87

Files

File, Directory, and Path??

- A file is a named entity that is used to read and write data from/to a hard drive.
- Directory/folder: is an organizing unit in computer's file system for storing and locating files. It can contain both subdirectories and files.
- Path: a sequence of directory names that specifies the exact location of the file.

Text Files

- Files are used to read and write data from/to a hard drive.
- Text File: a text file is a sequence of lines that are separated by newline characters.

Files: *open()* function

- In Python, we specify the name of the file and tell interpreter whether we want to read it or write it.
- Python has an open() function that is used to read or write files.

- Function open() returns a handle (file object) used to perform operations on the file.

How to Open Files?

- Function open takes **two arguments**.
  - Name and location of the file
  - Mode (optional): read, write, append etc.
  - Syntax: **fout = open(filename, mode)**
  - **fout:** is a new file object. You can use any name instead of fout.
  - **filename:** a string specifying the name and the location of the file.
  - **Mode:** special characters to specify what to do with the file. Default is to read a file if you skip this argument.

Reading Files - Assign Handle

- *Default mode is read.* We can either skip it or specify 'r' to read a file.

```
# read a file without a mode (default is 'r')
fr = open('test.txt')
```

or

```
fr = open('test.txt','r')
print(fr)
```

- If only file name is listed and not full location on the system, Python tries to find it in *current working directory (where your source code is).*

Read Entire File as String

File: Test.txt

```
>fasta sequence
ATGCATGCGCTGC
CGTCGCGTACGTA
```

```
fr = open('test.txt','r')
a = fr.read()
print(a)
```

- *reads entire file as a string*. We can split on newline character to get individual lines.

Read *N* Characters from File

File: Test.txt

```
>fasta sequence
```

ATGCATGCGCTGC

CGTCGCGTACGTA

```
fr = open('test.txt','r')
a = fr.read(7) # read first seven
print(a)
a = fr.read(15) # read next 15
print(a)
```

Reading Files with a *for* loop

- The lines from a file object can be read using a for loop.
  - Open a file in read mode
  - Read each line using a for loop
  - Perform actions on lines

```
fr = open('test.txt','r')
for line in fr:
  print(line)
```

Output:

```
>fasta sequence
  ATGCATGCGCTGC
  CGTCGCGTACGTA
```

Reading Files and *print()* Function

- The *print()* function adds a newline character to each line. Also, each line from the file has a new line.
- We can use *rstrip()* function to remove whitespaces from the right side of the string.

```
fr = open('test.txt','r')
for line in fr:
  print(line.rstrip())
```

Output:

```
>fasta sequence
  ATGCATGCGCTGC
  CGTCGCGTACGTA
```

Reading Files with a *for* loop

- We can apply conditions inside *for* loop to process only the desired lines.

```
fr = open('test.txt','r')
for line in fr:
    if not line.startswith('>'):
        print(line.rstrip())
```

Output:

```
ATGCATGCGCTGC
CGTCGCGTACGTA
```

Lesson-88

Files-II

Reading Files with *readline()*

- The *readline()* function reads next line from the file as a string.

```
fr = open('test.txt','r')
print(fr.readline().rstrip())
print(fr.readline()[:-1])
print(fr.readline().replace('\n',''))
fr.close()
```

Output:

```
>fasta sequence
ATGCATGCGCTGC
CGTCGCGTACGTA
```

Using *print()* Argument to Suppress newline

- We can also use *end* argument to suppress newline of *print()*.

```
fr = open('test.txt','r')
print(fr.readline())
print(fr.readline(),end='')
print(fr.readline(),end='')
```

Output:

```
>fasta sequence
```

ATGCATGCGCTGC

CGTCGCGTACGTA

***while* Loop to Read File**

- The while will terminate when the line reaches end of file

```
fr = open('test.txt','r')
line = fr.readline()
while line:
print(line.rstrip())
line = fr.readline()
```

Output:

>fasta sequence

ATGCATGCGCTGC

CGTCGCGTACGTA

Reading Files with *readlines()*

- The *readlines()* function reads the whole file's content as a list of lines.

```
fr = open('test.txt','r')
all_lines = fr.readlines()
print(all_lines)
```

Output:

['>fasta sequence\n', 'ATGCATGCGCTGC\n', 'CGTCGCGTACGTA\n']

```
fr = open('test.txt','r')
all_lines = fr.readlines()
for line in all_lines:
print(line,end='')
```

Output:

>fasta sequence

ATGCATGCGCTGC

CGTCGCGTACGTA

Writing Files

- The *fout = open('filename','w')* opens a file to write. If a file already exists, it deletes all previous content from the file.

```
fout = open('test_write.txt','w')
fout.write('This is an example\n')
fout.close()
```

- The *write()* function writes a string to the file. Use newline if you want to write line by line.
- The *close()* function saves the file once writing is complete and makes it available for other uses.

Writing Files: append mode

- The *fout = open('filename','a')* opens a file to write. If a file already exists, the new data is appended to previous content in the file.

```
fout = open('test_write.txt','a')
fout.write('This is a rewrite\n')
fout.close()
```

- The *write()* function writes a string to the file. Use newline if you want to write line by line.
- The *close()* function saves the file once writing is complete and makes it available for other uses.

Lesson-89

Tuples

Tuple

- Tuple are written as series of items in parenthesis (not square brackets).
- Tuple is a compound data type (collection of other data types or objects).
- These *immutable* – cannot be changed in-place. Similar to strings.
- Fixed size – if you want to modify it create a new tuple
- Like lists and strings, tuples are ordered collection of objects that **maintain left to right order**.
- Items are accessed by position based indexing just like in Lists and Strings. Tuple **support indexing and slicing operations**.
- Works exactly like lists and strings in terms of methods and operations.

Ways to Create a Tuple

- A comma after the single item, before the closing parenthesis tells Python that it's a single item tuple and not a regular expression (As we can write expression in parenthesis; we need to differentiate).

```
a = () # an empty tuple
```

```
x = (40) # An integer
y = (40,) # A one item tuple
z = (40,50) # A two item tuple
```

Examples

```
x = (40)
type(x)
```

OUTPUT: <class 'int'>

```
y = (40,)
type(y)
```

OUTPUT: <class 'tuple'>

```
x = (0,[4],1.2,'3')
print(x)
```

OUTPUT: (0, [4], 1.2, '3')

Nested Tuple

- Tuple allows nesting of different data types.

```
x = ('ATG',('ATA', 'TAA')) # tuple
print(x)
```

OUTPUT: ('ATG', ('ATA', 'TAA'))

```
y = (['ATA', 'TAA']) # a tuple???
print(type(y))
```

OUTPUT: <class 'list'>

```
z = (['ATA', 'TAA'],) #tuple?
print(type(z))
```

OUTPUT: <class ‘tuple'>

Index and Index of Index

- *Index & Index of index:* similar to strings and lists

```
x = ('ATG', 'ATA', 'TAA') # tuple
print(x[0],x[-1])
```

OUTPUT: ATG TAA

```
y = (('ATG','start'),('TAA','stop'))
print(y[0])
OUTPUT:('ATG', 'start')
print(y[0][1])
OUTPUT: start
  print(y[-1][1])
OUTPUT: stop
```

Slicing

- Similar to strings and lists. *Format: T[start:end]*

```
x = ('ATG', 'ATA', 'TAA', 'TAG')
print(x[:])
OUTPUT: ('ATG', 'ATA', 'TAA', 'TAG')
print(x[:3])
OUTPUT: ('ATG', 'ATA', 'TAA')
print(x[1:])
OUTPUT: ('ATA', 'TAA', 'TAG')
print(x[:-2])
OUTPUT: ('ATG', 'ATA')
print(x[5:6])
OUTPUT: ()
```

Length and Concatenate

- The *len()* method returns the number of elements in the tuple.

```
x = ('ATG', 'ATA')
print(len(x))
OUTPUT: 2
```

- The "+" operator will combine two tuples.

```
 y = ('TAA', 'TAG')
 print(x + y)
 OUTPUT: ('ATG', 'ATA', 'TAA', 'TAG')
```

Iteration

- We can use *for* and *while* loops.

```
x = ('ATG', 'ATA', 'TAA', 'TAG')
for entry in x:
    print(entry, end=',')
OUTPUT: ATG,ATA,TAA,TAG,
i = 0
while i < len(x):
    print(x[i], end=',')
    i += 1
OUTPUT: ATG,ATA,TAA,TAG,
```

Conversions: List to/from Tuple

```
T = ('cc', 'aa', 'dd', 'bb')
tmp = list(T) # Make a list from tuple
OUTPUT: ['cc', 'aa', 'dd', 'bb']
tmp.sort() # Sort the list
print(tmp)
OUTPUT: ['aa', 'bb', 'cc', 'dd']
T = tuple(tmp) # Make a tuple from list
print(T)
OUTPUT: ('aa', 'bb', 'cc', 'dd')
```

Conversions

- Converting string to tuple and sorted method.

```
T = tuple('spam')
print(T)
OUTPUT: ('s', 'p', 'a', 'm')
print(sorted(T)) # sorted built-in
OUTPUT: ['a', 'm', 'p', 's']
```

Lesson-90

Dictionaries

Dictionary

- The compound data types such as lists and strings use integer indices to access elements.
- Dictionaries are also compound data types (collection of other data types or objects).
- Dictionary are known as *mappings* because these simply map keys to their associated values (*"key-value"* pairs).
- These are useful whenever we need to associate a set of values with keys. e.g., name and phone number; protein name and sequence.
- Dictionary don't maintain left to right order. In fact no order at all because we will use keys to access them.
- These are ***mutable*** – can update elements, remove, and add new elements.

Ways to Create a dictionary

- Start with an empty dictionary and add elements.

```
a = {} #this step creates an empty dictionary
a['one'] = 1      #add 1st element
a['two'] = 'two'  #add 2nd element
a['three'] = 3.0   #add 3rd element
a['four'] = []    #add 4th element
print(a)
{'four': [], 'one': 1, 'three': 3.0, 'two': 'two'}
```

- Provide a list of "key:value" pairs.

```
a = {'four': [], 'one': 1, 'three': 3.0, 'two': 'two'}
print(a)
{'four': [], 'two': 'two', 'three': 3.0, 'one': 1}
```

- Note: key-value pairs are not in order.

Key and Values

- Elements (i.e., key-value pairs) in the dictionary are separated by commas.
- Each key can occur only once. Dictionary holds unique elements only.
- A key can be any immutable type (string, numbers, tuples). List is mutable and cannot be a key.

Keys are unique

- Each key can occur only once. Dictionary holds unique elements only.

```
a = {'four': [], 'two': 'two', 'three': 3.0, 'one': 1}
a['four'] = 4
print(a)
{'four': 4, 'two': 'two', 'three': 3.0, 'one': 1}
```

Mutable containers can't be Key

- A key can be any immutable type (string, numbers, tuples). List is mutable and cannot be a key.

```
a = {'four': [], 'two': 'two', 'three': 3.0, 'one': 1}
a[['A']] = 'a'
TypeError: unhashable type: 'list'
```

Remove Elements: del method

- The *del* statement removes a key-value pair from the dictionary. It's similar to *del* in list except that it accepts key not index.

```
a = {'four': [], 'two': 'two', 'three': 3.0, 'one': 1}
del a['four']
print(a)
{'three': 3.0, 'two': 'two', 'one': 1}
del a['Three']
KeyError: 'Three'
```

Remove Elements: pop method

The pop() function removes a key from the dictionary and returns it's value. It takes a key as an argument.

```
a = {'four': [], 'two': 'two', 'three': 3.0, 'one': 1}
a.pop('four')
[]
a.pop()
TypeError: pop expected at least 1 arguments, got 0
```

Finding the Number of Elements

- The len() function returns the number of key-value pairs.

```
a = {'four': [], 'two': 'two', 'three': 3.0, 'one': 1}
```

```
print(len(a))
```

4

```
dna = {}
print(len(dna))
```

0

Lesson-91

Sets

Sets

- An *unordered* collection of *unique and immutable* objects.
- Set class in Python is implemented using dictionaries so *all requirements are similar to dictionary keys*.
- Unique items: an item appears only once in a set regardless of how many times it is added.
- Sets ***cannot contain mutable types*** such as dictionaries and lists as elements. However, it can contain immutable types such as tuples.
- It is a compound data type and can contain variety of other objects or data types.
- Sets are traversable, can grow and shrink in size.
- These are neither sequences nor mappings.
- Sets are like valueless dictionaries (i. e., no key-pair mappings) in which elements do not maintain an order.
- Do not support indexing and slicing operations because these are not sequences.

How to Create a Set?

- To make a set object, pass in a sequence or other iterable object to the built-in *set()* function. Syntax: *set([iterable])*

```
a = set() # an empty set
print(a)
```

set()

- The iterable argument is optional. If it's not provided Python creates an empty set.

Unordered Collection and Optional Argument

```
a = set('spam') # string to set
print(a)
```

{'p', 'm', 'a', 's'}

- Note: Curly brackets, sets are more similar to dictionary than other objects.

```
y = set(list('happy')) # list to set
print(y)
{'p', 'h', 'a', 'y'}
```

- Note: duplicates are removed and no order among items.

Size of the Set Can Grow: Add an Element

```
a = set('spam') # string to set
print(a)
{'p', 'm', 'a', 's'}
a.add('k')
print(a)
{'p', 'm', 'a', 's', 'k'}
```

- The *add()* function can be used to add new element to the set.

Size of the Set Can Grow: Merging

```
a = set('spam') # string to set
print(a)
{'p', 'm', 'a', 's'}
a.update(set('xy'))
print(a)
{'p', 'a', 'y', 'm', 'x', 's'}
```

- The *update()* method or "|=" is used to merge two sets.

Size of the Set Can Shrink

- Various functions are available to delete elements from the sets.

s.remove(x): removes element x from set s; raises KeyError if not present.

s.discard(x): removes x from set s if present. All or only one x?

s.pop(): remove and return an arbitrary element from s; raises KeyError if empty

s.clear(): remove all elements from sets

- As an exercise, try these functions on sets are check their functionality.

Set Operations

Operation	Equivalent	Result
len(s)		cardinality of set *s*
x in s		test *x* for membership in *s*
x not in s		test *x* for non-membership in *s*
s.issubset(t)	s <= t	test whether every element in *s* is in *t*
s.issuperset(t)	s >= t	test whether every element in *t* is in *s*
s.union(t)	s | t	new set with elements from both *s* and *t*
s.intersection(t)	s & t	new set with elements common to *s* and *t*
s.difference(t)	s - t	new set with elements in *s* but not in *t*
s.symmetric_difference(t)	s ^ t	new set with elements in either *s* or *t* but not both
s.copy()		new set with a shallow copy of *s*

S1 = {1, 2, 3, 4}

S1 & {1, 3} # Intersection

{1, 3}

{1, 5, 3, 6} | S1 # Union

{1, 2, 3, 4, 5, 6}

S1 - {1, 3, 4} # Difference

{2}

S1 > {1, 3} # Superset

True

Lesson-92

Arguments

Defining New Functions

‘’

Syntax for function definition

>>> def any_name(list of parameters):

statement one

---

statement last

return answer

Parameters: argument required to make function work.

Calling a Function

- The syntax for a function call is:

```
>>> def multiply(x, y):
    return x * y
>>> multiply(5, 4)
 20
```

- Argument is a value passed to a function (or method) when calling the function.

Function Arguments

- There are two types of arguments:
- *keyword argument*: an argument preceded by an identifier (e.g. name=) in a function call or passed as a value in a dictionary preceded by **.
- *positional argument*: an argument that is not a keyword argument. Positional arguments can appear at the beginning of an argument list and/or be passed as elements of an iterable preceded by *.

Keyword Arguments

- We can call a function with some or all of its arguments out of order as long as we specify their names

```
def example(x, y, z):
 return x-y
example(2, 1, 43)
1
example(z=43, y=1, x=2)
1
example(2, z=43, y=1)
1
example(z=43,2,y=1)
SyntaxError: non-keyword arg after keyword arg
example(**{'x'=2, 'y'=1, 'z'=43})
1
```

Positional Arguments

Positional arguments can appear at the beginning of an argument list and/or be passed as elements of an iterable preceded by *.

```
def example(x, y, z):
 return x-y

example(2, 1, 43)

1

example(*(2,1,43))

1

example(*[2,1,43])

1
```

Python's Higher-Order Functions

- Python supports higher-order functions that operate on lists or iterable.
- The *map()* method applies a given function to each element of a sequence. It returns a new sequence that holds the return values of each of the function calls. *map(function, sequence)*.
- The *filter()* method takes a function that returns true or false. Applies the function to each item in a list. A new list is returned that contains all the items for which the function was true. filter(function, sequence)

```
def square(x):
    return x*x
def even(x):
    return 0 == x % 2
list(map(square, range(10,20)))
  [100, 121, 144, 169, 196, 225, 256, 289, 324,          361]
  list(filter(even, range(10,20)))
  [10, 12, 14, 16, 18]
list(map(square, filter(even, range(10,20))))
  [100, 144, 196, 256, 324]
```

- More than one sequence may be passed; the function must then have as many arguments as there are sequences.
- The function is called with the corresponding item from each sequence (or None if some sequence is shorter than another).

```
seq = range(8)
def add(x, y):
    return x+y
map(add, seq, seq)
```

[0, 2, 4, 6, 8, 10, 12, 14]

Lesson-93

Comprehensions

List Comprehensions

- A list *comprehension* is a programming language construct for creating a list based on existing lists.
- A powerful and popular feature in Python
  - Generate a new list by applying a function to every member of an original list
- Python's notation:

  [expression for name in list]

```
>>> li = [3, 6, 2, 7]
>>> [elem*2 for elem in li]
[6, 12, 4, 14]
```

For each member of the list, the list comprehension

1. Sets variable elem equal to that member,
2. Calculates a new value using expression elem*2,
3. It then collects these new values into a list which is the return value of the list comprehension.

- If list contains elements of different types, then expression must operate correctly on the types of all of list members.
- If the elements of list are other containers, then the name can consist of a container of names that match the type and "shape" of the list members.

```
li = [('a', 1), ('b', 2), ('c', 7)]
print([ n * 3 for (x, n) in li])
[3, 6, 21]
```

- Expression can also contain user-defined functions.

```
def add(a, b):
  return a + b
Tlist = [(6, 3), (1, 7), (5, 5)]
print([add(x, y) for (x, y) in Tlist])
[9, 8, 10]
```

Filtered List Comprehensions

- An *if* clause after the *for* loop allow us to select values based on a certain condition.

```
>>> [x for x in range(5) if x % 2 == 0]
[0, 2, 4]
>>> list(filter((lambda x: x % 2 == 0), range(5))) # using filter method
[0, 2, 4]
res = []   # regular coding
for x in range(5):
    if x % 2 == 0:
        res.append(x)
>>> res
[0, 2, 4]
```

Nested List Comprehensions

- We can code any number of nested *for* loops in a list comprehension, and each may have an optional associated *if* test.
- The general structure of list comprehensions looks like this:

```
[expression for target1 in iterable1 [if condition1]
        for target2 in iterable2 [if condition2]
        for targetN in iterableN [if conditionN]     ]
```

List Comprehensions and Matrixes

- A way to code matrix (i. e., multidimensional array) is to use nested list structures.

```
>>> M = [[1, 2, 3],
        [4, 5, 6],
        [7, 8, 9]]
```

- We can use normal index operations to get rows and particular columns within rows.

```
>>> M[1]
   [4, 5, 6]
>>> M[1][2]
    6
```

List Comprehensions and Files

- The *readlines()* method returns lines with "\n" end-of-line characters at the end.

```
>>> open('myfile').readlines()
 ['aaa\n', 'bbb\n', 'ccc\n']
>>> [line.rstrip() for line in open('myfile').readlines()]
['aaa', 'bbb', 'ccc']
>>> [line.rstrip() for line in open('myfile')]
['aaa', 'bbb', 'ccc']
>>> list(map((lambda line:line.rstrip()), open('myfile')))
['aaa', 'bbb', 'ccc']
```

Dictionary and Sets Comprehensions

```
>>> {base:0 for base in "ATGC"}
   {"T":0, "C":0, "G":0, "A":0}
>>> {base for base in "ATGCA"}
   {"T", "C", "G", "A"}
```

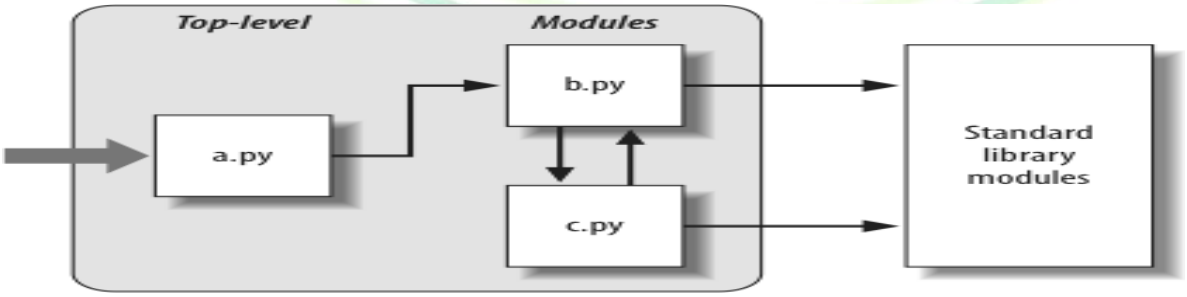
Lesson-94

Modules

Modules

- Script files contain Python variables, statements, and functions (for example name.py)
- In simple terms, every file of Python source code whose name ends in a *.py* extension is a *module*.
- Module is the highest-level program organizational unit. It is the collection of related functions.
- Large programs are divided into multiple modules (or files), that import content from each other.
- One of the module act as the main or top-level file. It is executed to start the entire program.

Program Architecture in Python



File: b.py

```
def spam(text):
  print(text,“spam”)
```

File: a.py

```
import b
b.spam(“test”)
```

Importing Modules

- To import entire module:

import module_name

e.g., import math

math.sqrt(4)

- To import specific functions

from module_name import function_name

- To import all the functions

*from module_name import **

- If functions (rather than module) are imported than there is no need to use the dot notation.

Importing Whole Module

File: module1.py

```
def spam(text):
  print(text, "spam")
def printer(x):
  print(x)
```

```
>>> import module1
>>> module1.printer("welcome!")
welcome!
>>> printer("welcome!")
NameError: name 'printer' is not defined
```

Importing Specific Functions from Modules

File: module1.py

```
def spam(text):
  print(text, "spam")
def printer(x):
  print(x)
```

```
>>> from module1 import printer
>>> printer("welcome!")
welcome!
>>> spam("hello")
NameError: name 'spam' is not defined
>>> module1.spam("hello")
NameError: name 'module1' is not defined
```

How import Works?

- Python performs three steps when a Module is imported.
  - Finding the module file
  - Compiling it to byte code
  - Running to modules code
- The first one is most important because you may need to tell Python where to find files to import.
- You need to know how module search path works in order to customize it.

Lesson-95

Packages

Packages

- Each package in Python is a directory, which must contain a special file called "__init__.py".
- The "__init__.py" file can be empty, and it indicates that the directory it contains is a Python package.
- The package can be imported the same way a module can be imported.
- If we create a directory called foo, which marks the package name, we can then create a module inside that package called bar.
- We need to add the "__init__.py" file inside the foo directory.
- To use the module bar, we can import it in two ways:
  - import foo.bar
  - from foo import bar

Useful Modules/Packages

Plotting: Matplotlib

Scientific: Scipy, Python bindings for R

Web Development: Django, web2py

scikit-learn: classification, regression, clustering, dimensionality reduction, model selection, preprocessing data.

Others: numpy, pandas, os, sys, csv

Visit the following URL for details:
https://pypi.python.org/pypi?%3Aaction=search&term=bioinformatics

Lesson-96

Classes

Everything in Python is Object

- Everything in Python is object.
  - Lists are objects

```
type([])
<class 'list'>
```

  - 42 is an object

```
type(42)
<class 'int'>
```

  - Functions are objects

```
def foo():
  pass
type(foo)
  <class 'function'>
```

Object: collection of data + methods

- Objects are data structures containing data (or attributes or variables) and methods (or functions).

```
s = "ACAGATC" # object of type string
```

  - objects can contain data

```
s = "ACAGCTC"
```

  - It can be modified/manipulated

```
s.count('A')
s.lower()
```

Standard data types *vs.* User-defined types

- Python standard data types

For most simple programs, we can use standard Python data types. E.g., numbers, strings, tuples, lists, sets, dictionaries.

- Defining your own data types

It might be useful though to create your own data types: objects built to your own specifications and organized in the way convenient to you

Object Oriented Programming (OOP)

- Python class is used to implement user-defined data types.
- Classes apply and extend the ideas we have covered so far.
- Classes are packages of functions that use and process built-in object types.

OOP is Optional in Python

- OOP is optional in Python and we don't need to declare a class to get started.
- Classes require some upfront planning, they are more useful for the long-term project development.
- The proper use of classes can cut the development time significantly.

How to Declare Class?

- To declare a class, use class statement followed by a name and a colon.

class <name>:

<body>

- Assignment statements inside class generates class attributes.

class FirstClass:

x = 0

y = 0

Generating Instances and Accessing Attributes

class FirstClass:

x = 0

y = 0

- Calling a class generates it's instances. We can generate as many instances as we want.

p1 = FirstClass()

```
print(p1.x)     # use dot notation to access
0
p1.x = 8
print(p1.x)
8
class FirstClass:
        x = 0
    y = 0
```

- We can generate as many instances as we want.

```
p1 = FirstClass()
p2 = FirstClass()
p1.x = 8
p2.x = 5
print(p1.x, p2.x)
8 5
```

Defining Class Methods

- The function's def statements inside a class generates methods to process data.

```
def name(self, parameter,..,parameter):
        statements
```

- *self* must be the first parameter to any object method. Attributes must be accessed through self.

sign of bright future

Lesson-97

Classes-II

Constructors

- A constructor is a special method with the name *__init__*.

```
def __init__(self, parameter,..,parameter):
        statements
class FirstClass:      # example
    def __init__(self, m, n):
```

```
        self.x = m
        self.y = n
p = FirstClass(3,4)
print(p.x, p.y)
```

3 4

Superclass

- A class can extend the definition of another class
  - Allows use (or extension) of methods and attributes already defined in the previous one.
  - New class is called *subclass*
  - Original class is called *parent*, *ancestor* or *superclass*

```
class subclass(superclass):
```

- Multiple inheritance is also supported.

```
class subclass(superclass,superclass):
class C1 (C2, C3):
        statements
```

- In terms of search tree, an instance inherits attributes from it's class, and a class inherits attributes from all classes above it in the tree.
- Search for a attribute starts from bottom to top, and from left to right.

Redefining Methods in Subclass

- To redefine a method of the superclass, include a new definition using the same name in the subclass.
  - The old code in the superclass won't get executed.

```
class FirstClass:
    x, y = 0, 0
        def add(self, m, n):
            self.x += m
            self.y += n
class SecondClass (FirstClass):
    z = 0
    def add(self, t):
            self.z += t
p = SecondClass()
```

```
p.add(3)
print(p.x, p.y, p.z)
```

0 0 3

Extending Constructors

- Like other methods, we can call constructor of the superclass in subclass.

```
class FirstClass:
        def __init__(self, m, n):
            self.x = m
            self.y = n
class SecondClass (FirstClass):
    def __init__(self, m, n, t):
            FirstClass.__init__(self, m, n)
            self.z = t
p = SecondClass(6, 9, 3)
print(p.x, p.y, p.z)
```

6 9 3

Private Data and Methods

- Any attribute or method with two leading under-scores in its name (but none at the end) is private and can't be accessed outside of class

```
class FirstClass:
        def __init__(self, m, n):
            self.__x = m
            self.__y = n
p = Firstlass(6, 9)
print(p.__x, p.__y)
```

AttributeError: 'FirstClass' object has no attribute '__x'

Private Methods

- **Private methods.**

```
class FirstClass:
        def __init__(self, m, n):
            self.__x = m
```

```
        self.__y = n
    def __add(self):
        return self.__x + self.__y
    def getHidden(self):
        print(self.__add())
p = Firstlass(6, 9)
p.__add()
AttributeError: 'FirstClass' object has no attribute '__add'
p.getHidden()
15
```

Positional and Keyword Arguments

- The positional and keyword arguments hold in case of classes as well.

```
class FirstClass:
        def __init__(self, m, n):
            self.x = m
            self.y = n
p = Firstlass()
TypeError: __init__() missing 2 required positional arguments: 'm' and 'n'
```

- The positional and keyword arguments hold in case of classes as well.

```
class FirstClass:
        def __init__(self, m = 5, n = 4):
            self.x = m
            self.y = n
p = Firstlass()
print(p.x, p.y)
5 4
p1 = Firstlass(n=10)
print(p1.x, p1.y)
5 10
p2 = Firstlass(n=10,m = 50)
```

```
print(p2.x, p2.y)
```

```
50 10
```

Lesson-98

Exceptions

Exceptions

- Thus far when our Python scripts have crashed we've assumed there is something wrong with the script and tried to fix it.

```
dictionary = {}
data = open(filename, 'r')
for line in data:
    key, value = line.split()
    dictionary[key] = value
data.close()
```

- Sometimes external factors can cause our script to crash.

```
>>> line = "Too many values"
>>> key, value = line.split()
Traceback (most recent call last):
File "<stdin>", line 1, in <module>
ValueError: too many values to unpack (expected 2)
>>> line = "notenough!"
>>> key, value = line.split()
Traceback (most recent call last):
File "<stdin>", line 1, in <module>
ValueError: need more than 1 value to unpack
```

- The goal is to write scripts that can deal with failures caused by "unexpected" changes in external circumstances.

Error Handling

- When an error occurs in our script, Python stops it.
- So we need to take control from Python (when errors occurs).
- We need to define some remedial actions by ourselves. This is called error handling.

How does Python deal with errors?

- Errors in Python are called ***"exceptions"***.
- When something goes wrong, Python will ***"raise an exception"***, i.e. generate an error.
- The code that deals with such errors is called an ***"exception handler"***.
- If our script does not have an exception handler for the exception that occurred, then ***Python's default exception handler*** is used.

Python's default exception handler

- It halts our script and prints out some information about what has gone wrong.

wrong1.py

Traceback (most recent call last):

File "wrong1.py", line 1, in <module>

print(Hello) 

NameError: name 'Hello' is not defined

Types

- Python has a variety of different types of exception. These are called *built-in exceptions*.
- We define our own types of exceptions. These are called *user-defined exceptions*.

Exception handling: try…except

```
try:
        Python commands
except:
        Exception handler
try:
        Python commands
except ExceptionType:
        Exception handler
dictionary = {}
data = None
try:
    data = open('filename', 'r')
    for line in data:
```

```
                key, value = line.split()
                dictionary[key] = value
        data.close()
except IOError:
        print('Problem with file')
        print('Aborting')
        if type(data) != type(None):
                data.close()
```

Fix the Exception handler

```
dictionary = {"A":4,"C":5}
try:
    print (dictionary["5"])
except IOError:
    print('Problem with the dictionary')
```

Lesson-99

BioPython

BioPython

- The Biopython Project is an international association of developers of freely available Python (http://www.
- python.org) tools for computational molecular biology.
- The Biopython web site (http://www.biopython.org) provides an online resource for modules, scripts,
- and web links for developers of Python-based software for bioinformatics use and research.
- Python is easy to learn, has a very clear syntax and can easily be extended with modules written in C, C++ or FORTRAN

What can I find in the Biopython package

The ability to parse bioinformatics les into Python utilizable data structures, including support for

the following formats:

- Blast output - both from standalone and WWW Blast
- Clustalw
- FASTA

- GenBank
- PubMed and Medline
- ExPASy les, like Enzyme and Prosite
- SCOP, including `dom' and `lin' les
- UniGene
- SwissProt

- Files in the supported formats can be iterated over record by record or indexed and accessed via a Dictionary interface.
- Code to deal with popular on-line bioinformatics destinations such as:

  NCBI { Blast, Entrez and PubMed services

  ExPASy-Swiss-Prot and Prosite entries, as well as Prosite searches
- Interfaces to common bioinformatics programs such as:

  Standalone Blast from NCBI

  Clustalw alignment program

  EMBOSS command line tools
- A standard sequence class that deals with sequences, ids on sequences, and sequence features.
- Tools for performing common operations on sequences, such as translation, transcription and weight calculations.
- Code to perform classication of data using k Nearest Neighbors, Naive Bayes or Support Vector Machines.
- Code for dealing with alignments, including a standard way to create and deal with substitution matrices.
- Code making it easy to split up parallelizable tasks into separate processes.
- GUI-based programs to do basic sequence manipulations, translations, BLASTing, etc.
- Extensive documentation and help with using the modules, including this le, on-line wiki documentation,the web site, and the mailing list.
- Integration with BioSQL, a sequence database schema also supported by the BioPerl and BioJava projects.

Lesson-100

Installing Biopython

Installing Biopython

(http://biopython.org/wiki/Download)

This is usually as simple as the standard commands:

python setup.py build

python setup.py test

sudo python setup.py install

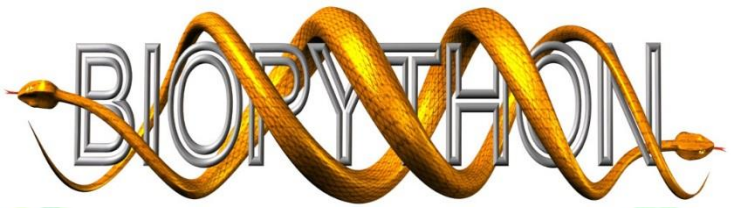
The longer version of our installation instructions covers installation of Python, Biopython dependencies and Biopython itself. It is available in

PDF http://biopython.org/DIST/docs/install/Installation.pdf

HTML formats http://biopython.org/DIST/docs/install/Installation.html

Lesson-101

File Parsing Biopython



Simple FASTA parsing example

If you open the lady slipper orchids FASTA le ls orchid.fasta in your favourite text editor, you'll see that the le starts like this:

```
>gi|2765658|emb|Z78533.1|CIZ78533 C.irapeanum 5.8S rRNA gene and ITS1 and ITS2 DNA
CGTAACAAGGTTTCCGTAGGTGAACCTGCGGAAGGATCATTGATGAGACCGTGGAATAAACGATCGAGTG
AATCCGGAGGACCGGTGTACTCAGCTCACCGGGGGCATTGCTCCCGTGGTGACCCTGATTTGTTGTTGGG
```

```
from Bio import SeqIO
for seq_record in SeqIO.parse("ls_orchid.fasta", "fasta"):
    print(seq_record.id)
    print(repr(seq_record.seq))
    print(len(seq_record))
```

You should get something like this on your screen:

```
gi|2765658|emb|Z78533.1|CIZ78533
Seq('CGTAACAAGGTTTCCGTAGGTGAACCTGCGGAAGGATCATTGATGAGACCGTGG...CGC', SingleLetterAlphabet())
740
...
gi|2765564|emb|Z78439.1|PBZ78439
Seq('CATTGTTGAGATCACATAATAATTGATCGAGTTAATCTGGAGGATCTGTTTACT...GCC', SingleLetterAlphabet())
592
```

GenBank file ls_orchid.gbk

```
from Bio import SeqIO
for seq_record in SeqIO.parse("ls_orchid.gbk", "genbank"):
    print(seq_record.id)
    print(repr(seq_record.seq))
    print(len(seq_record))
```

This should give:

```
Z78533.1
Seq('CGTAACAAGGTTTCCGTAGGTGAACCTGCGGAAGGATCATTGATGAGACCGTGG...CGC', IUPACAmbiguousDNA())
740
...
Z78439.1
Seq('CATTGTTGAGATCACATAATAATTGATCGAGTTAATCTGGAGGATCTGTTTACT...GCC', IUPACAmbiguousDNA())
592
```

Lesson-102

Quick Start Biopython

Working with sequences

>>> from Bio.Seq import Seq

>>> my_seq = Seq("AGTACACTGGT")

>>> my_seq

Seq('AGTACACTGGT')

>>> print(my_seq)

AGTACACTGGT

>>> my_seq.alphabet

Alphabet()

>>> my_seq

Seq('AGTACACTGGT')

```
>>> my_seq.complement()

Seq('TCATGTGACCA')

>>> my_seq.reverse_complement()

Seq('ACCAGTGTACT')
```

Lesson-103

Sequence and Alphabets Biopython

Simple FASTA parsing example

The alphabet object is perhaps the important thing that makes the Seq object more than just a string.

The currently available alphabets for Biopython are dened in the Bio.Alphabet module.

IUPAC alphabets (http://www.chem.qmw.ac.uk/iupac/)

Bio.Alphabet.IUPAC provides basic denitions for proteins, DNA and RNA, but additionally provides the ability to extend and customize the basic denitions

```
>>> from Bio.Seq import Seq
>>> my_seq = Seq("AGTACACTGGT")
>>> my_seq
Seq('AGTACACTGGT')
>>> my_seq.alphabet
Alphabet()
```

However, where possible you should specify the alphabet explicitly when creating your sequence objects, in this case an unambiguous DNA alphabet object:

```
>>> from Bio.Seq import Seq
>>> from Bio.Alphabet import IUPAC
>>> my_seq = Seq("AGTACACTGGT", IUPAC.unambiguous_dna)
>>> my_seq
Seq('AGTACACTGGT', IUPACUnambiguousDNA())
>>> my_seq.alphabet
IUPACUnambiguousDNA()
```

Unless of course, this really is an amino acid sequence:

```
>>> from Bio.Seq import Seq
>>> from Bio.Alphabet import IUPAC
>>> my_prot = Seq("AGTACACTGGT", IUPAC.protein)
>>> my_prot
Seq('AGTACACTGGT', IUPACProtein())
>>> my_prot.alphabet
IUPACProtein()
```

Lesson-104-110 Is not included

Lesson-111

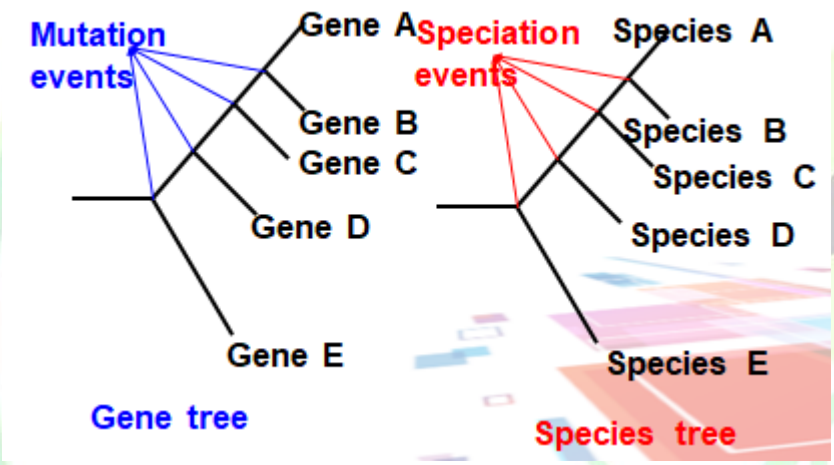
Phylogenetics

Outline

- Introduction
- Vocabulary
- Conclusion

Introduction

- It is the study of inferring evolutionary relationships
- Phylogenetic Trees



Vocabulary

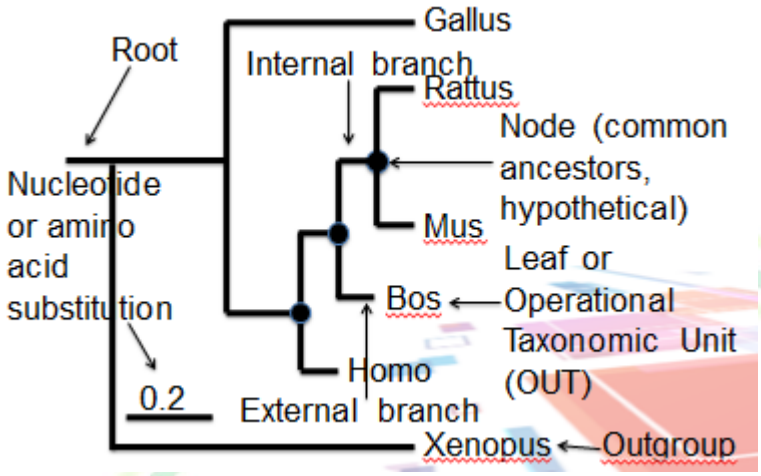
- **Nodes:** Represent taxonomic units

  Internal nodes correspond to ancestral species that are not part of the data
- **Internal branch:** Between 2 nodes

  Internal nodes are connected by internal brances
- **Leaf / Operational Taxonomic Units:**

  The terminal units appear at the tip of the branches (Species, genes or populations)
- External branch: Between a node and a leaf or OTU (Operational Taxonomic Units)

  Leaves are connected to the rest of the tree by the external branches emanating from an internal node



Conclusion

It is the study of evolutionary relationships

Lesson-112

Phylogenetic Analysis

- Introduction
- Choosing a study group
- Sampling a group of interest
- Sequence retrieval
- Data alignment and tree construction
- Conclusion

Introduction

The main aim of phylogeny reconstruction is to describe evolutionary relationships in tems of common ancestry

Choosing a study Group:

- A good first step is to survey the litaerature pertaining to the group of interest
- This will inform about the chocie of species and genes to be included in phylogeny reconstruction

Sampling the Group of Interest:

Four main sources of data for reconstructing the phylogeny of a gene family:

- Published sequences of characterized genes
- Gene databases such as NCBI
- EST project databases
- Unpublished data from colleagues

Sequence Retrieval:

When sequences are retrieved from BLAST searches they are allocated an e-score

- The closer the e-value is to 0, the higher the degree of similarity between two sequences

Sequence Alignment

Sequence alignment can be achieved automatically or manually

- There are many softwares for the data alignment and tree construction (e.g., ClustalW, MEGA etc.)

Conclusion

The main aim of phylogeny reconstruction is to describe evolutionary relationships in tems of common ancestry

Lesson-113

Phylogenetic Analysis and MSA

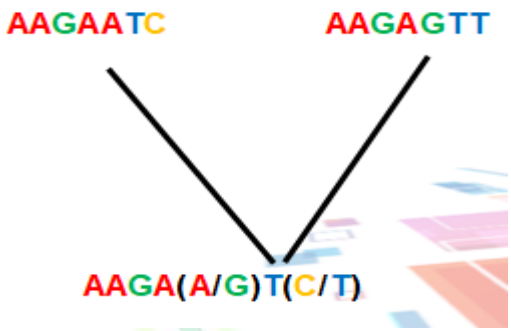
Introduction

Correct and accurate phylogenetic analysis depends upon quality of multiple sequence alignment

Significance

If two sequences in two organisms are very similar they are assumed to be derived from common ancestor

- The sequence alignments reveal which positions are conserved from the ancestor sequence



Progressive Approach

The progressive multiple alignment of a group of sequences first align ost similar pair

- Then it adds the more distant pairs
- The alignment is influenced by the “most similar“ pairs
- Does not always correclty represent the evolutionary history of the occured changes

Alternative Methods

- Most phylogenetic methods assume that each position in a sequence can change independently from the other positions

Gaps Resolution

- Gaps in alignements represent mutations in sequences
  - insertion, deletion, genetic rearrangements
- Gaps are treated in various ways by the phylogenetics methods.
- Most of them ignore gaps

Alternative to treat gaps is by **using sequence similarity scores** as the base for phylogenetic analysis

- instead of using the alignment itself, and trying to decide what happened at each position

The similarity scores based on scoring matrices (with gap scores) are used by the **Distance methods**

Conclusion

Correct and accurate phylogenetic analysis depends upon quality of multiple sequence alignment

Lesson-114

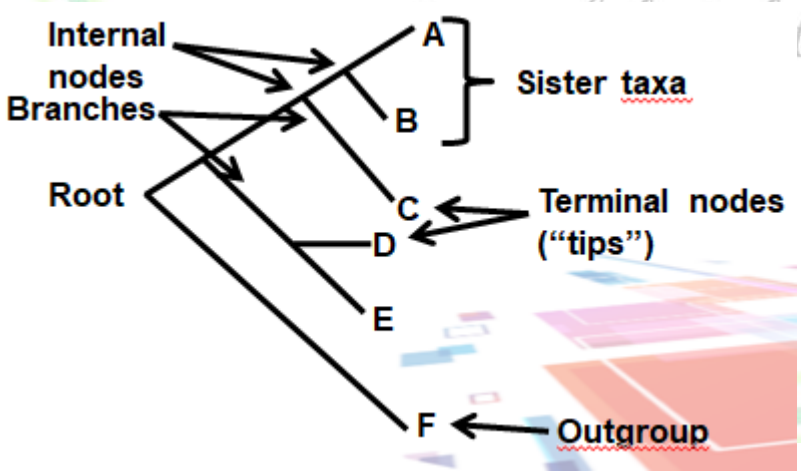
Evolutionary Tree

- Introduction
- Topologies of evolutionary tree
- Types of evolutionary tree
- Conclusion

Introduction

Also known as phylogenetic tree or dendograms

- It is a representation of evolutionary relationships between a set of organisms



Topologies of Evolutionary Tree

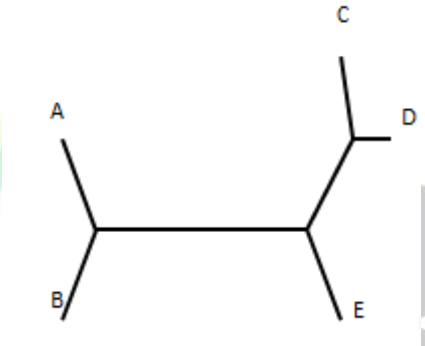
- Rooted tree
- Unrooted Tree

Rooted trees

- In a rooted tree, here exists a particular node called root, from which a unique path leads to any other node
- The direction of each path correspond to the evolutionary time
- The root is the common ancestor of all the OTU (Operational Taxonomic Units) under study

Unrooted trees

- A tree that only specifies the relationships among the OTUs but does not define the evolutionary path.
- Un-rooted trees do not make assumptions or require knowledge about common ancestors



Number of Trees

$N_R = (2n-3)!/2^{n-2}(n-2)!$

$N_U = (2n-5)!/2^{n-3}(n-3)!$

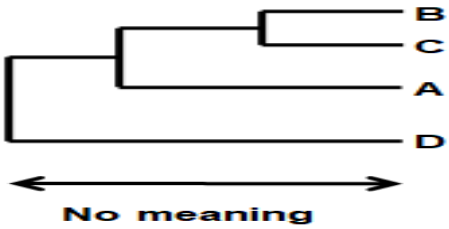
n	N_R	N_U
2	1	1
3	3	1
4	15	3
5	105	15
10	34459425	2027025

Types of Evolutionary Tree

- Cladogram
- Phylogram
- Ultrametric Tree

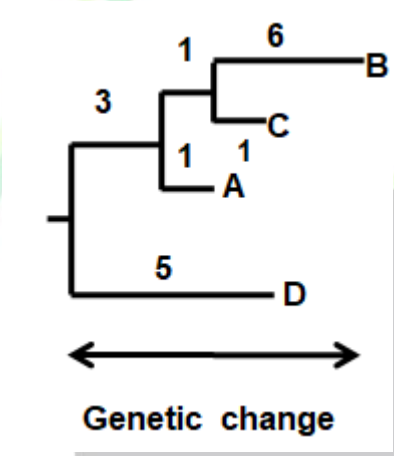
Cladogram

- They depict only the branching order which shows relationship among organisms
- The branches can be traced back to the point where they branch off representing a hypothetical ancestor



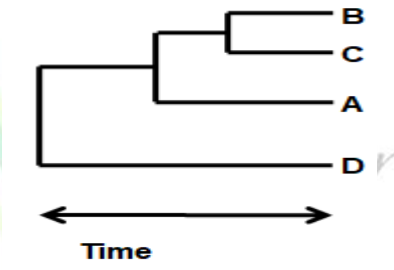
Phylogram

- These trees present branch lengths as proportional to some measure of divergence between species
- These trees typically include a scale bar to indicate the degree of divergence represented by a given length of branch



Ultrametric Tree

An ultrametric tree can be rooted so that all paths from the root to leaf have the same length



Conclusion

- Evolutionary tree is a representation of evolutionary relationships between a set of organisms

Lesson-115

Molecular Phylogenetics

Introduction

Study of evolutionary relationships between genes and species

- The actual pattern of evolutionary history is the phylogeny or evolutionary tree which we try to estimate

Evolutionary tree

- A tree is a mathematical structure which is used to model the actual evolutionary history of a group of sequences or organisms

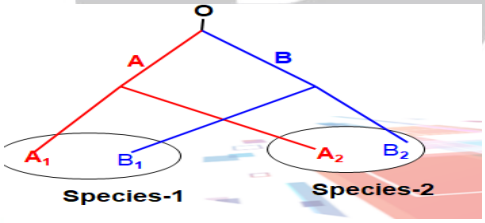
Evolutionary Process

Three major forces are at work in modifying the genetic information in any genome:

- Expansion (gene duplication)
- Deletion (gene loss)
- Exchange (HGT)

Phylogenetic Tree Reconstruction

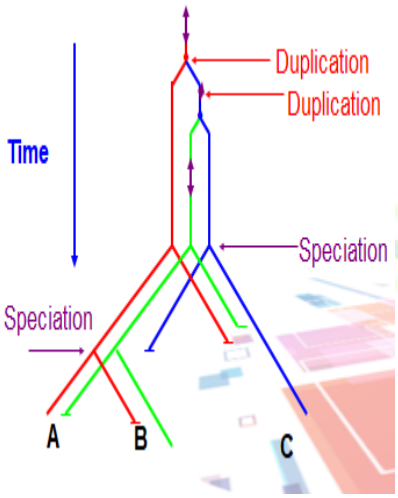
- Molecular phylogeny methods allow;
- Reconstruction of phylogenetic trees (inferred trees) from a given set of aligned sequences
- Aiming at reconstructing the history of successive divergence which took place during the evolution, between the considered sequences and their common ancestor
- These trees may not be the same as the true tree



Homologs: A_1, B_1, A_2, B_2

Paralogs: A_1 vs B_1 and A_2 vs B_2

Orthologs: A_1 vs A_2 and B_1 vs B_2



Evolutionary Process

Reconstruction of phylogenetic trees is a statistical problem

- reconstructed tree is an estimate of a true tree and the accuracy of this estimation should be statistically established

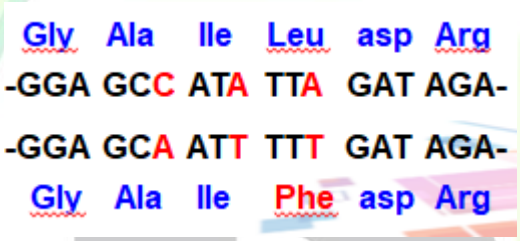
Nucleotides vs Amino Acids

- The nucleotide sequences of a pair of homologous genes have a **higher information content** than the amino acid sequences of the corresponding proteins
- synonymous changes alter the DNA sequence but do not affect the amino acid sequence

DNA yields more phylogenetic information than proteins

Synonymous vs non- Synonymous mutations

synonymous changes alter the DNA sequence but do not affect the amino acid sequence



Conclusions

Study of evolutionary relationships between genes and species

- The actual pattern of evolutionary history is the phylogeny or evolutionary tree which we try to estimate

Lesson-116[not available]

Tree Reconstruction Methods

Lesson-117

Maximum Parsimony Method

Maximum Parsimony Approach

Also known as minimum evolution method

- Evolution takes the shortest route
- To find a tree that explains the data with as few evolutionary changes as possible

Maximum Parsimony Fundamentals

1:- Mutations are exceedingly rare events

2:- A model that invokes more unlikely events (like mutations) is less likely correct

- The relationship that requires the fewest number of mutations to explain the data is more likely a correct relationship

Information content

- Sequence positions or columns in **MSA** (Multiple Seuence Alignment) may be informative or non-informative

Informative sites

- The sites or positions having **at least 2 different nucleotides**, and **each of these nucleotides has to be present at least twice** are informative

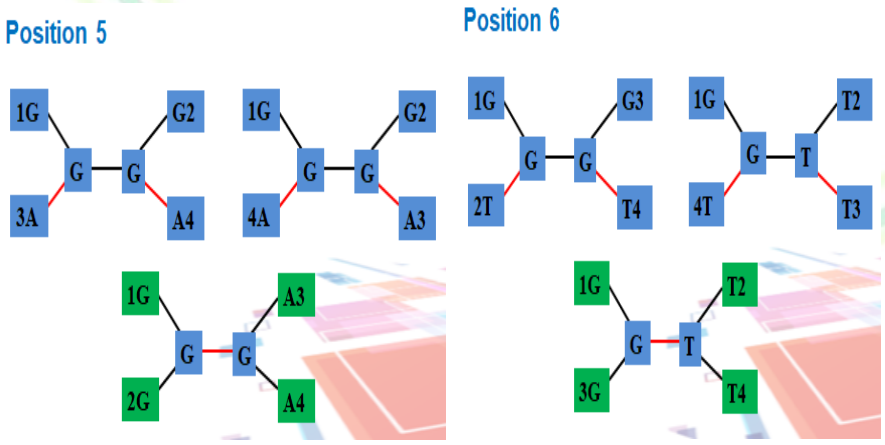
	1	2	3	4	5	6
1	G	G	G	G	G	G
2	G	G	G	A	G	T
3	G	G	A	T	A	G
4	G	A	T	C	A	T

Position 1 is said invariant and therefore uninformative, because all trees invoke the same number of mutations (0);

Position 2 is uninformative because 1 mutation occurs in all three possible trees;

Position 3 idem, because 2 mutations occur; Position 4 requires 3 mutations in all possible trees.

Positions 5 and 6 are informative, because one of the trees invokes only one mutation and the other 2 alternative trees both require 2 mutations.



Conclusions

- Also known as minimum evolution method
- To find a tree that explains the data with as few evolutionary changes as possible

Lesson-118

Maximum Likelihood Method

Introduction

Maximum Likelihood (ML) method finds the tree that maximizes the probability of genetic data given the tree

- Using a tree model for nucleotide substitutions, it will try to find the most likely tree out of all the trees given in the data set

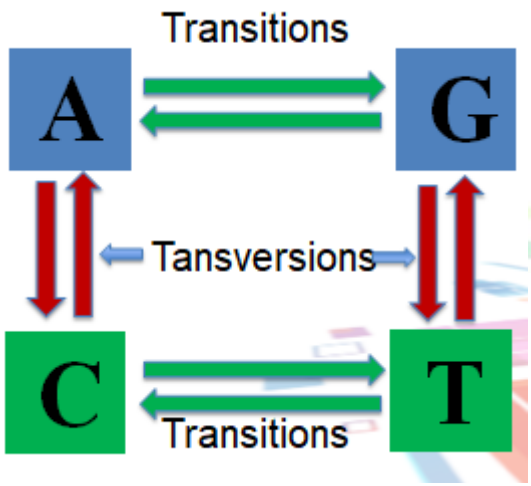
Approach

The bases are considered independent

- log-likelihood of having these bases are computed for a given topology by using a particular probability model
- This log-likelihood is added for all sites, and the sum of the log-likelihood is maximized to estimate the branch length of the tree
- This procedure is repeated for all possible topologies, and the topology that shows the highest likelihood is chosen as the final tree

Statistical Approach

- ML approach is purely a ststistical approach
- Probabilities are considered for each nucleotide substitution in a set of aligned sequences
- Thorne et al. (1991,1992) have introduced the method of sequence alignment based on a model (Bishop and Thompson 1986)
- Predicts the manner in which DNA sequences change during evolution



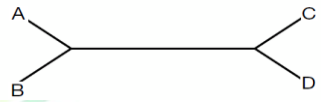
Statistical Approach

- Transitions are more common than tansversions
- The sequence with C and T are more closely related to each other than they are to the sequence with G

A. Sequences

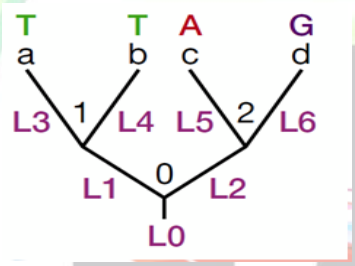
sequence a	A C G C G T T G G G
sequence b	A C G C G T T G G G
sequence c	A C G C A A T G A A
sequence d	A C A C A G G G A A

B. An unrooted phylogenetic tree for the sequences A-D

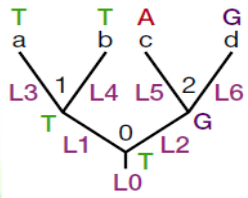


C: A rooted phylogenetic tree for one set of aligned sequences

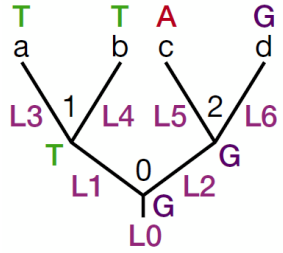
L1-L6 : Six Likelihoods



D: A rooted phylogenetic tree for one set of aligned sequences with one version of bases assigned to nodes 0,1, 2



E: A rooted phylogenetic tree for one set of aligned sequences with another version of bases assigned to nodes 0,1, 2



Likelihood of the tree

Likelihood L of the tree is

L(Tree) = L(Tree1)+L(Tree2)+---+L(Tree64)

Since there are three interior nodes (0,1,2) & there are 4X4X4=64 possible base asignments

Likelihood L of the tree1 is

$L(Tree1) = L_0 X L_1 X L_2 X L_3 X L_4 X L_6$

Likelihood of;

L_0 = starting with any nucleotide = ¼=o.25

Transitions = $2X10^{-6}$

Travrsions = 10^{-6}

Likelihood L of the tree1 is

= $0.25X2X10^{-6}X10^{-6}$

= $5X10^{-13}$

Conclusions

ML approach is a purely statistically based method. Probabilities are considered for every individual nucleotide substitution in a set of sequence alignment

Lesson-119

Distance Matrix

Distance Based Methods

- Feng and Doolittle (1996)
- Phylogenetic methods at first compute the pairwise distances from the sequences and then use these distances to obtain the trees
- Different phylogenetic programs have different distance calculation methods
  - Distance matrices are used as inputs to infer phylogenies using different Algorithms

Convert sequence data into a set of discrete pairwise distance values, arranged into a matrix

- Distance methods fit a tree to this matrix

$D_{i,j}$ = the distance between *i* and *j sequences*

$d_{i,j}$ = sum of branches on the tree path from i to j

- The phylogeny makes an estimation of the distance for each pair as the sum of branch lengths in the path from one sequence to another through the tree

Distance

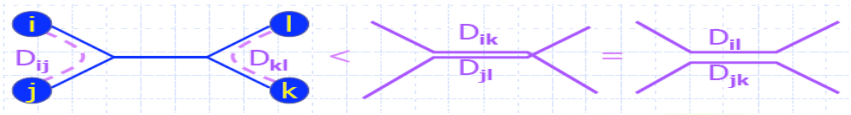
- A measure of how close is the tree to D is given by the least square criterion;
- $\sum(D_{i,j} - d_{i,j})^2 / D^2_{ij}$

Additivity

D is additive if and only if:

- For every four indices i,j,k,l, the maximum and median of the three pairwise sums are identical:

  $D_{ij}+D_{kl} < D_{ik}+D_{jl} = D_{il}+D_{jk}$



A. Sequences

sequence A	ACGCGTTGGGCGATGGCAAC
sequence B	ACGCGTTGGGCGACGGTAAT
sequence C	ACGCATTGAATGATGATAAT
sequence D	ACACATTGAGTGATAATAAT

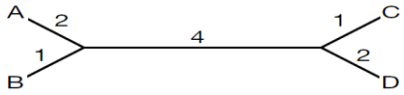
B. Distances between sequences, the number of steps required to change one sequence into the other.

n_{AB}	3
n_{AC}	7
n_{AD}	8
n_{BC}	6
n_{BD}	7
n_{CD}	3

C. Distance table

	A	B	C	D
A	–	3	7	8
B	–	–	6	7
C	–	–	–	3
D	–	–	–	–

D. The assumed phylogenetic tree for the sequences A-D showing branch lengths. The sum of the branch lengths between any two sequences on the trees has the same value as the distance between the sequences.



Conclusions

Distance based methods at first compute the pairwise distances from the sequences and then use these distances to obtain the trees

Lesson-120

UPGMA

UPGMA

- (Unweighted Pair – Group Method using arithmetic Averages)
- Assume constant mutation rate
- i.e Molecular clock hypothesis
- The two sequences with the shortest evolutionary distance between them are assumed to be the last two diverged and represented by the most racent internal node

Distance Matrix

	A	B	C	D	E
A		22	39	39	41
B			41	41	43
C				18	20
D					10
E					

Merge closely related pair

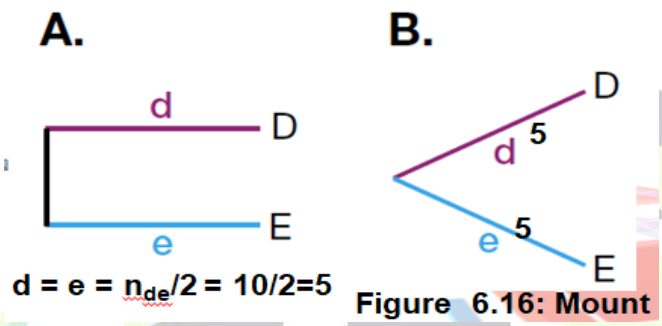
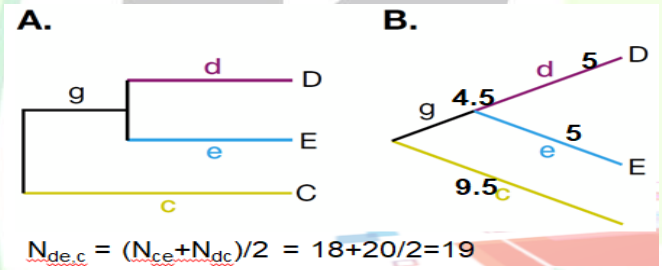


Figure 6.16: Mount



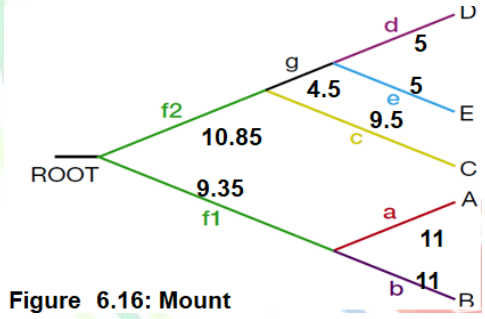


Figure 6.16: Mount

Conclusions

UPGMA assumes constant mutation rate i.e Molecular clock hypothesis

Lesson-121

Fitch-Margoliash Method

Introduction

Fitch-Margoliash (1987) method does not make the assumption of constant mutation rate

- Assume that the distances are additive

Methodology

- The sequences are combined in the groups of threes to define the branches of predicted tree and to calculate the branch lengths

Methodology

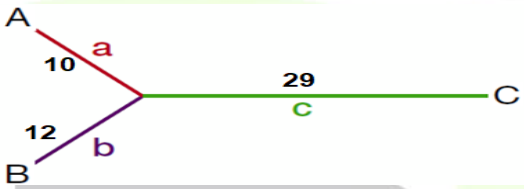
- Branch lengths a-c can be calculated using Algebra

	A	B	C
A		22	39
B			41
C			

distance from A to B $= a + b = 22$ (1)
distance from A to C $= a + c = 39$ (2)
distance from B to C $= b + c = 41$ (3)
subtract (3) from (2), $a - b = -2$ (4)
add (1) and (4), $2a = 20$, $a = 10$
from (1) and (2), $b = 12$, $c = 29$

Tree

- Showing relationship between a, b and c



Weaknesses

- This method of combining groups performs well for trees with short branches
- Presence of long branches decreases the reliability of results

Conclusions

- Assume that the **distances are additive**
- The sequences are combined in the groups of threes to define the branches of predicted tree and to calculate the branch lengths

Lesson-122

Microarray

Microarray

Tool for analyzing gene expression

- consists of a small membrane or glass slide containing samples of many genes arranged in a regular pattern
- have become incredibly popular since their inception in 1995 (Schena et al. (1995) *Science* **270**:467-70)

Gene Expression

Analyzing gene expression involves

- **Detection of mRNA** (transcriptome) present in a cell or tissue at a particular point in time
- mRNA isolated from a cell gives a snap-shot of the events occurring at that time

Gene Expression Studies

Analyzing gene expression leads to;

- Methods capable of giving a "snapshot" of RNA expression of all genes
- Can be used as diagnostic profile (e.g cancer diagnosis)
- Can show how RNA levels changes during development, exposure to stimulus, cell cycle, etc
- Provides large amounts of data
- Can help understand functions of whole systems

Principal

- The underlying principle of microarray technology is the **ability of DNA to bind to itself and to RNA**

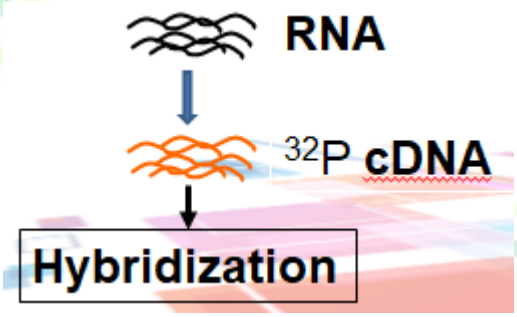
Evolution of microarray

One gene / one experiment

- Northern blots
- RNase assays
- RT-PCR

Macroarray

- Nylon Membranes
- ^{32}P labelled probes



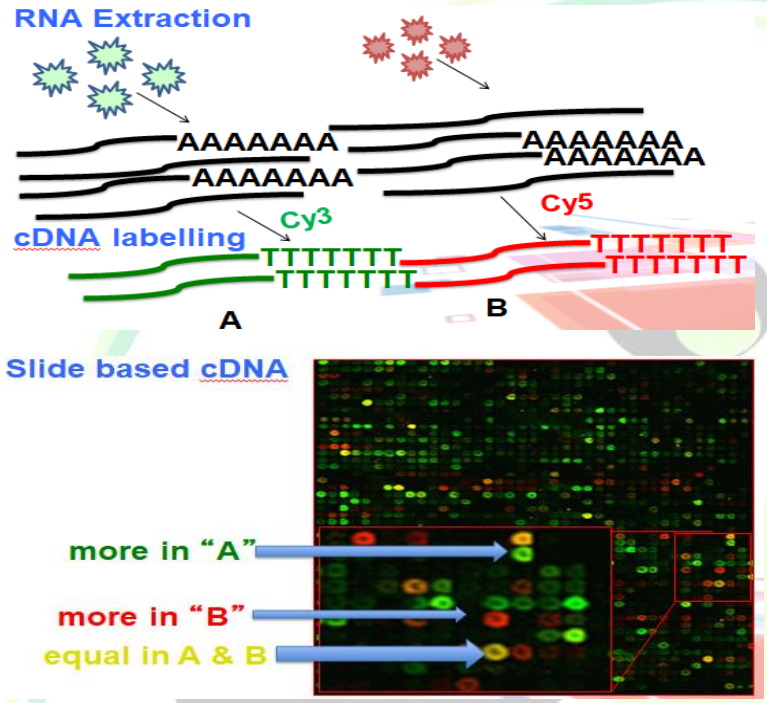
Slide based Arrays

The probe DNA is affixed directly to the surface of a glass microscope slide

- The probe DNA can range from a medium-length oligonucleotide (e.g., 60 nt) to an entire cDNA clone or larger

cDNA arrays

- >1000 nucleotides in length
- cDNA libraries



Uses of microarrays

- Gene discovery
- tissue profiles
- time course data
- altered genetic backgrounds
- Comparing tissues/genotypes

Conclusions

- Microarrays are used to investigate gene expression
- The underlying principle of microarray technology is the **ability of DNA to bind to itself and to RNA**

References

- Trevino, V., Falciani, F., and Barrera-Saldana, H.A. 2007. DNA microarrays: a powerful genomic tool for biomedical and clinical research. Mol Med 13: 527-541.
- Kato, H., Saito, K., and Kimura, T. 2005. A perspective on DNA microarray technology in food and nutritional science. Curr Opin Clin Nutr Metab Care 8: 516-522.

Lesson-123

Microarray Types

Introduction

Microarrays are used to investigate gene expression

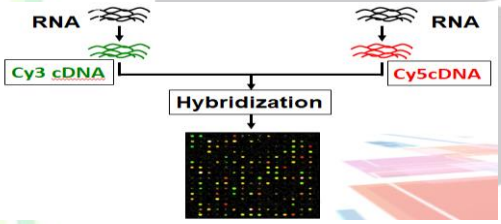
Main types of microarrays

- Slide based Arrays
  - cDNA arrays
  - oligonucleotide arrays
- Affymetrix

Slide based Arrays

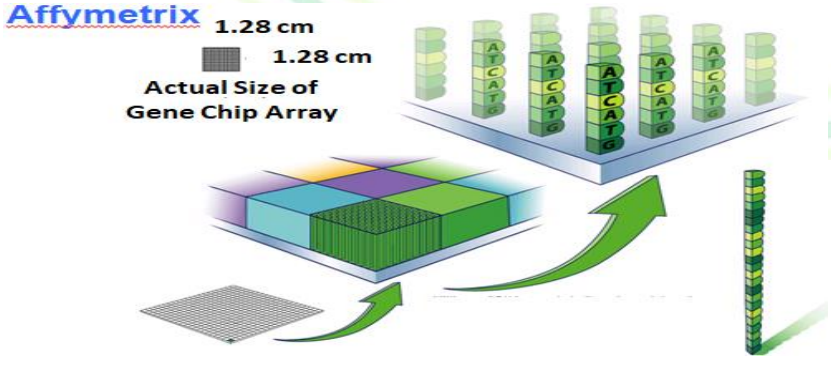
The probe DNA is affixed directly to the surface of a glass microscope slide

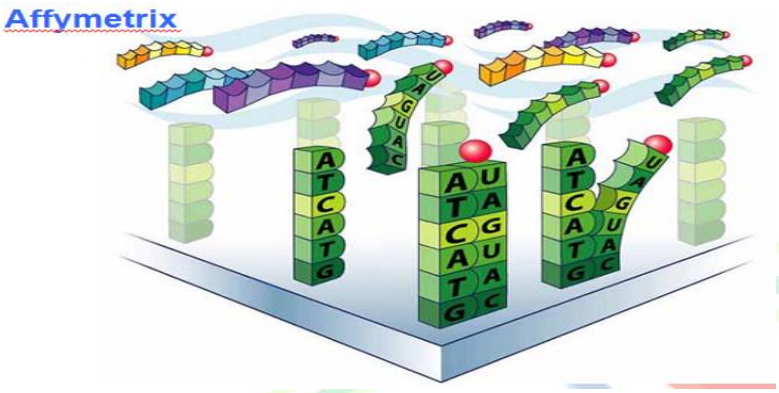
- The probe DNA can range from a medium-length oligonucleotide (e.g., 60 nt) to an entire cDNA clone or larger

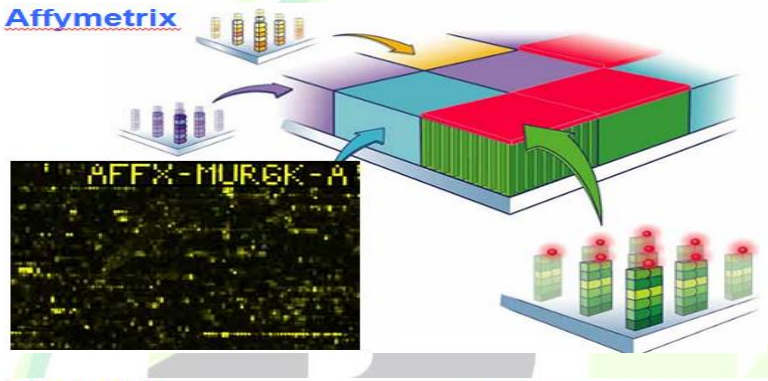


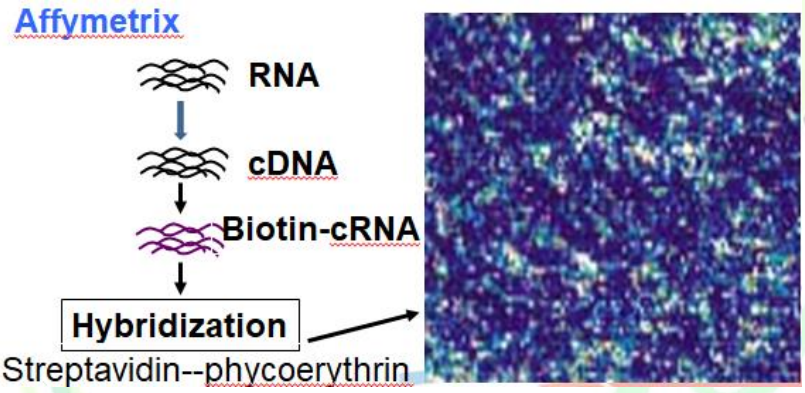
Affymetrix

- 25 nucleotides in length
- Mismatch pair
  - Single mismatch at centre of probe
  - Used to detect & eliminate false results
- 11 probe pairs per gene
- Photolithography









Slide vs Affymetrix

Affymetrix (high density oligonucleotide) and **glass slide** (cDNA, long oligo, etc) are conceptually similar, with differences in manufacture and details of design and analysis

Conclusions

Microarrays are used to investigate gene expression

Main types of microarrays

- Slide based Arrays
  - cDNA arrays
  - oligonucleotide arrays
- Affymetrix

References

- Trevino, V., Falciani, F., and Barrera-Saldana, H.A. 2007. DNA microarrays: a powerful genomic tool for biomedical and clinical research. Mol Med 13: 527-541.
- Kato, H., Saito, K., and Kimura, T. 2005. A perspective on DNA microarray technology in food and nutritional science. Curr Opin Clin Nutr Metab Care 8: 516-522.

Lesson-124

Array design

Introduction

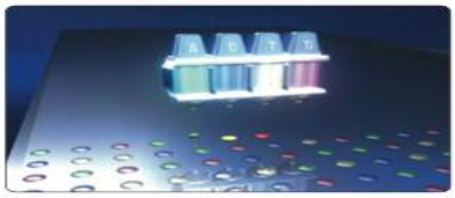
- Microarrays are based on the ability of complementary strands of DNA (or DNA and RNA) to hybridize to one another in solution with high specificity

Array Manufacturing

The DNA is deposited on the slide by any of a number of these methods;

- **"Printing"** with what is essentially an ink-jet printer
- Spotting by a robotically controlled set of fine-tipped metal pins
    - Spotted arrays

InkJet (HP/Canon) technology



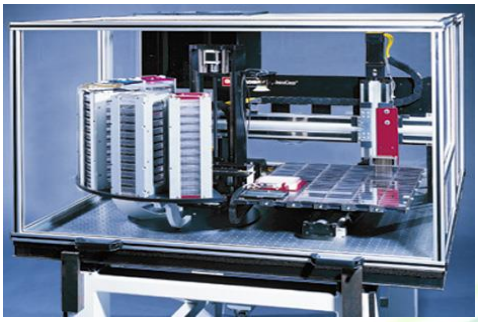


DNA spotting

- DNA attached to solid support
    - Glass, plastic, or nylon
- RNA is labeled
    - Usually indirectly
- Bound DNA is the probe
    - Labeled RNA is the "target"



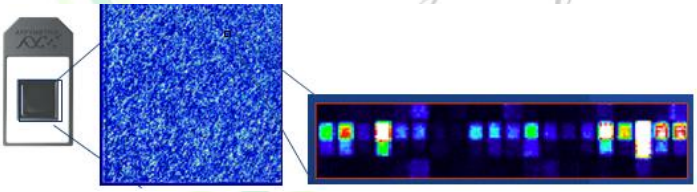
Commercial DNA Spotter

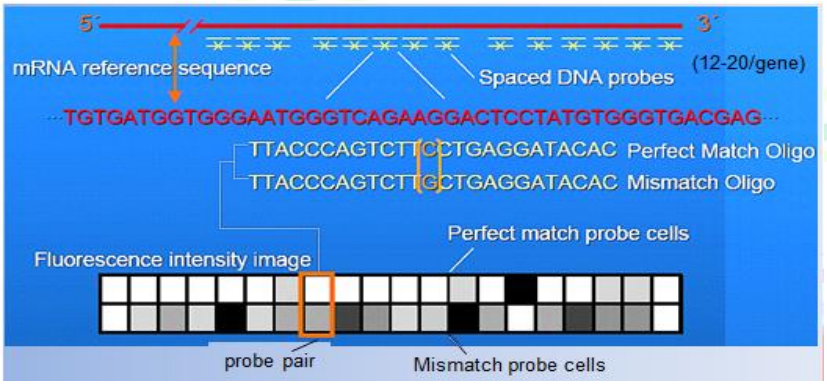


Affymetrix GeneChips

- Oligonucleotides synthesized on silicon chip
  - One base at a time
- Uses the process of photolithography
  - Developed for printing computer circuits
- Oligonucleotides for each gene selected by computer program to be the following:
  - Unique in genome
  - Nonoverlapping
  - Each gene is represented by a "probe set" consisting of 12-20 probes of 25 nt each
  - Each probe has a corresponding "mismatch" probe with a single base difference at the 13th nucleotide
  - Labeled RNA is hybridized to the array
  - Abundance is calculated using hybridization for the entire probe set, correcting the mismatch probes (indicates non-specific effects)

Benfey and Protopapas, "Genomics" © 2005 Prentice Hall Inc. / A Pearson Education Company / Upper Saddle River, New Jersey 07458





- Can hybridize only one sample/chip (i.e., no *direct* comparisons of 2 samples)
  - Standardized production tends to give good reproducibility
- Limited amount of probe sequence can be problematic
  - miss alternative splices, bias toward one end of transcript, (dependent on good genome annotation)

Attachment of sequences to slide

Covalent:

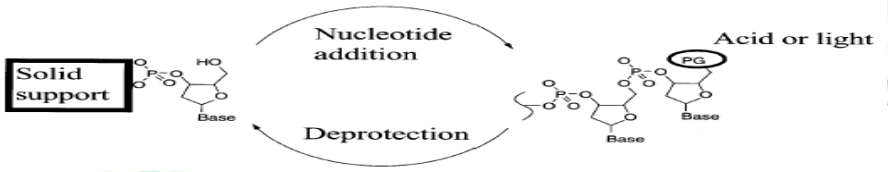
- Aliphatic amine group is added to 5' end of probe
- Probes are attached to glass from 5' end

Non-covalent:

- electrostatic attraction between amine and phosphate on probe backbone
- cDNA is attached to glass by the backbone
- only cDNA microarrays
- (oligonucleotides are too short for attraction)

In-Situ Synthesized Oligonucleotide Arrays

- built base-by-base on array surface
- Covalent reaction between the 5' hydroxyl and phosphate groups
- protective group on added 5' position to prevent multiple additions
- protective group is then converted to a hydroxyl group



Labelling

biotin: oligonucleotide Affymetrix

fluorescent dyes:

- **Cy3** (excited by laser at approximate wavelength 532 nm, fluoresces green)
- **Cy5** (excited by laser at approximate wavelength 635 nm, fluoresces red)

Conclusions

- In microarrays, complementary strands of DNA (or DNA and RNA) to hybridize to one another
- **Affymetrix** (high density oligonucleotide) and **glass slide** (cDNA, long oligo, etc) are two main methods

References

- Trevino, V., Falciani, F., and Barrera-Saldana, H.A. 2007. DNA microarrays: a powerful genomic tool for biomedical and clinical research. Mol Med 13: 527-541.
- Kato, H., Saito, K., and Kimura, T. 2005. A perspective on DNA microarray technology in food and nutritional science. Curr Opin Clin Nutr Metab Care 8: 516-522.

Lesson-125

Array Applications

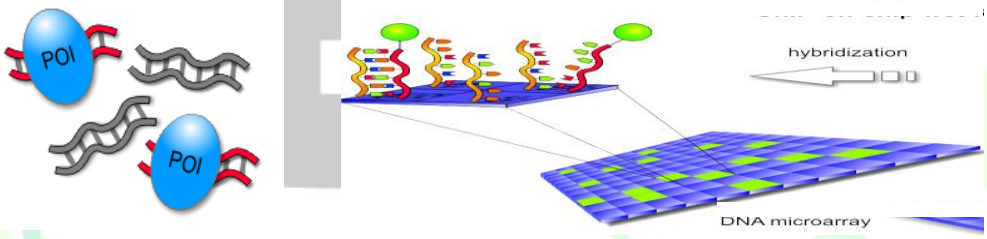
Introduction

Microarray can be combined with other methods in addition to looking at transcription ("transcriptional profiling")

- For instance, it can be used along with ChIP Chromatin Immuno-Precipitation4

This works best with whole genome tiling arrays to look at;

- Transcription factor binding
- Post Translation Modifications (PTMs) to histone proteins



RIP-chip

- Similar to ChIP-chip but for discovering RNA binding proteins rather than DNA binding proteins

aCGH

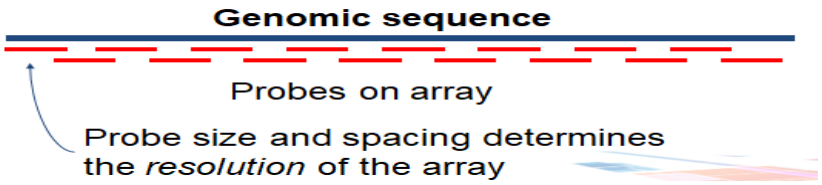
CGH (comparative genomic hybridization) looks at cytogenetic abnormalities

- Genomic DNA hybridized to array
- Often uses large clones (e.g., BACs) as array features

Tiled Arrays

- Cover a genomic region (or the whole genome!) at high coverage
- Probes are designed to cover whole sequence usually excluding only simple sequence repeats
- There is no bias toward known transcribed regions

sign of bright future



Protein binding microarrays (PBMs)

Protein-binding microarrays can be used to identify transcription factor binding sequences (motifs)

- double-stranded DNA probes used on array
- purified protein hybridized to array
- detected by antibody to protein
- can use real genomic sequence or carefully designed oligonucleotides
- possible to look at all possible 10-mer nucleotide sequences on a single array!

Conclusions

- In microarrays, complementary strands of DNA (or DNA and RNA) to hybridize to one another
- **Affymetrix** (high density oligonucleotide) and **glass slide** (cDNA, long oligo, etc) are two main methods

References

- Trevino, V., Falciani, F., and Barrera-Saldana, H.A. 2007. DNA microarrays: a powerful genomic tool for biomedical and clinical research. Mol Med 13: 527-541.
- Kato, H., Saito, K., and Kimura, T. 2005. A perspective on DNA microarray technology in food and nutritional science. Curr Opin Clin Nutr Metab Care 8: 516-522.

sign of bright future

Lesson-126
Experimental Design

Introduction

There are a number of important experimental design considerations for a microarray experiment:

- **Replicates**
- **amplification of RNA**
- **dye swaps**
- **reference samples**

Replicates

Replicates reduce variability in summary statistics and allow to use formal statistical methods

- At least three replicates per treatment are recommended

Lee et al. (1999)

Technical vs biological replicates

Technical replicates are repeat hybridizations using the same RNA isolate

Biological replicates use RNA isolated from separate experiments/experimental organisms

- Although technical replicates can be useful for reducing variation due to hybridization, imaging, etc., biological replicates are necessary for a properly controlled experiment

RNA Amplification

- Amplification methods can be used to increase the amount of RNA so that microarray experiments can be performed using very small numbers of cells

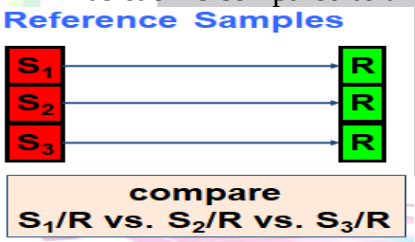
Dye Swaps

- When using 2-color arrays, it's important to hybridize replicates using a dye-swap strategy in which the colors (labels) are reversed between the two replicates.
- There can be biases in hybridization intensity due to **which dye is used**



Reference Samples

- One common strategy is to use a ***reference sample*** in one channel on each array.
- This is usually something that will hybridize to most of the features (e.g., a complex RNA mixture).
- Using a reference sample allows comparisons to be made between different experimental conditions, as each is compared to the common reference



Data Validation

It's strongly recommended that any key findings be verified by independent means

- Northern blots and quantitative RT-PCR are the typical ways of doing this; real-time, quantitative RT-PCR is generally the method of choice

MIAME (Minimal Information About a Microarray Experiment)

To publish a microarray experiment, it is expected to make available the following minimal information
This allows others to evaluate the data and compare it to other experimental results

EXPERIMENT DESIGN

- Type, factors, number of arrays, reference sample, qc, database accession (ArrayExpress, GEO)

Samples used, preparation and labeling
Hybridization procedures and parameters
Measurement data and specifications

- Quantitations, hardware & software used for scanning and analysis, raw measurements, data selection and transformation procedures, final expression data

Array design

- Platform type, features and locations, manufacturing protocols etc.

Conclusions

There are important experimental design considerations for a microarray experiment:

- **Replicates**
- **Amplification of RNA**
- **dye swaps**
- **reference samples**

It is expected to make available the minimal information about array

References

- Trevino, V., Falciani, F., and Barrera-Saldana, H.A. 2007. DNA microarrays: a powerful genomic tool for biomedical and clinical research. Mol Med 13: 527-541.
- Kato, H., Saito, K., and Kimura, T. 2005. A perspective on DNA microarray technology in food and nutritional science. Curr Opin Clin Nutr Metab Care 8: 516-522.

Lesson-127
Microarray Image processing

Introduction

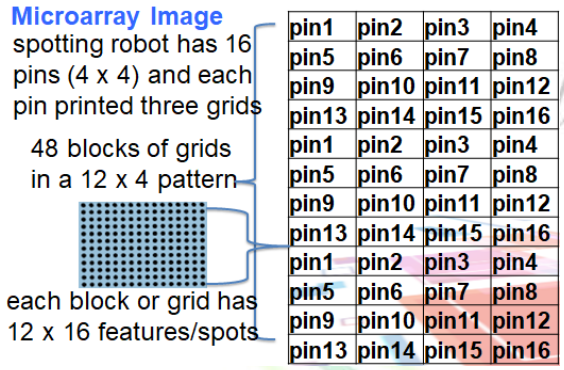
- Need to transform the fluorescence intensity image obtained after scanning into a measurement of gene expression for each spot
- Image processing for feature extraction affects the quality of the data
- In-situ synthesized oligonucleotide have integrated image-processing
- Pin-spotted arrays have multiple image processing approaches

Feature Extraction

Quantification of the digital TIFF image

Four steps:

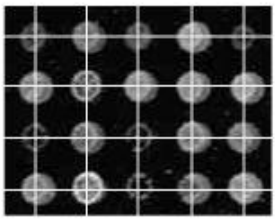
1. Identify the positions of the features on the microarray
2. For each feature, identify the pixels of the feature or foreground.
3. For each feature, identify background pixels
4. Calculate intensity of the feature, background and quality control



Feature Identification

Problems when identifying the position of the features:

- Shifted grid positions
- Curve within a grid
- Uneven feature spacing
- Uneven feature size



Software determines which pixels are in the area of a feature (segmentation):
• Fixed size circle (features may have different size)
• Variable circle (features may have irregular shape)
• Histogram (uses the frequency of intensities of the pixels)
• Adaptive shape (from center, extends to pixels with similar intensity)

Background Identification

Background adjustment accounts for:
- non-specific hybridization
- fluorescence from the glass

Image Quantification

Image-processing software provides:
• **Signal or foreground mean:**
mean of the pixels in the feature
• **Background mean:**
mean of the pixels in the background
• **Signal median:** median of pixels
(more robust to outliers than mean)
• **Background median:**
median of the pixels in the background

Image-processing software provides:
• **Signal standard deviation:**
standard deviation of pixels in the feature
• **Background standard deviation**
Median vs. standard deviation plot

File Format

The structure of a typical data file (for example .gpr file), after the images are scanned and the fluorescence intensities are translated into numbers, consists of:
- rows= microarray probe, and
- columns= information on the microarray probe (i.e. location, intensity value)

First lines describe the settings

ATF 1
15 9
..
DateTime=2006/01/05 00:32:05
Settings=
"Wavelengths=635 532"
"ImageFiles=C:\array1.tif 0 C:\array1.tif 1"
NormalizationMethod=None
..
Creator=GenePix Pro 5.0.1.24

Creator=GenePix Pro 5.0.1.24
Scanner=GenePix 4000B [50000]
Temperature=26.66
"ScanPower=100 100"

block	column	row	Flags	cy5_fore	cy5_back	cy3_fore	cy3_back
3	2	3	0	500	40	1000	50
3	2	3	0	1460	180	460	80
3	3	3	0	1030	55	1490	45
3	2	3	-100	25	20	30	21
.	.	.	.	.	.	.	.
32	24	20	0	20000	80	1000	60

Block	Column	Row	ID	F635 Median	B635 Median	F532 Median	B532 Median	Flags
1	1	1	EST1	65535	89	19969	59	0
1	2	1	EST2	45379	89	12517	59	0

Conclusions

Need to transform the fluorescence intensity image obtained after scanning into a measurement of gene expression for each spot

- Image processing for feature extraction affects the quality of the data

References

- Trevino, V., Falciani, F., and Barrera-Saldana, H.A. 2007. DNA microarrays: a powerful genomic tool for biomedical and clinical research. Mol Med 13: 527-541.
- Kato, H., Saito, K., and Kimura, T. 2005. A perspective on DNA microarray technology in food and nutritional science. Curr Opin Clin Nutr Metab Care 8: 516-522.

sign of bright future

Lesson-128
Microarray Normalization

Introduction

- Normalization comprises of a set of methods to remove systematic experimental errors and bias
- The normalization process includes:
  - **Data Cleaning and Transformation**
  - **Within-Array Normalization (for 2-dye data)**
  - **Between-Array Normalization (for multiple arrays)**

Data Cleaning and Transformation

The process involves:

- **Identification of flagged features**
- **Background subtraction**
- **Log transformations**

Identification of flagged features

Flag values represent quality of features like;

- Bad features (high coefficient of variation)
- Negative features (background > foreground)
- Dark features (weak foreground signal)

Flagged features

Good quality features have flag of 0

Flagged features may be dealt as;

- remove flagged features from the data set
- refer back to the original image
- adopt criterion

Background Subtraction

Background signal represents:

- non-specific hybridization to the glass
- natural fluorescence of the glass slide itself

We need to subtract background signal from the feature intensity

When background > feature intensity (foreground – background < 0):

- Remove these features from the analysis
- Use the lowest signal-intensity as background-subtracted intensity
- Estimate the true feature intensity

Log2 Transformation

Log2(Cy5/Cy3 raw intensities) = log2(Cy5)-log2(Cy3)

- Log ratio = 1 = 2-fold up-regulated gene in Cy5 sample
- Log ratio = -2 = 4-fold down-regulated in Cy5 sample
- Log ratio = 0 = not differentially expressed

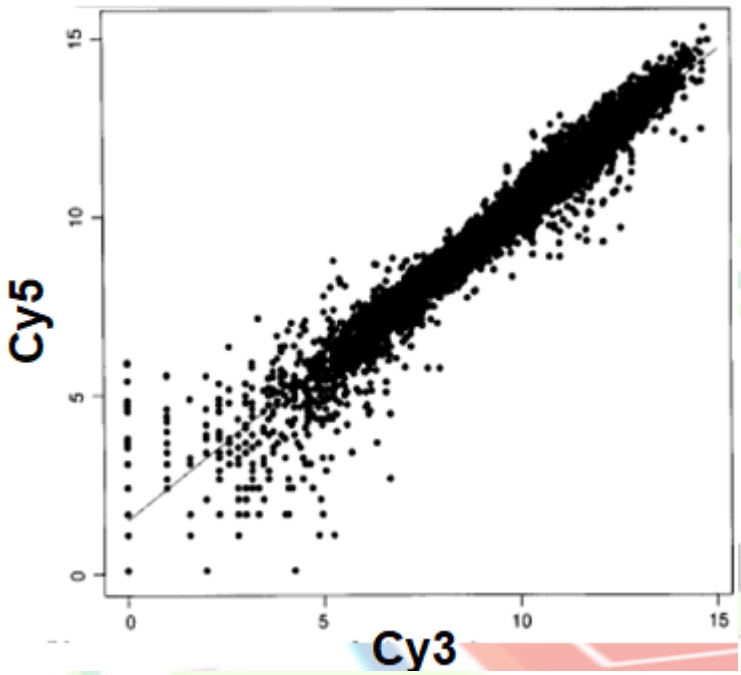
N/D = 1/16, 1/8, 1/4, 1/2, 0 1, 2, 4, 8,16

log2 = -4, -3, -2, -1, -INF, 0, 1, 2, 3, 4

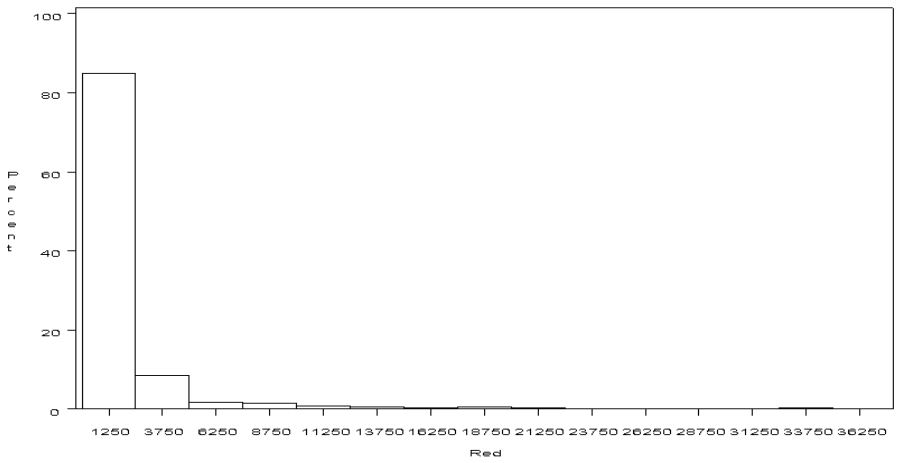
Transformation

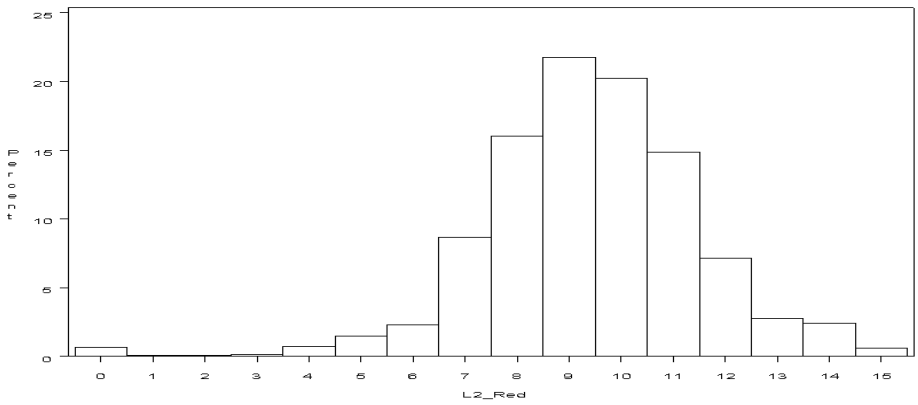
Plot of log2(Cy3) against log2 Cy5

sign of bright future



If intercept = 0 slope = 1the channels behave identically





Conclusions

Normalization removes experimental errors and bias, process includes:

- Data Cleaning and Transformation
- Within-Array Normalization
- Between-Array Normalization

References

- Trevino, V., Falciani, F., and Barrera-Saldana, H.A. 2007. DNA microarrays: a powerful genomic

tool for biomedical and clinical research. Mol Med 13: 527-541.
- Kato, H., Saito, K., and Kimura, T. 2005. A perspective on DNA microarray technology in food and nutritional science. Curr Opin Clin Nutr Metab Care 8: 516-522.

Lesson-129
Within-Array Normalization

Introduction

- Bias, experimental error must be removed prior to analysis of samples
- Most within-array normalizations are for spotted two-dye platforms
- Two-dye system measurements may introduce bias

Biases because of 2 Dye system

- Cy3 and Cy5 labels differentially incorporated (at different abundances)
- Cy3 and Cy5 dyes have different emission or response to the lasers (at different abundances)
- Cy3 and Cy5 emissions differentially measured at different intensities
- Cy3 and Cy5 intensities can vary between regions (array was not horizontal when scanned)

Genes selection

The set of genes to use for normalization must be identified:

a) All genes on the array. Assumes:

- small proportion of genes are expected to be differentially expressed
- there is a symmetry in the expression of the up/down-regulated genes
- remaining genes have constant expression

b) Constantly expressed or housekeeping genes

- expected to have a constant expression across a variety of conditions

Limitations:

- housekeeping genes tend to be highly expressed
- may not be representative of other genes of interest

c) Controls (spiked controls or titration series of control sequences)

i) Spiked controls

synthetic or foreign organism DNA sequences included in the two mRNA samples at equal amount

- expected to have equal red and green intensities

ii) Titration series approach:

- spots of the same sequence at different concentrations (intensities)
- red and green intensities are expected to be equal across concentrations

Methods to correct differential response of Cy3 and Cy5 channels

- All methods assume that most genes are not differentially expressed Otherwise use reference sample and control or spiked sequences

MA or RI plot

scatter plot of log ratio against average of log intensities of each feature

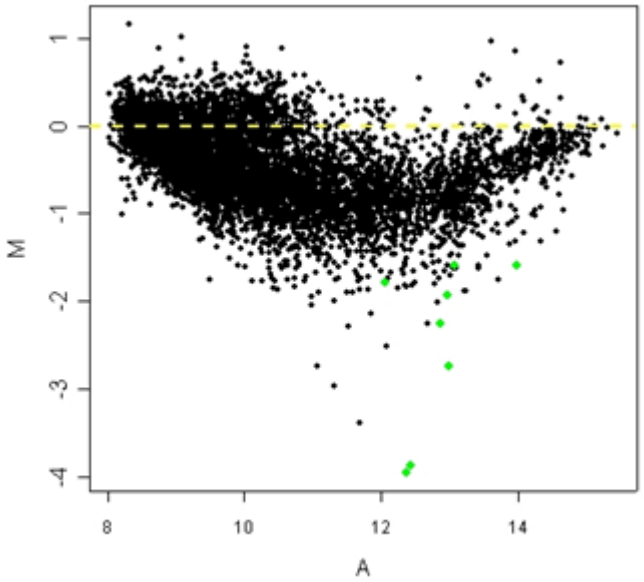
- Each point on the MA graph is a feature
- **x** coordinate: average intensity of the Cy3 and Cy5 channels
- **y** coordinate: log of the Cy5/Cy3 ratio
- Geometrically related to the log of Cy5 against the log of Cy3 scatter plots

MA or RI plot

$\mathbf{M=\log_2 R - \log_2 G}$

$\mathbf{A=(\log_2 R+\log_2 G)/2}$

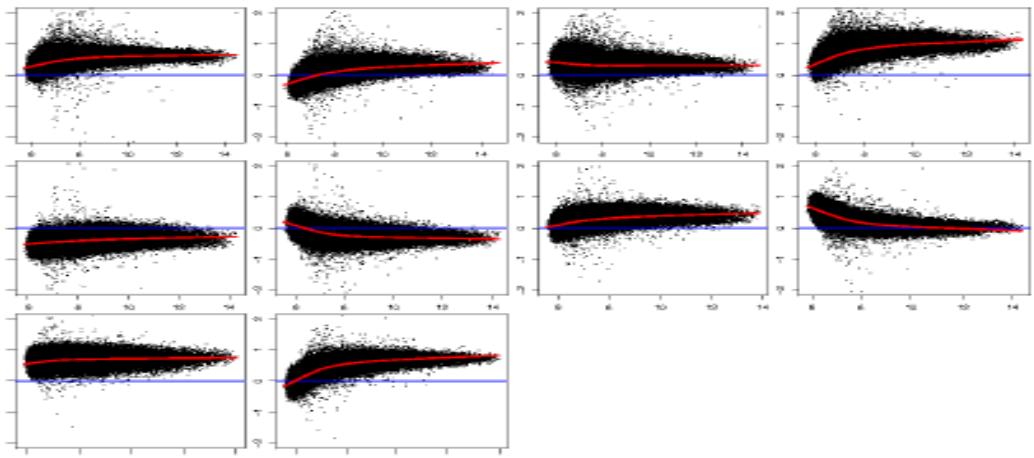
Values should scatter around 0



Linear normalization using MA plot:

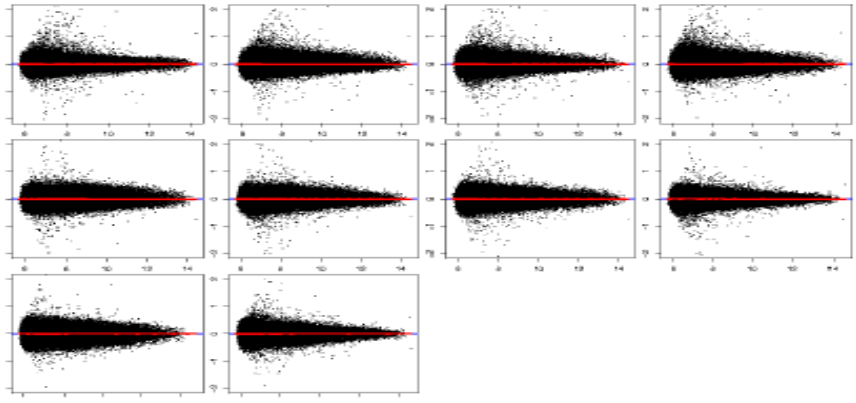
1. Perform a regression of the log ratio on the average log intensity **(MvsA)**
2. For each feature, the normalized log ratio is the observed M or log ratio value minus the predicted M value

Pairwise MA plot for 5 arrays



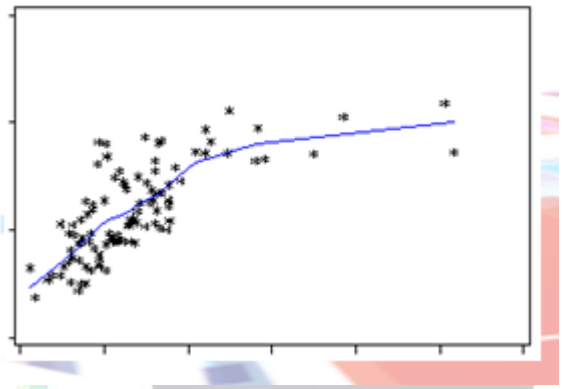
$M=\log_2 R - \log_2 G$
$A=(\log_2 R+\log_2 G)/2$

Pairwise MA plot after normalization



$M=\log_2 R - \log_2 G$
$A=(\log_2 R+\log_2 G)/2$

LOESS normalization
LOcally WEighted Scatterplot Smoothing
Fit a smooth curve
Use robust local linear fits
Effectively applies different scaling factors at different intensity levels
$Y = f(X)$
Transform X to $X' = f(X)$
Y and X' are comparable



Conclusions
Bias, experimental error must be removed prior to analysis of samples
Most within-array normalizations are for spotted two-dye platforms
Two-dye system measurements may introduce bias

References

- Trevino, V., Falciani, F., and Barrera-Saldana, H.A. 2007. DNA microarrays: a powerful genomic tool for biomedical and clinical research. Mol Med 13: 527-541.
- Kato, H., Saito, K., and Kimura, T. 2005. A perspective on DNA microarray technology in food and nutritional science. Curr Opin Clin Nutr Metab Care 8: 516-522.

sign of bright future

Lesson-130

Between-Array Normalization

Introduction

To compare samples in different arrays

- hybridization reactions may vary
- correct for the variability among arrays.
- variations in the distributions are due to experiment

Centring Normalization

The means of all the distributions are equal

Method: subtract the array mean log ratio or intensity from each feature

- All arrays will have a mean of zero after scale normalization.
- Alternative: median is more robust to outliers

Scaling normalization

The means and the standard deviations (SD) of all distributions are equal

- **Also known as standardization**
- subtract the mean of the array from the feature and divide by the standard deviation
- The mean of the measurements on each array will be zero
- The standard deviation will be 1 after scaling
- Alternative: median and median absolute deviation from median (MAD)
- Mean is preferred for computing distances and correlation coefficients

Quantile or distribution normalization

- The distributions of the data on each of the arrays are identical

1. Standardize the data.
2. For each array, order centered measurements from lowest to highest.
3. Compute a new distribution (use the average across all arrays for each ordered feature)
4. Replace each measurement with the corresponding new average

Original data

	A1	A2	A3
G1	1	6	7
G2	2	5	8
G3	3	4	9

Ranked data

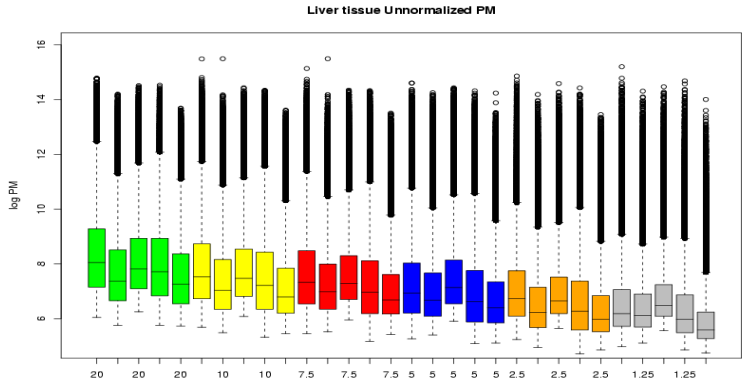
A1	A2	A3
1	4	7
2	5	8
3	6	9

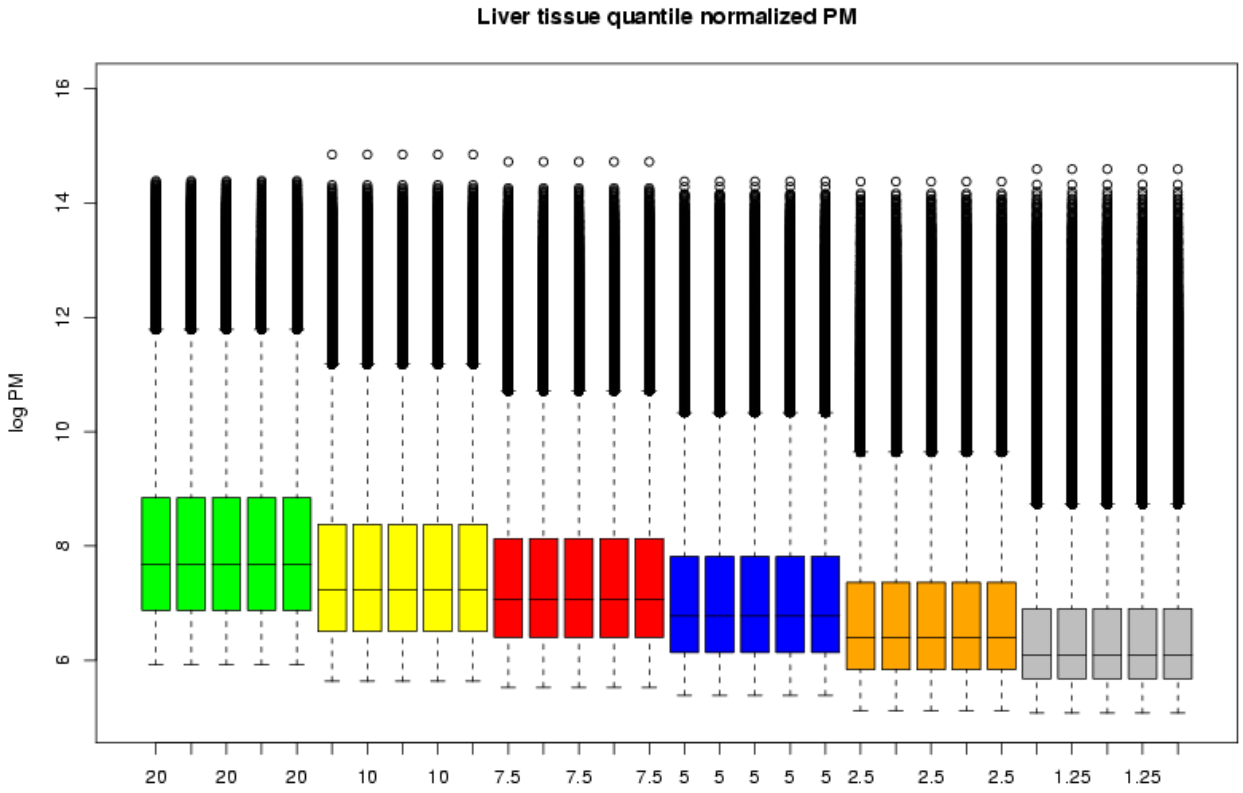
Average

4
5
6

Normalized values

	A1	A2	A3
G1	1	6	7
G2	2	5	8
G3	3	4	9





Conclusions

To compare samples in different arrays

- hybridization reactions may vary
- correct for the variability among arrays.
- variations in the distributions are due to experiment

References

- Trevino, V., Falciani, F., and Barrera-Saldana, H.A. 2007. DNA microarrays: a powerful genomic tool for biomedical and clinical research. Mol Med 13: 527-541.
- Kato, H., Saito, K., and Kimura, T. 2005. A perspective on DNA microarray technology in food and nutritional science. Curr Opin Clin Nutr Metab Care 8: 516-522.

Lesson-131

Data Analysis

Introduction

Microarray and RNASeq can measure the expression of thousands of genes simultaneously

- Vast amounts of data require computers

Types of analysis

Gene-by-gene

- Method: Statistical techniques

Categorizing groups of genes

- Clustering algorithms

Deducing patterns of gene regulation

- Under development

Defining Gene Expression

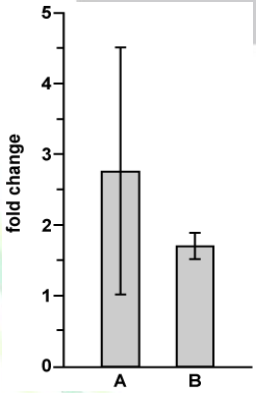
Ramsköld, *et al*. 2009, defines a threshold of

- 0.3 RPKM is optimal at different sequencing depth
- (for human and mouse data).

Differential Gene Expression

log2(A/B) = log2(average expression A) - log2 (average expression B)

- **2** $^{[\log2(A/B)]}$ = fold change in expression between conditions A and B
- Ignores variation of expression within and across conditions
- Does not discriminate absolute magnitude of expression e.g 10000/100 = 100/1



Benfey and Protopapas, "Genomics" © 2005 Prentice Hall Inc. / A Pearson Education Company / Upper Saddle River, New Jersey 07458

"A" has a fold change >2.5, but varies greatly between replicate experiments.

"B" has a fold change of only 1.75, but changes reliably each time the experiment is performed.

Hypothesis Testing

Null hypothesis: no differential expression.

Alternative hypothesis: differential expression

Hypothesis testing steps:

1) Compute test statistic of differential expression from experiment

2) Compare test statistic to distribution of test statistics

3) **Obtain P-value:** probability of a test statistic more extreme than that obtained in the experiment, by chance (if we run another experiment)

4) **Identify differentially expressed genes** as those with P-value < threshold

The smaller the P-value, the more statistically significant the differential expression of the gene and the less likely the result is to be false positive

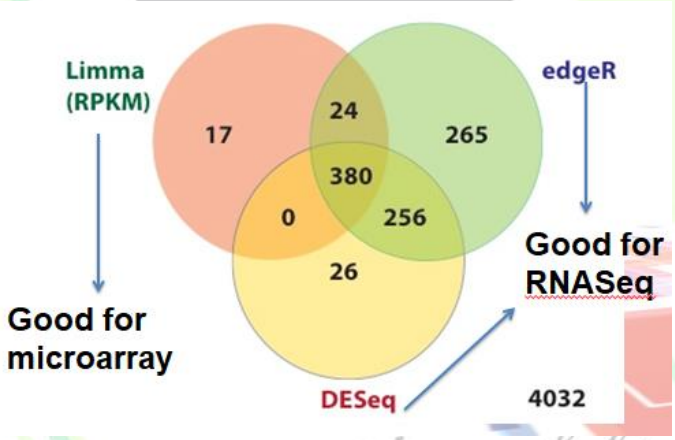
(that the observation have occurred by chance)

Bio-conductor packages:

Limma RPKM + t tests : linear models (Gordon Smyth, *2005*)

edgeR RPM + empirical Bayes estimation+exact tests : negative binomial distribution (Robinson and Oshlack, 2010)

DESeq RPM+ Estimate variance-mean dependence: negative binomial Anders and Huber, 2010)



Conclusion

Microarray and RNASeq can measure the expression of thousands of genes simultaneously

- Vast amounts of data require computers
- Bioconductor packages are efficiently used

References

- Trevino, V., Falciani, F., and Barrera-Saldana, H.A. 2007. DNA microarrays: a powerful genomic tool for biomedical and clinical research. Mol Med 13: 527-541.
- Kato, H., Saito, K., and Kimura, T. 2005. A perspective on DNA microarray technology in food and nutritional science. Curr Opin Clin Nutr Metab Care 8: 516-522.

Lesson-132

t test

Introduction

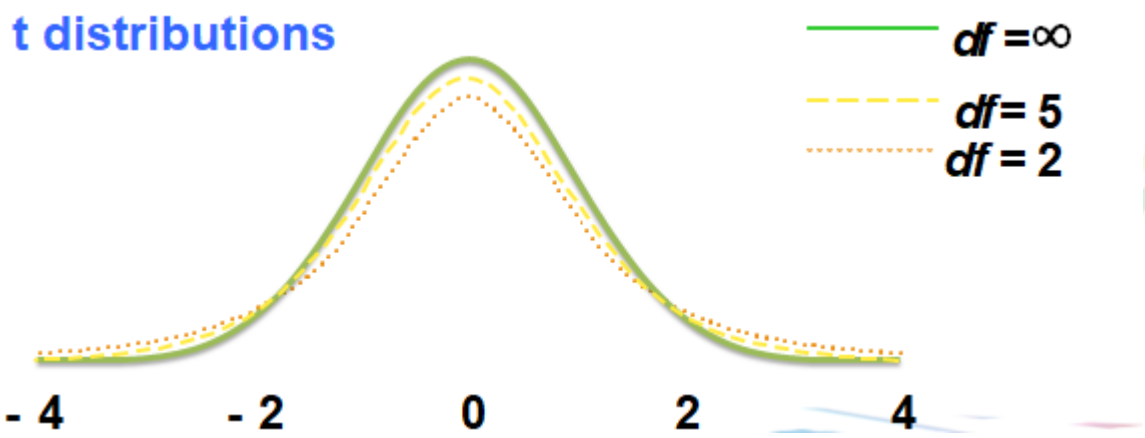
The Student's *t*-distribution is bell shaped and centered at 0 — like the Standard Normal distribution but more variable (larger spread)

- GOSSET, William Sealy 1876-1937



Assumptions

1. Population is normal although this assumption can be relaxed if sample size is "large"
2. Random sample was drawn from the population of interest

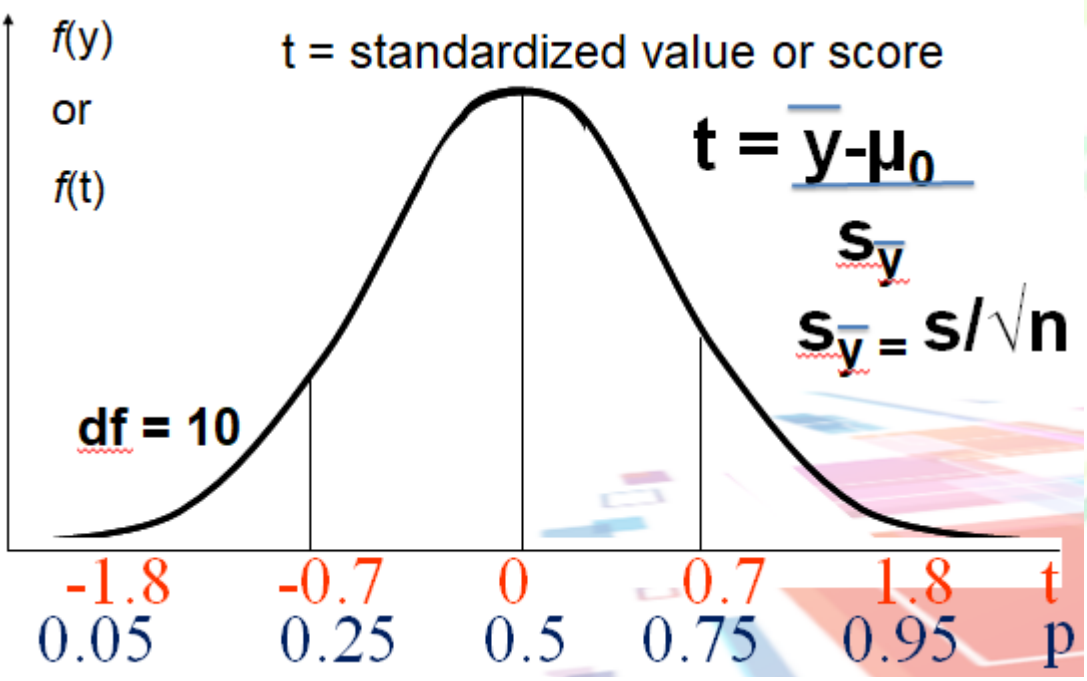


The t-distribution has one parameter that controls it's shape called the degrees of freedom.

As df increases, the t-distribution becomes more and more like the standard normal.

Degree of Freedom





(There are two red fours in a deck of 52, the 4 of hearts and the 4 of diamonds).

t or Z

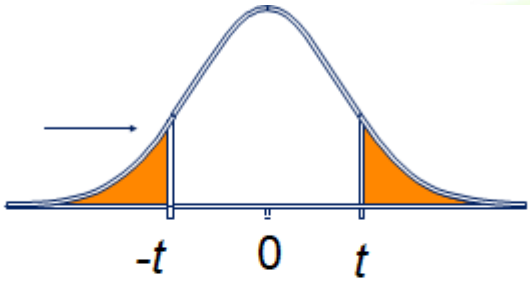
If n < 30 apply t else Z

t approaches z if $n \geq 30$

Form of Hypothesis

Ho: $\mu = \mu_o$

H_A: $\mu \neq \mu_o$ (two-tailed test)

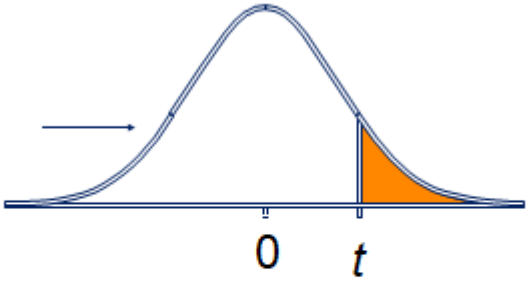


P-value = Shaded Area

t is either positive or negative

Ho: $\mu \leq \mu_o$

H_A: $\mu > \mu_o$ **(right-tailed test)**

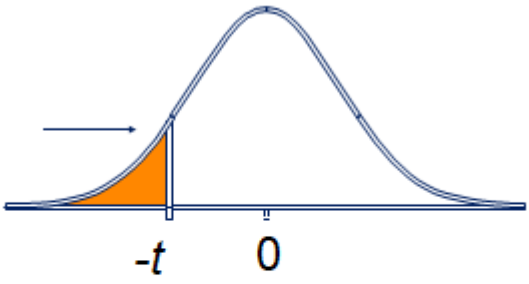


P-value = Shaded Area

t is positive

Ho: $\mu \geq \mu_o$

H_A: $\mu < \mu_o$ (left-tailed test)



P-value = Shaded Area

t is negative

One Sample t test

Testing against a standard or a claim

e.g Fuel efficiency of a car is less than companies claim

Gene expression is greater than 10 RPKM

References

- Trevino, V., Falciani, F., and Barrera-Saldana, H.A. 2007. DNA microarrays: a powerful genomic tool for biomedical and clinical research. Mol Med 13: 527-541.
- Kato, H., Saito, K., and Kimura, T. 2005. A perspective on DNA microarray technology in food and nutritional science. Curr Opin Clin Nutr Metab Care 8: 516-522.

Lesson-133

t Test Example

sign of bright future

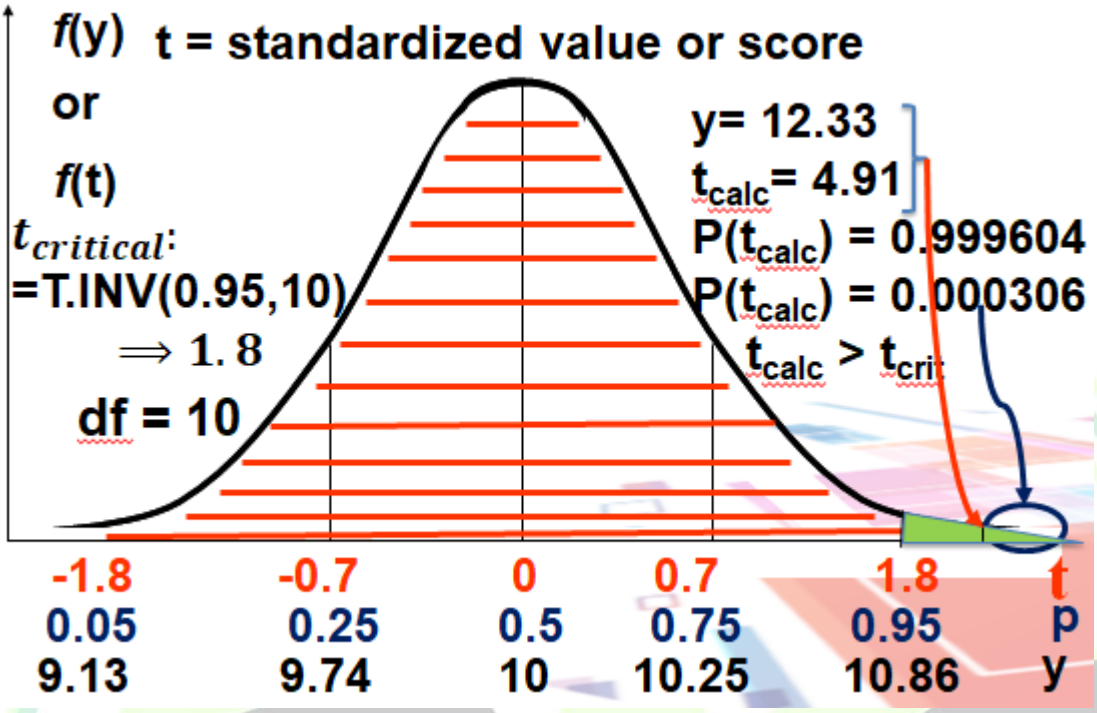
Example:

Expression of TP53 gene family is greater than 10 RPKM

We sample 11 genes belonging to TP53 gene family and found that the mean expression was 12.35 RPKM with a standard deviation of 1.5

n = 11, y = 12.35 s = 1.5

sy = s/√n = 1.5/ √10 = 0.475



(There are two red fours in a deck of 52, the 4 of hearts and the 4 of diamonds).

Binomial Tests

Since RNASeq data is a binomial data, t tests that assume normal distributions may not be appropriate for that

Negative binomial tests are used by Bioconductor packages like **edgeR** and **DESeq**

Conclusions

P < 0.05 (alpha)

0.000306 << 0.05

We conclude that the expression of TP53 gene family is greater than 10 RPKM

Probability of the expression > 10 is 0.0003 i.e 3/10000

References

- Trevino, V., Falciani, F., and Barrera-Saldana, H.A. 2007. DNA microarrays: a powerful genomic tool for biomedical and clinical research. Mol Med 13: 527-541.
- Kato, H., Saito, K., and Kimura, T. 2005. A perspective on DNA microarray technology in food and nutritional science. Curr Opin Clin Nutr Metab Care 8: 516-522.

Lesson-134
Two sample t Test

Introduction

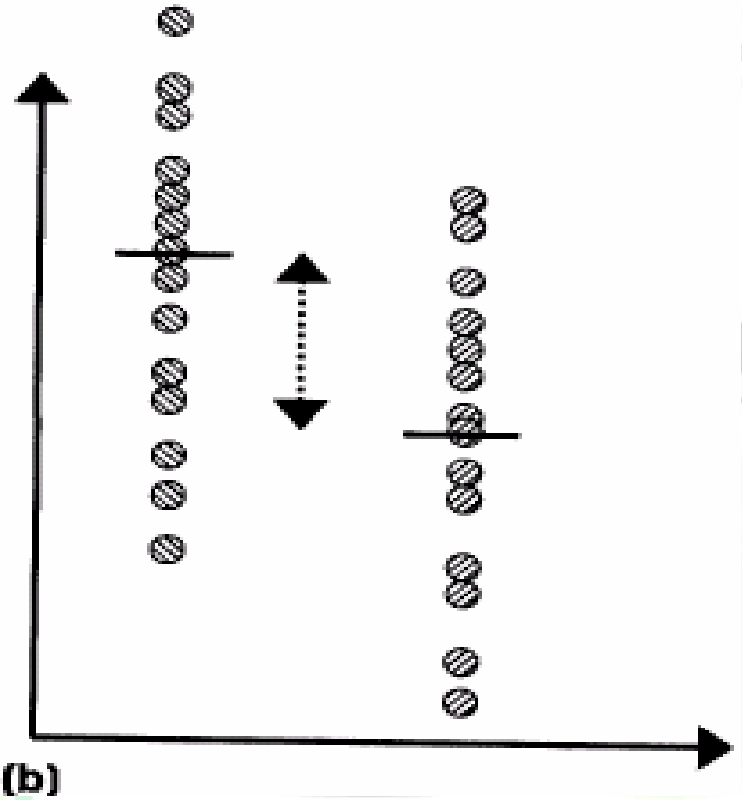
- Differential gene expression studies involve comparison of gene expression across two or more treatments

- Most models assume independence of observations i.e the value of one observation does not give information about other

Unpaired t-test

Gene expression from two groups of samples (one per condition)

- Test if the difference between the means of two conditions is zero



(b)

$y1$ = average of Trt1
$y2$ = average of Trt2
S_1 = Std dev of Trt1
S_2 = Std dev of Trt2

$$S_{\bar{Y}_B - \bar{Y}_A} = \sqrt{\frac{S_1^2}{n_1} + \frac{S_2^2}{n_2}}$$

$$t_{calculated}: \frac{(\bar{Y}_B - \bar{Y}_A) - \mu_0}{S_{\bar{Y}_B - \bar{Y}_A}}$$

Consider the expression of **TP53** gene in patients treated with $ZnCl_2$ vs normal persons as control

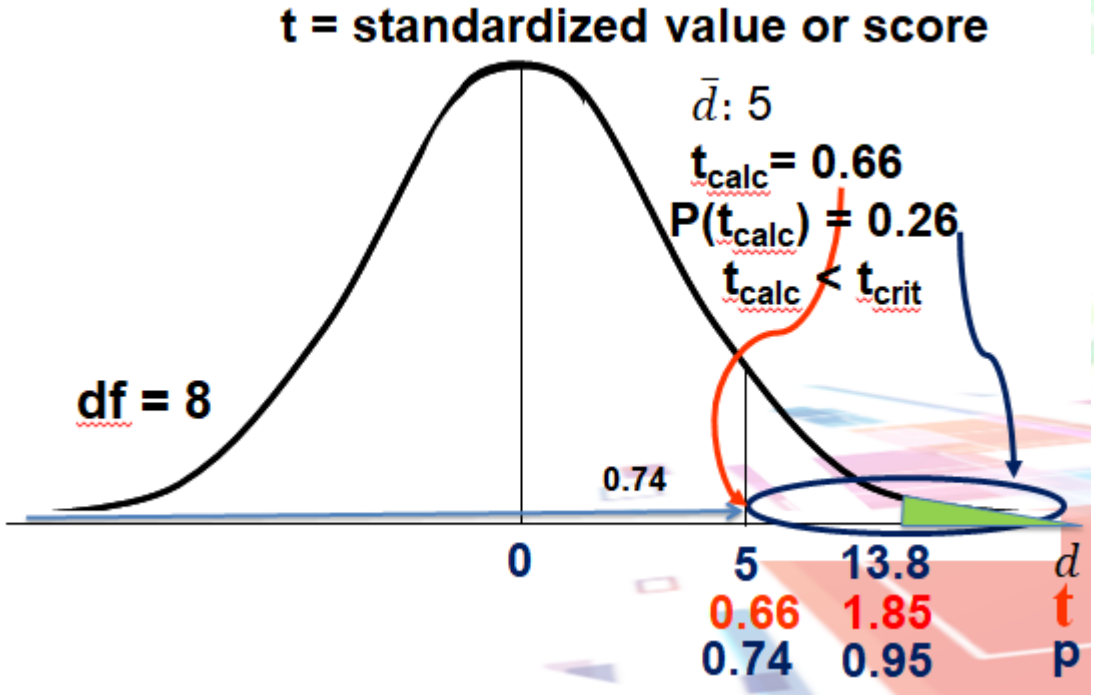
Control	Treatment
12	15
61	64
44	45
10	13
47	49

Notice that we have two **independent** or **uncorrelated** samples in this question
So we have to run a t-test for two **independent** samples

$\boldsymbol{H_0}$: $\boldsymbol{\mu_d \leq 0}$
$\boldsymbol{H_A}$: $\boldsymbol{\mu_d > 0}$
$\bar{Y}_B - \bar{Y}_A$: $46 - 41 = 5$

$S_{\bar{Y}_B - \bar{Y}_A} = \sqrt{\frac{143.2}{6} + \frac{194.8}{6}} = 7.5$
$\boldsymbol{t_{calculated}}$: $\frac{5-0}{7.5} = 0.66$

Null Distribution
If H_0 is true, $t_{calculated}$ follows t distribution with $(n_A - 1) + (n_B - 1)$
$= 5 - 1 + 5 - 1 = 8$
degrees of freedom



(There are two red fours in a deck of 52, the 4 of hearts and the 4 of diamonds).

Conclusions

Differential gene expression studies involve comparison of gene expression across two or more treatments
Most models assume independence of observations

References

- Trevino, V., Falciani, F., and Barrera-Saldana, H.A. 2007. DNA microarrays: a powerful genomic tool for biomedical and clinical research. Mol Med 13: 527-541.
- Kato, H., Saito, K., and Kimura, T. 2005. A perspective on DNA microarray technology in food and nutritional science. Curr Opin Clin Nutr Metab Care 8: 516-522.

Lesson-135

Paired t Test

Introduction

Most models assume independence of observations i.e the value of one observation does not give information about other

Some times observations are dependent

Dependencies

Dependencies may arise:

- within array duplicate spots located next to each other
- across arrays same sample used in multiple slides

Dependencies may arise:

- across samples

samples from same patient or subject

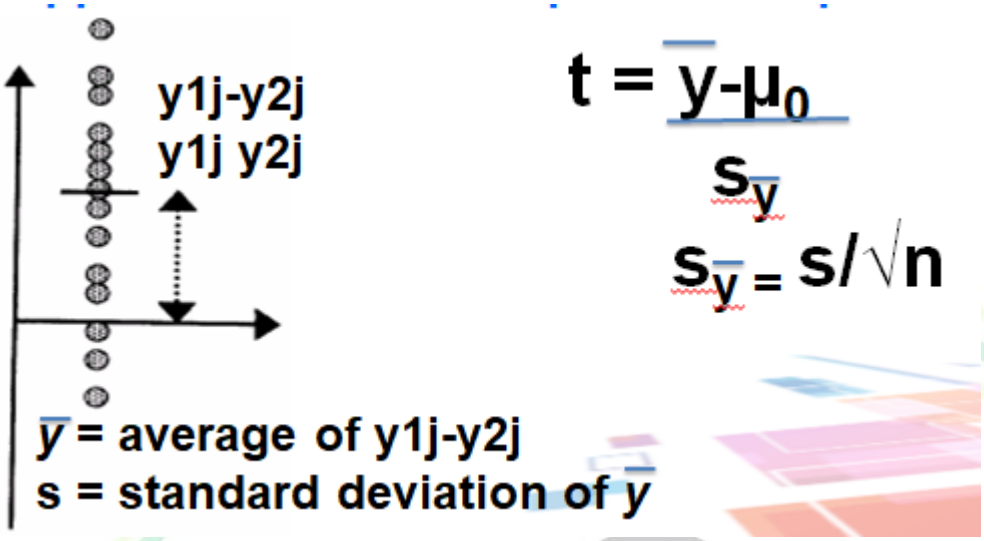
Approaches to resolve potential dependencies:

1) Use models that account for correlations

2) Combine dependent observations into a single observation

i) Average observations from replicated spots within microarray

ii) Pair data: when different conditions are measures on the same subject otherwise, unpaired data analysis



Paired t-test

- Data points are somehow correlated
- We have to pair up the data points and calculate their differences
- Test our hypothesis on the distribution of these differences
- Consider the expression of **TP53** gene in patients before and after a treatment with $ZnCl_2$

Patient	Cont.	Trt.
1	12	15
2	61	64
3	44	45
4	10	13
5	47	49

Test whether the drug is effected or not?

TP53 encodes a tumor suppressing protein **P53**

We want to see a higher expression of TP53 to control tumor

correlated samples

we have to run a paired t-test. Compute the difference in expression for each pair of genes

d = Treatment - Control

Patient	Cont.	Trt.	d
1	12	15	3
2	61	64	3
3	44	45	1
4	10	13	3
5	47	49	2

we will only use d in our statistical test

One sample T-test

H_0: $\mu_d \leq 0$

H_A: $\mu_d > 0$

$\bar{d}$: 2.4 (this is the average of all d values and our sample mean for d)

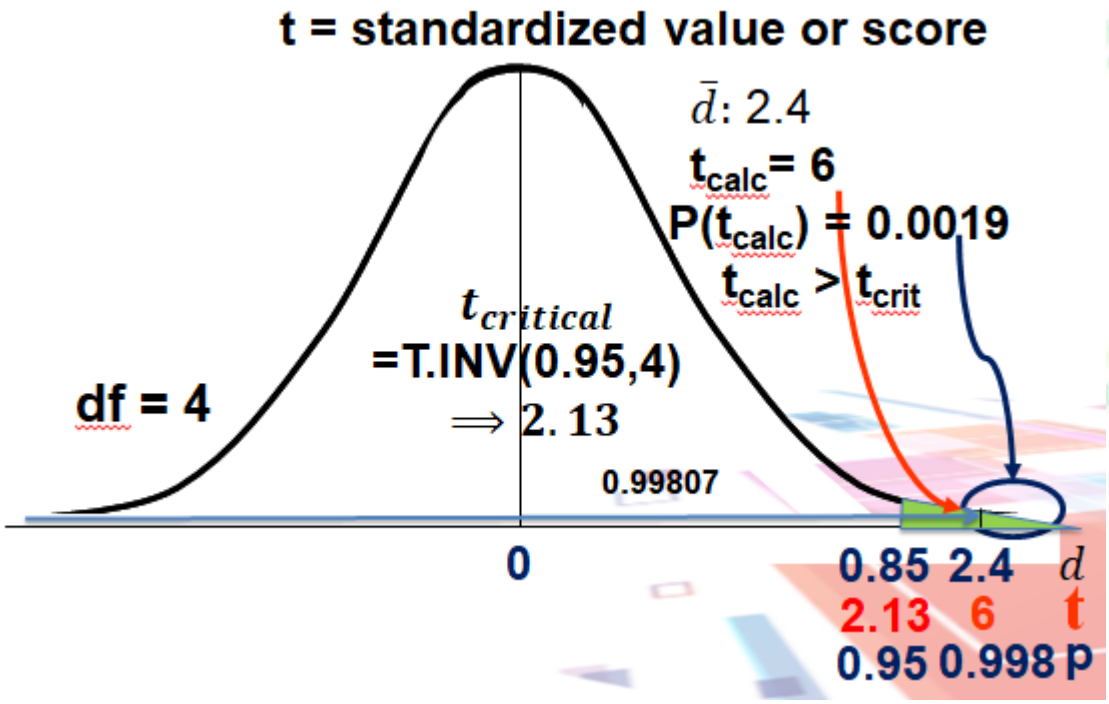
S_d: 0.89 (this is the sample standard deviation of the d values)

$S_{\bar{d}} = \frac{S_d}{\sqrt{n}} = \frac{0.89}{\sqrt{5}} = 0.40$

$t_{calculated}$: $\frac{\bar{d}-\mu_d}{S_{\bar{d}}} = \frac{2.4-0}{0.40} = \mathbf{6.0}$

Null Distribution:

If H_0 is true, $t_{calculated}$ follows t distribution with $(n-1) = 5-1 = 4$ degrees of freedom (because we have n=5 d values)



Potential disadvantages of combining data:

- Loose information, does not account for original variability
- Assumes all combinations are equally effective

References

- Trevino, V., Falciani, F., and Barrera-Saldana, H.A. 2007. DNA microarrays: a powerful genomic tool for biomedical and clinical research. Mol Med 13: 527-541.
- Kato, H., Saito, K., and Kimura, T. 2005. A perspective on DNA microarray technology in food and nutritional science. Curr Opin Clin Nutr Metab Care 8: 516-522.

Lesson-136

Multiplicity Issue

P values

Using statistical tests is usually a better way of determining which genes are truly changed in an experiment

The p-value tells you how much confidence to place in your result

Sensitivity

How many of the truly changed genes you detect without sacrificing specificity (i.e., incorrectly thinking that a gene's expression is altered)

Sensitivity vs Specificity

Type I error (α, "p-value") false positives

Type II error (β)

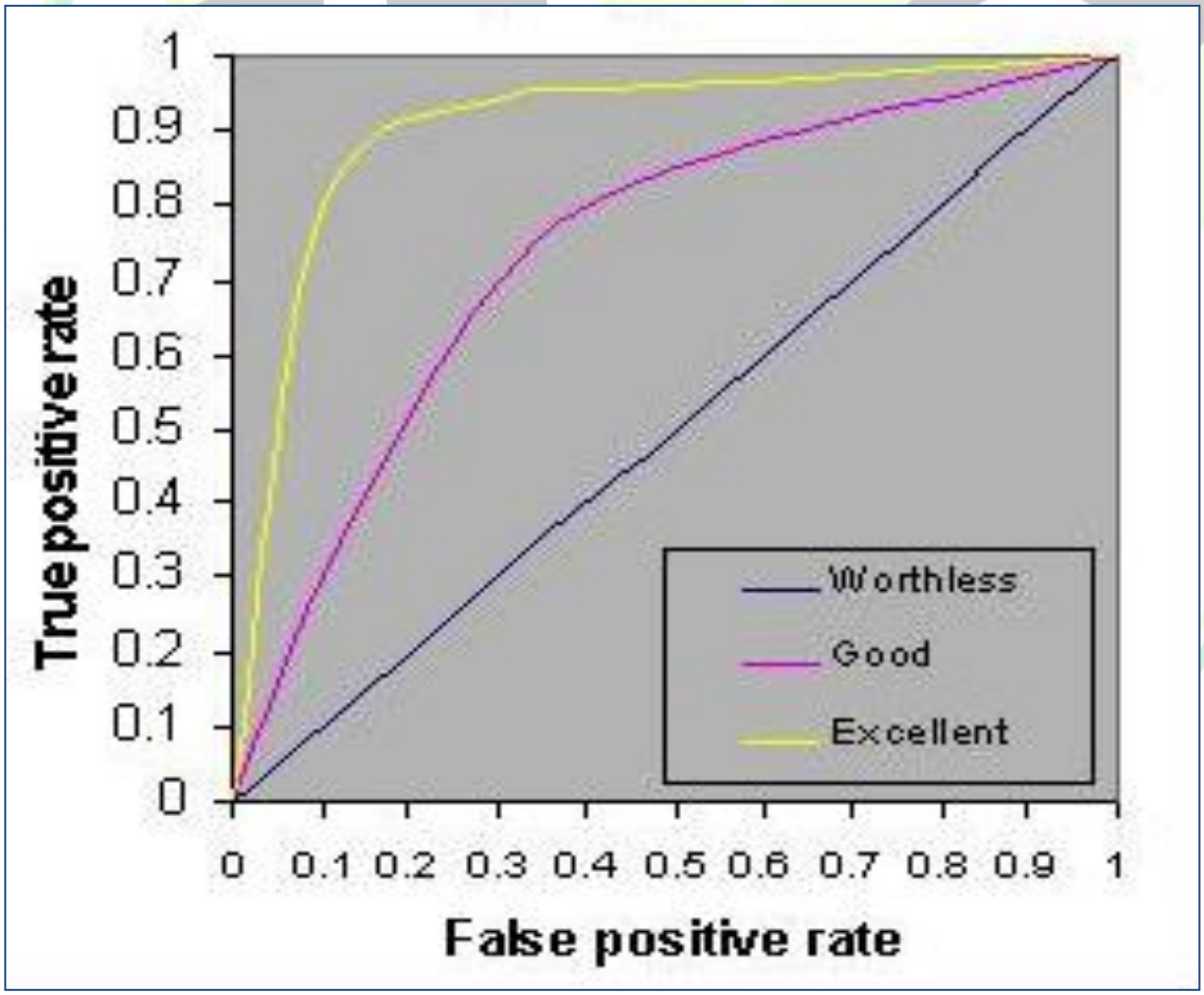
false negatives

the higher the one,

the lower the other

sign of bright future

Decision	Null Hypothesis	
	Correct	Wrong
Reject H_0	Type I error α	Correct $1 - \beta$
Accept H_0	Correct $1 - \alpha$	Type II error β



"ROC" (receiver-operator characteristic) curve

Multiplicity Issue

Gene	WT1	WT2	WT3	MU1	MU2	MU3	Unpaired	Paired
G1	2.2	2.3	1.8	20.9	21.2	20.7	0.00	0.00
G6	3.8	4.6	4.7	4.3	5.2	5.2	0.12	0.00
G4	34.3	34.3	33.6	2.0	2.8	1.3	0.00	0.00
G3	2.6	2.8	3.4	12.7	12.5	13.4	0.00	0.00
G23	19.5	21.5	16.7	33.0	34.6	29.6	0.00	0.00
G72	1.8	2.3	1.9	21.1	22.2	22.2	0.00	0.00
G9	6.7	9.4	12.1	58.3	59.5	65.6	0.00	0.00
G10	2.4	2.5	1.6	31.7	33.2	30.1	0.00	0.00

Multiplicity Issue

Multiple testing problem occurs when the chance of anyone of the results being called significant while in really it is not is small, but since many tests are performed altogether, guaranteed that many of the results will be incorrect (false positives)

Expression studies pose unique statistical challenges due to interrogation of so many gene simultaneously. This leads to extreme cases of what statisticians refer to as the *multiple testing problem*.

Example

At a = 0.05, 5 of every 100 results might be called significant by mistake (p<0.05)

How many tests will be wrong in an experiment with 20,000 tests

Bonferoni corrrection

Set α to desired α/number of tests

i.e 0.05/20,000 =

Flase Discovery Rate (FDR)

It is often OK to have *some* false positives, just not too many to control the *False Discovery Rate (FDR)*

Often, this is an acceptable trade-off as it allows you to find many more genes

(FDR q-value)

The expected proportion of false-positives among the positive results

i.e., at q=0.05, if you call 1000 genes significantly changed, up to 50 of those might be false positives, but the

rest will be true positives

Conclusions

Multiple testing problem occurs many tests are performed altogether

It is often OK to have *some* false positives, just not too many to control the *False Discovery Rate (FDR)*

References

- Trevino, V., Falciani, F., and Barrera-Saldana, H.A. 2007. DNA microarrays: a powerful genomic tool for biomedical and clinical research. Mol Med 13: 527-541.
- Kato, H., Saito, K., and Kimura, T. 2005. A perspective on DNA microarray technology in food and nutritional science. Curr Opin Clin Nutr Metab Care 8: 516-522.

Lesson-137

cBioPortal Database for Cancer Genomics

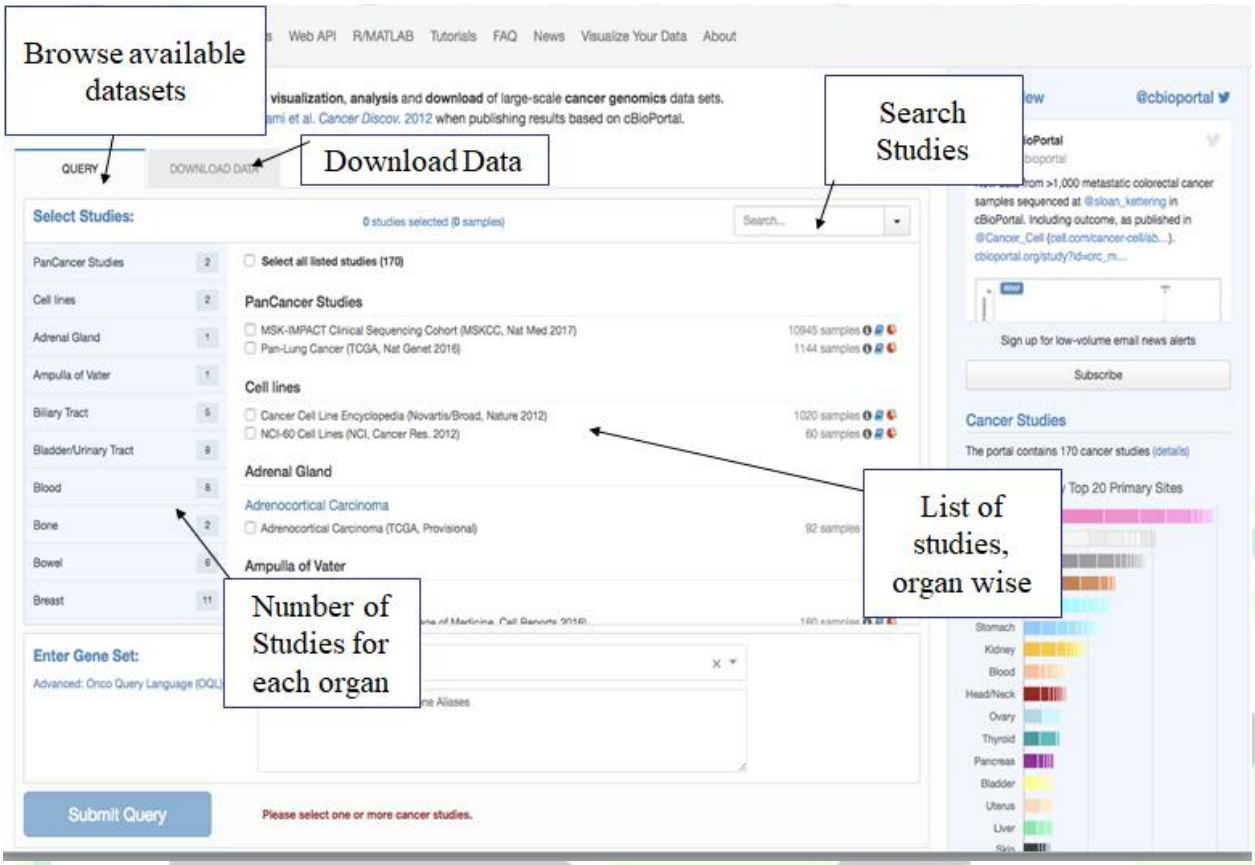
Introduction

- The cBioPortal for Cancer Genomics is an open-access, open-source resource for interactive exploration of multidimensional cancer genomics data sets.
- The cBioPortal significantly lowers the barriers between complex genomic data and cancer researchers
- Rapid, intuitive, and high-quality access to molecular profiles and clinical attributes from large-scale cancer genomics projects are always required for researches.

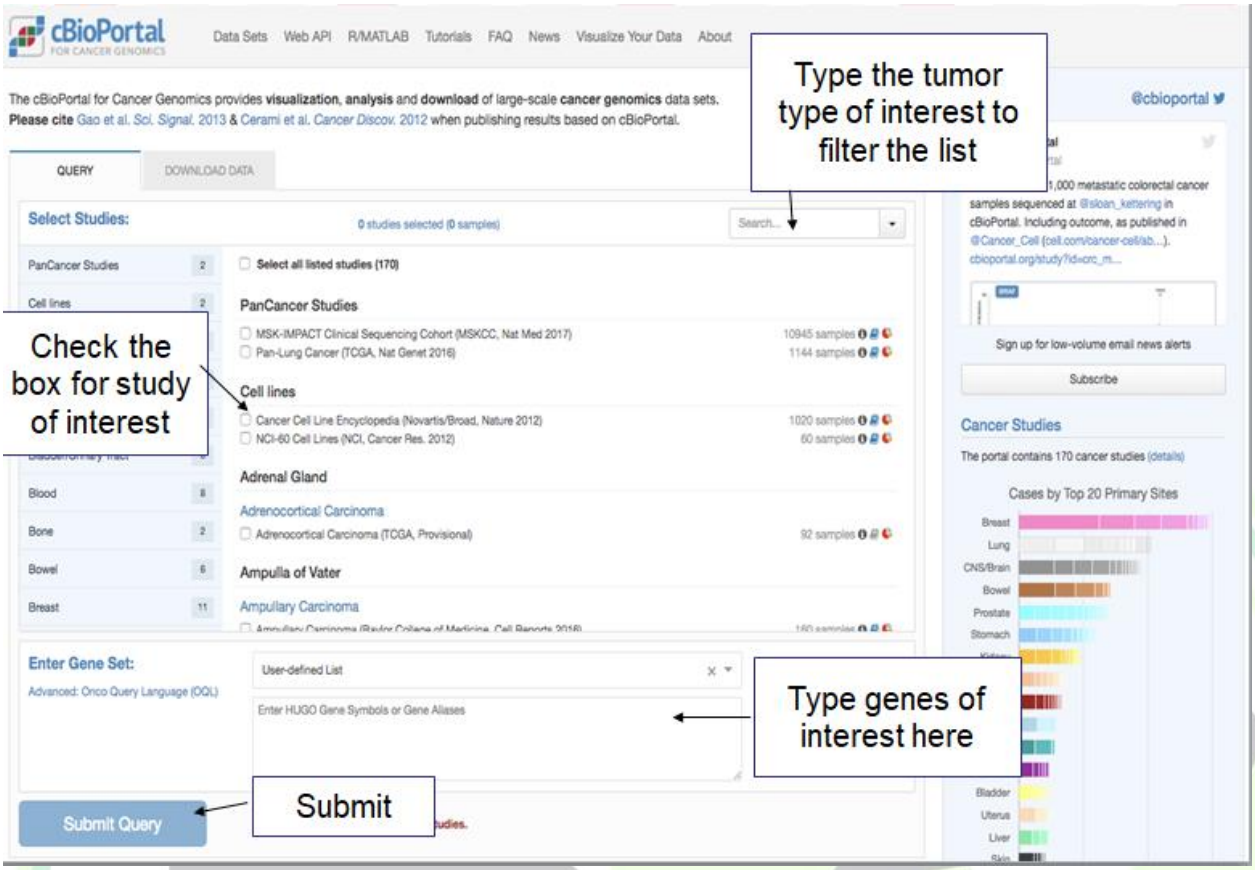
Data Types

- The portal stores
  - DNA copy-number data
  - mRNA and microRNA expression data,
  - non-synonymous mutations,
  - protein-level and phosphoprotein level (RPPA) data,
  - DNA methylation data, and
  - limited de-identified clinical data

Main Page:



Submitting a query

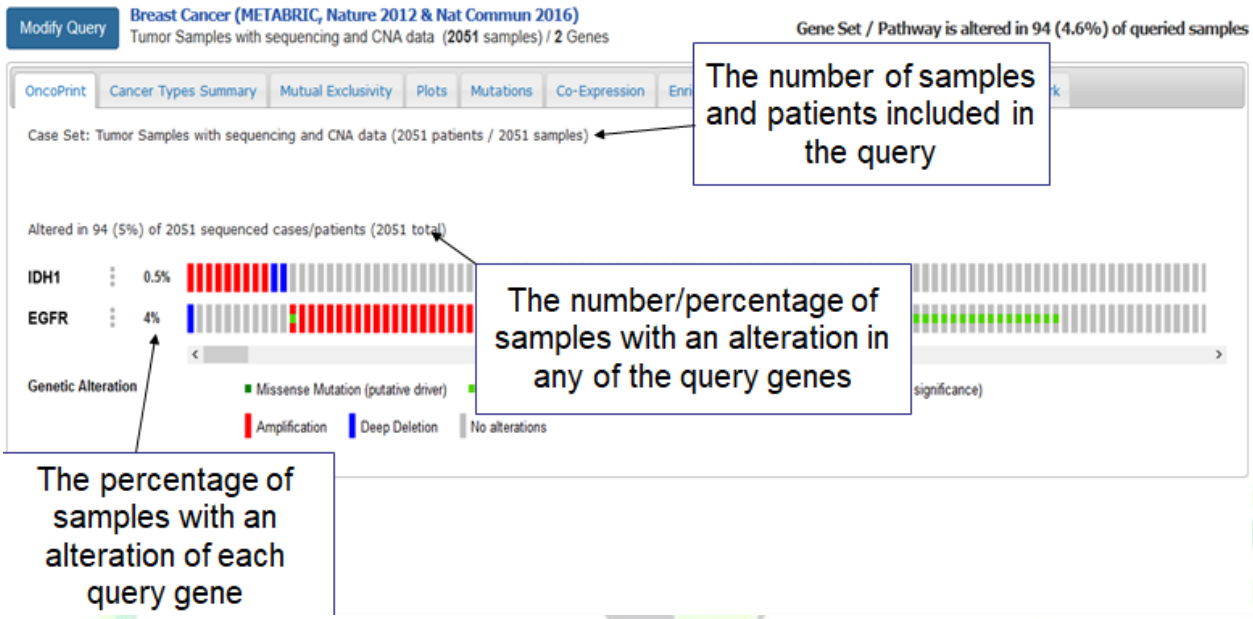


Querying multiple genes in a single dataset

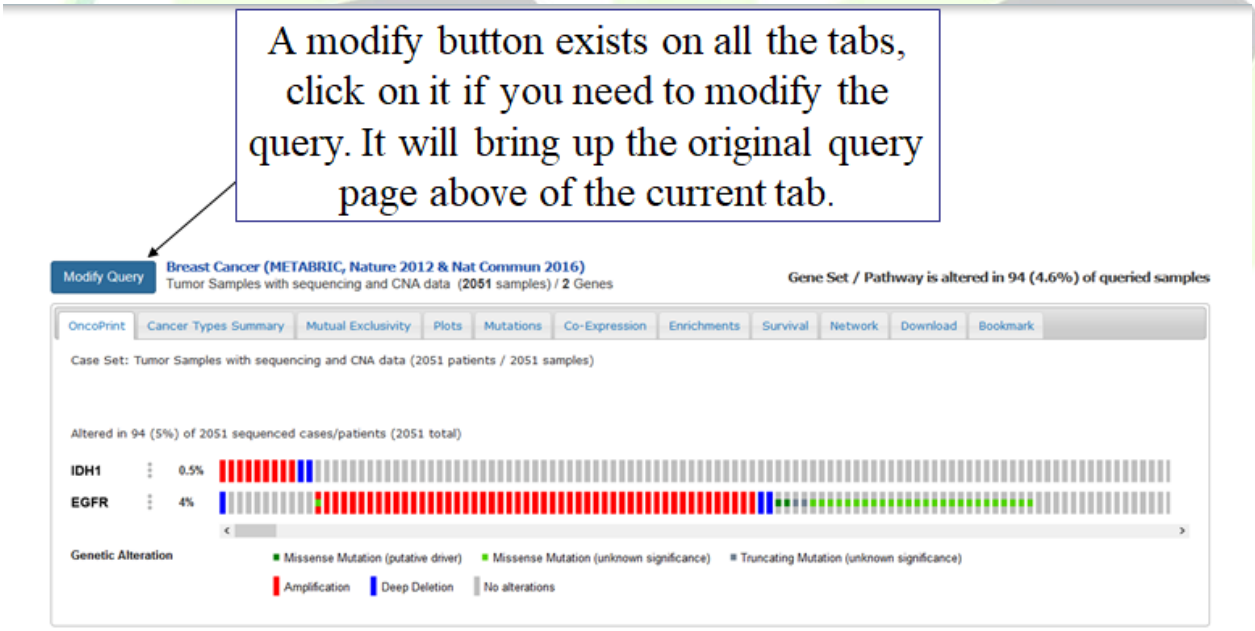
- Data/analysis tabs:
    - OncoPrint
    - Mutual Exclusivity
    - Plots
    - Mutations
    - Co-expression
    - Enrichments
    - Survival
    - Network
    - Download
    - Bookmarks

Performing the previous query will bring you to the OncoPrint Tab

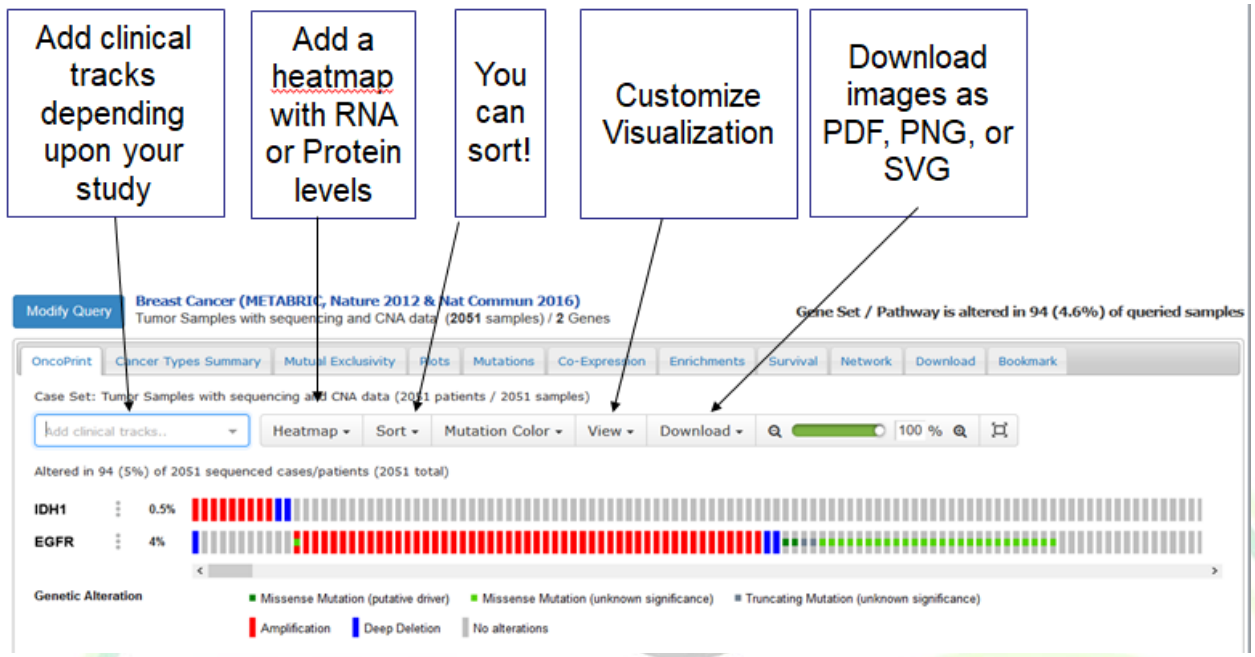
OncoPrint

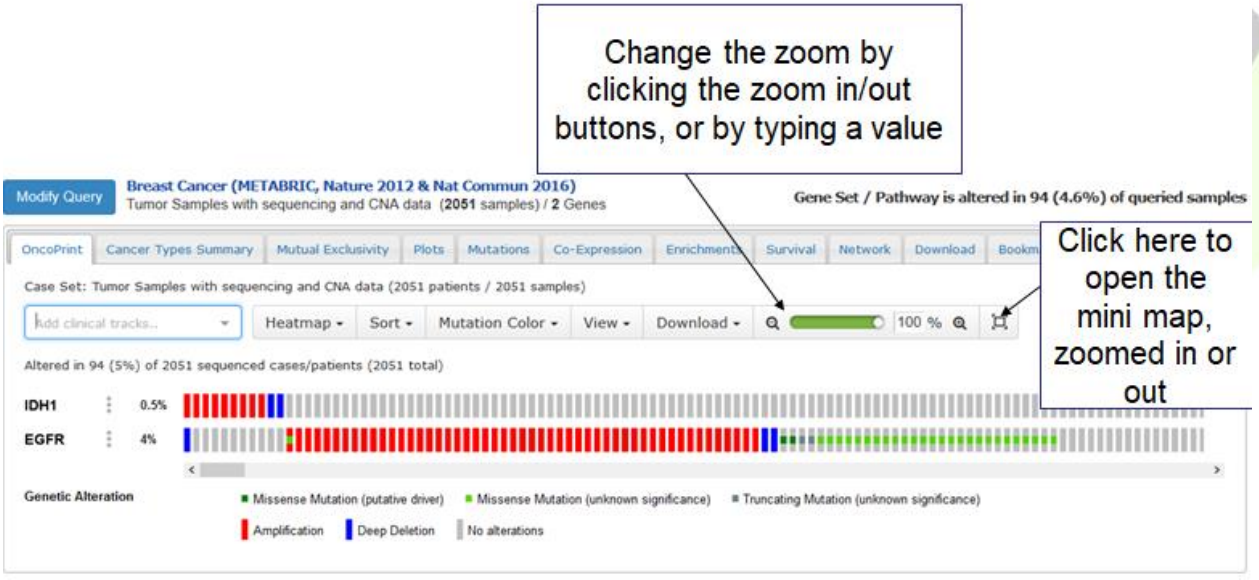


Modifying a query

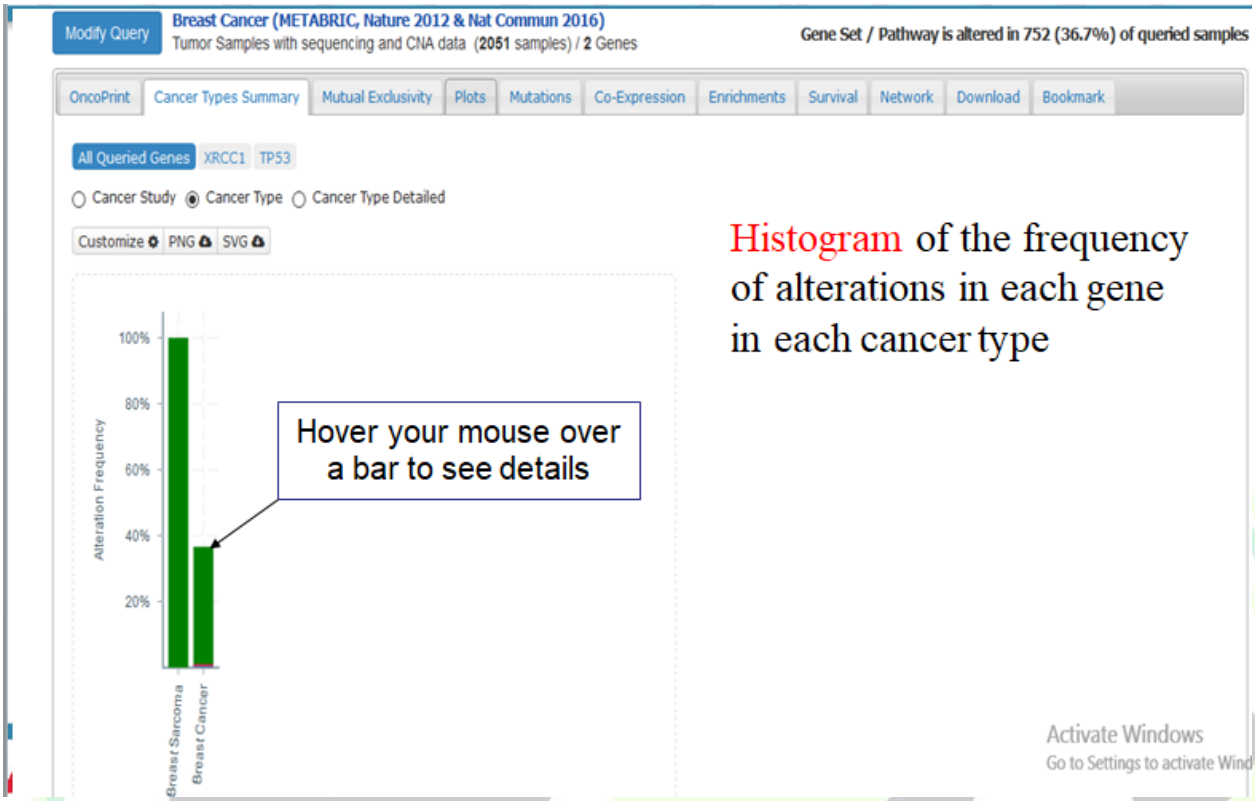


OncoPrint: Advanced Features



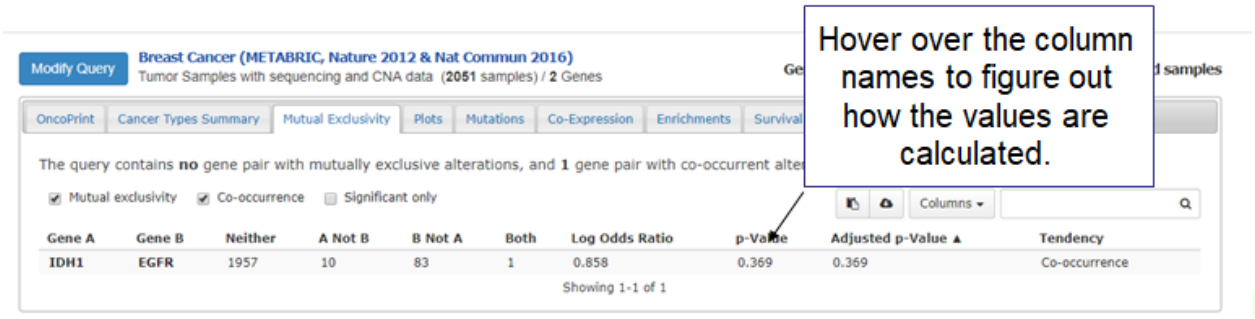


Cancer types Summary

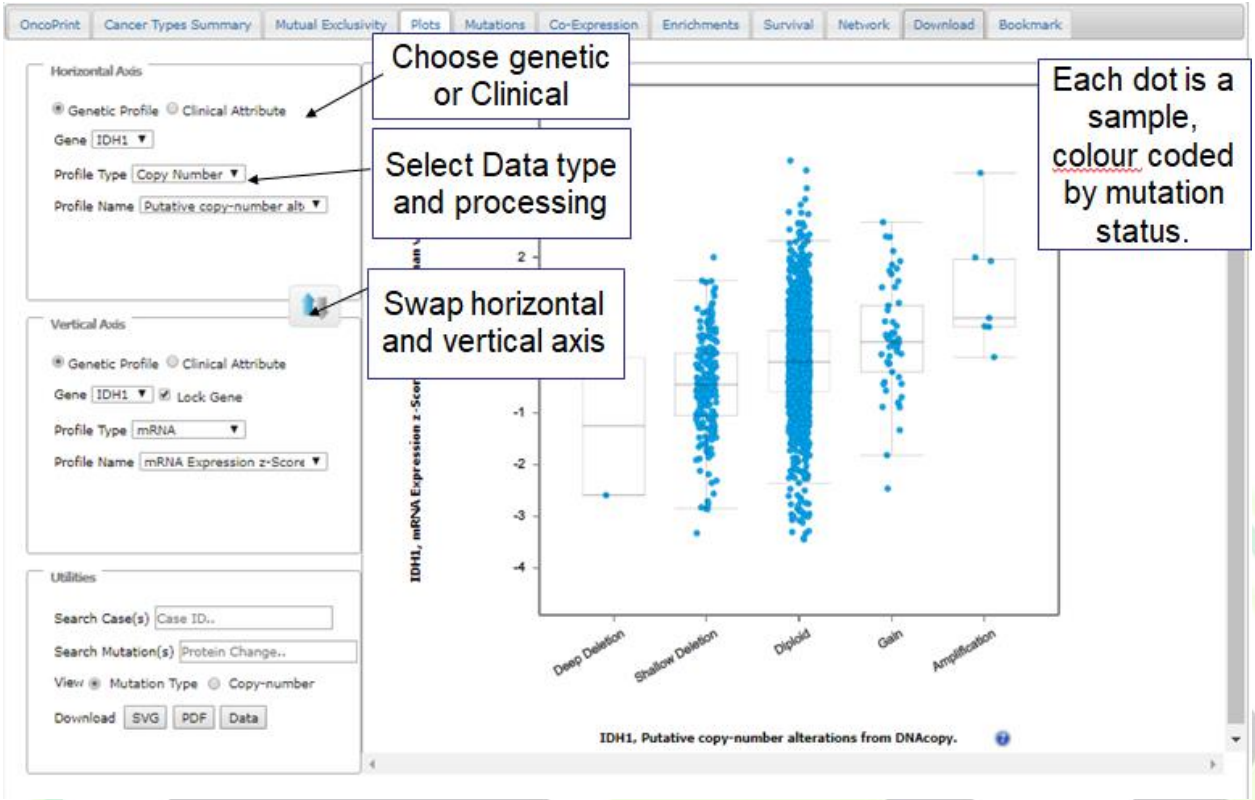


Mutual Exclusivity

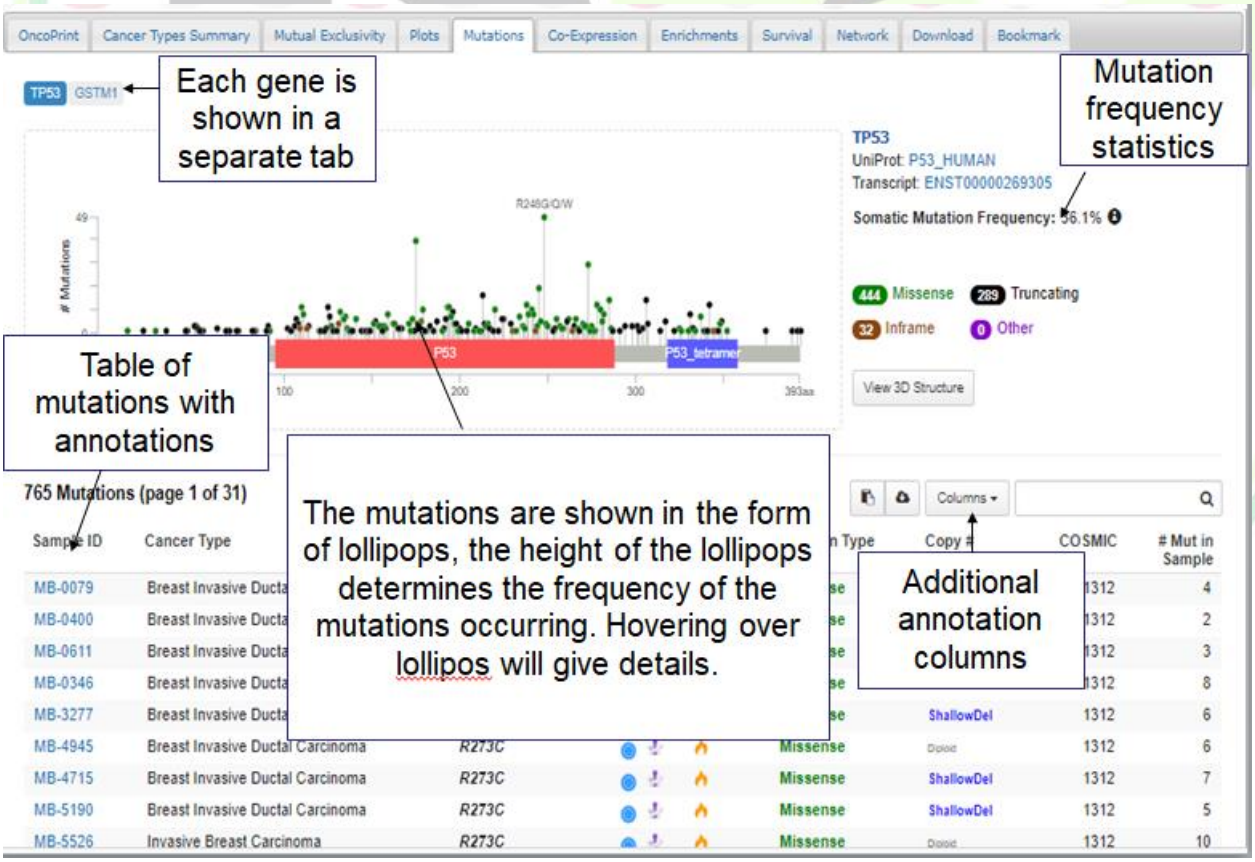
- The Mutual exclusivity tab provides a statistical analysis of the alterations depicted by the OncoPrint tab.
- All the pairwise combinations of query genes are analyzed for mutual exclusivity or co-occurrence.



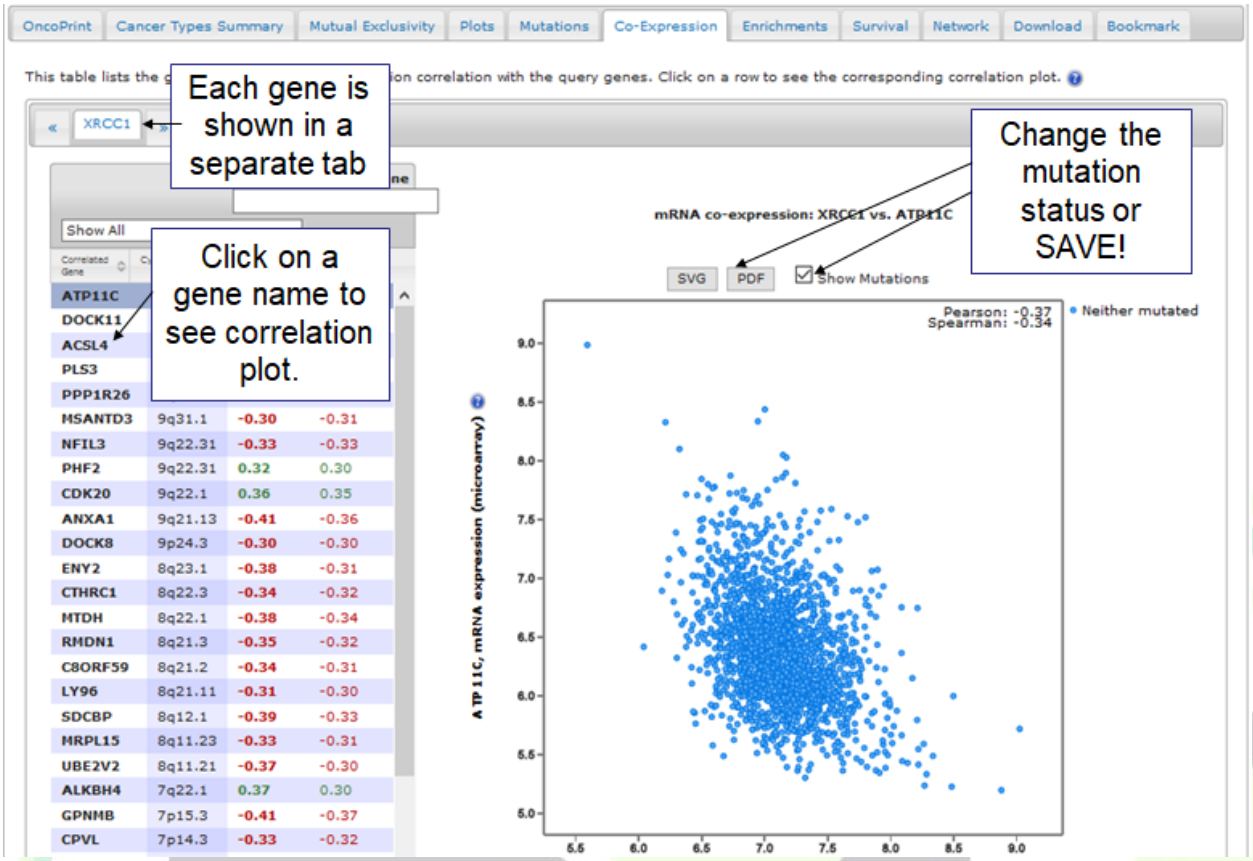
Plots



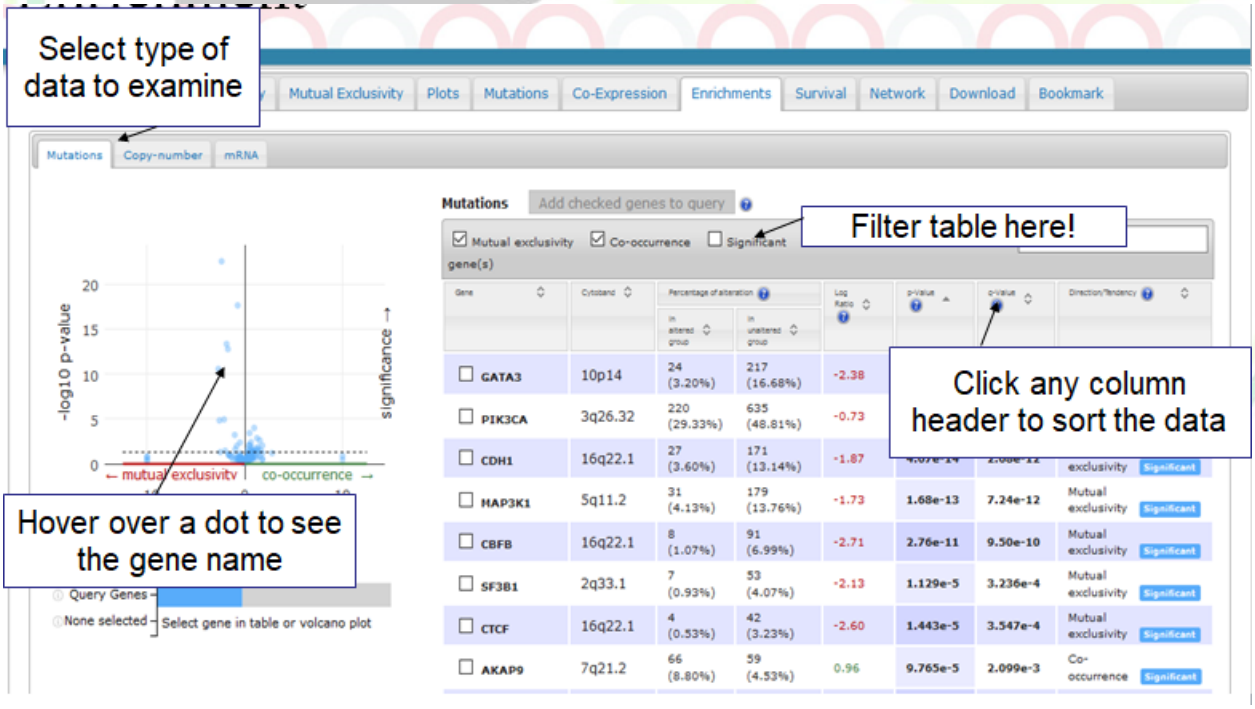
Mutations



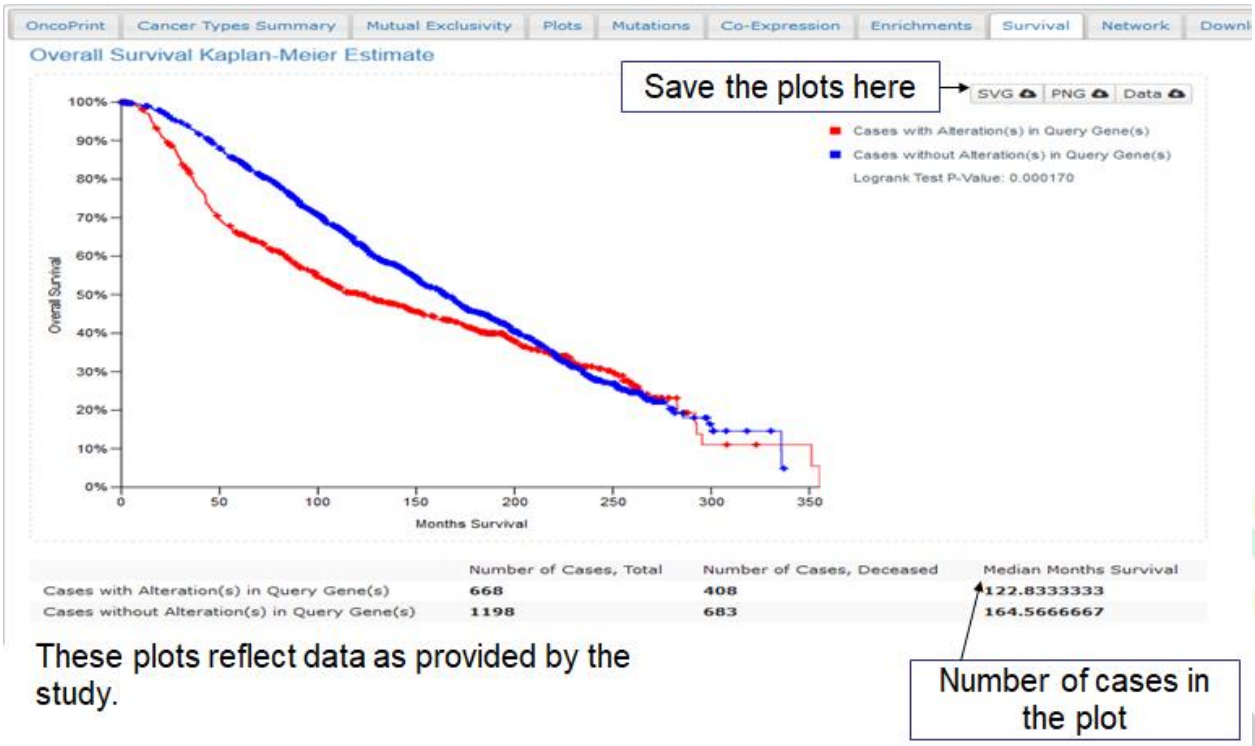
Co-Expression



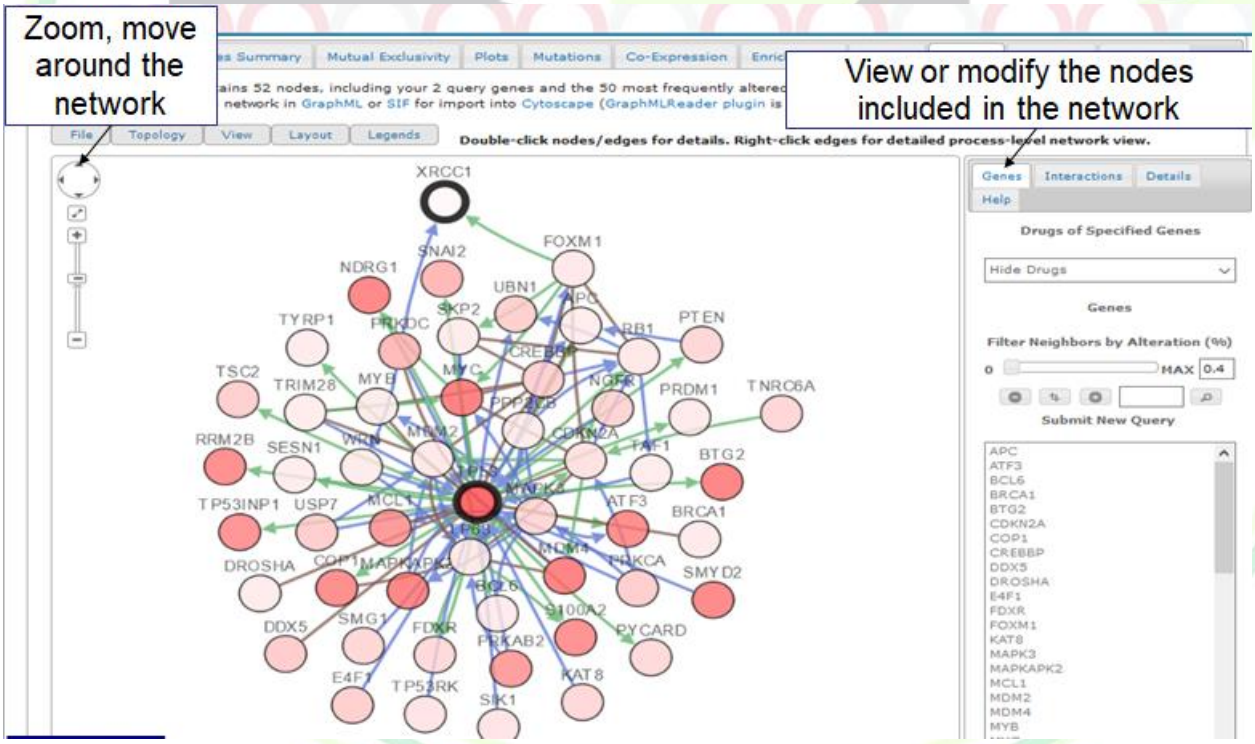
Enrichment

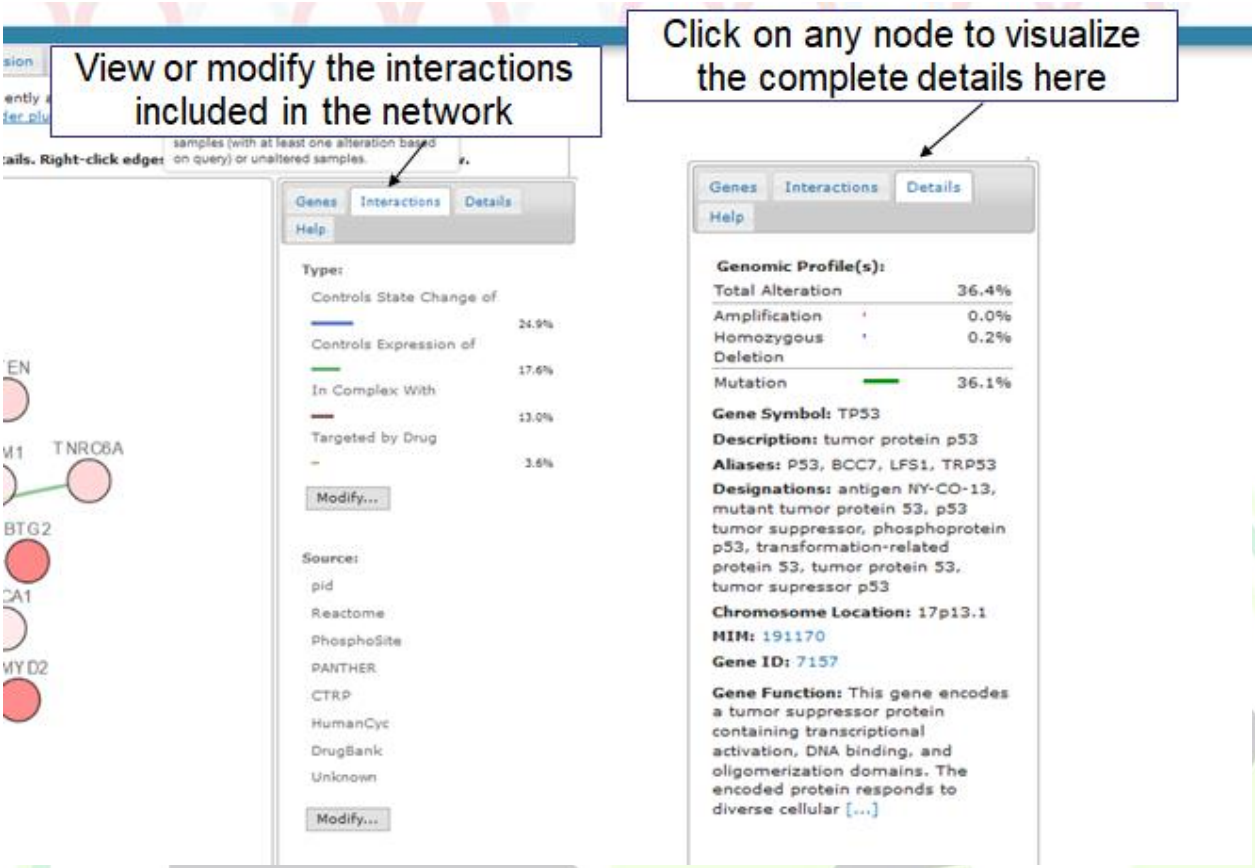


Survival

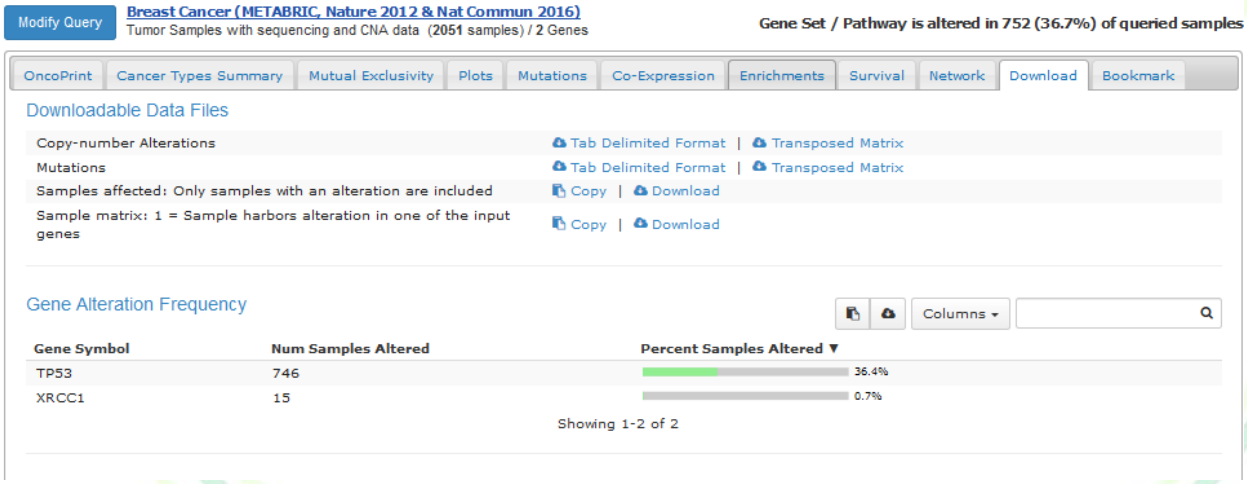

OncoPrint
Cancer Types Summary
Mutual Exclusivity
Plots
Mutations
Co-Expression
Enrichments
Survival
Network
Overall Survival Kaplan-Meier Estimate
Save the plots here
SVG
PNG
Data
Cases with Alteration(s) in Query Gene(s)
Cases without Alteration(s) in Query Gene(s)
Logrank Test P-Value: 0.000170
Overall Survival
Months Survival
Number of Cases, Total
Number of Cases, Deceased
Median Months Survival
Cases with Alteration(s) in Query Gene(s) 668 408 122.8333333
Cases without Alteration(s) in Query Gene(s) 1198 683 164.5666667
These plots reflect data as provided by the study.
Number of cases in the plot

Network


Zoom, move around the network
View or modify the nodes included in the network
File
Topology
View
Layout
Legends
Double-click nodes/edges for details. Right-click edges for detailed process-level network view.
Genes
Interactions
Details
Help
Drugs of Specified Genes
Hide Drugs
Genes
Filter Neighbors by Alteration (%)
MAX
Submit New Query
XRCC1
SNAI2
FOXM1
NDRG1
UBN1
SKP2
PRKDC
TYRP1
RB1
PTEN
TSC2
TRIM28
MYB
MYC
PRDM1
TNRC6A
RRM2B
SESN1
BTG2
TP53INP1
USP7
MCL1
ATF3
BRCA1
DROSHA
MDM4
SMYD2
DDX5
SMG1
FDXR
PYCARD
E4F1
TP53RK
KAT8
SIK1



Download



Bookmark

